CEH v13 AI (312-50)
Study Guide

Module 18: IoT and OT Hacking

EC-Council Certified Ethical Hacker v13 AI

Exam-focused study guide built from official Module 18 theory video screenshots

Table of Contents

1. Module at a Glance 6

2. Big Picture Map 8

3. Topic-by-Topic Guide 10

3.1 Learning Objectives of Module 18 10
3.2 What is the IoT? How IoT Works 11
3.3 IoT Architecture (5 Layers) 12
3.4 IoT Technologies and Protocols 12
3.5 OWASP Top 10 IoT Threats 13
3.6 OWASP IoT Attack Surface Areas 14
3.7 IoT Vulnerabilities (17 Named Items) 15
3.8 IoT Threats Overview (21 Named Attacks) 17
3.9 DDoS Attack (IoT Botnets) 18
3.10 Exploit HVAC 19
3.11 Rolling Code Attack 20
3.12 Jamming Attack 21
3.13 Hacking Smart Grid/Industrial Devices: Remote Access using Backdoor 22
3.14 SDR-Based Attacks on IoT 23
3.15 Identifying and Accessing Local IoT Devices (DNS Rebinding) 24
3.16 Fault Injection Attacks 25
3.17 Other IoT Attacks (Sybil, Exploit Kits, MITM, Replay, Forged Device, Side-Channel, Ransomware) 26
3.18 IoT Malware 27
3.19 Information Gathering using Shodan 28
3.20 Information Gathering through Sniffing 29
3.21 Vulnerability Scanning using IoTSeeker 30
3.22 Vulnerability Scanning using Genzai 31
3.23 Rolling Code Attack using RFCrack 31
3.24 Hacking Zigbee Devices with Open Sniffer 32
3.25 BlueBorne Attack Using HackRF One 33
3.26 Replay Attack using HackRF One 34
3.27 Identifying IoT Communication Buses and Interfaces 35
3.28 NAND Glitching 36
3.29 Exploiting Cameras using CamOver 36
3.30 Gaining Remote Access using Telnet 37
3.31 Maintain Access by Exploiting Firmware 38

3.32 Firmware Analysis and Reverse Engineering 39
3.33 IoT Hacking Tools (CatSniffer and Others) 40
3.34 How to Defend Against IoT Hacking 41
3.35 General Guidelines for IoT Device Manufacturers 42
3.36 OWASP Top 10 IoT Vulnerabilities Solutions 43
3.37 IoT Hardware Security Best Practices 44
3.38 IoT Device Management 45
3.39 What is OT? 46
3.40 IT/OT Convergence (IIoT) 47
3.41 The Purdue Model 48
3.42 OT Technologies and Protocols (by Purdue Level) 49
3.43 Challenges of OT 50
3.44 OT Vulnerabilities (10 Named Items) 51
3.45 MITRE ATT&CK for ICS (Part 1) 52
3.46 MITRE ATT&CK for ICS (Part 2) 53
3.47 OT Threats Overview (14 Named Threats) 54
3.48 HMI-based Attacks 55
3.49 Side-Channel Attacks (OT) 57
3.50 Hacking Programmable Logic Controller (PLC) 58
3.51 Evil PLC Attack 59
3.52 Hacking Industrial Systems through RF Remote Controllers: Replay and Command Injection 60
3.53 Hacking Industrial Systems through RF Remote Controllers: Re-pairing and Malicious Reprogramming 61
3.54 OT Supply Chain Attacks 62
3.55 OT Malware: Fuxnet 63
3.56 Information Gathering: Identifying ICS/SCADA Systems using Shodan 64
3.57 Information Gathering: Gathering Default Passwords using CIRT.net 65
3.58 Information Gathering: Scanning ICS/SCADA Systems using Nmap 65
3.59 Information Gathering: Analyzing Modbus/TCP Traffic Using Wireshark 67
3.60 Information Gathering: Discovering ICS/SCADA Network Protocols using Malcolm 67
3.61 Vulnerability Scanning Using Nessus 68
3.62 Fuzzing ICS Protocols 69
3.63 Vulnerability Scanning using Skybox 70
3.64 Hacking ICS Hardware 71
3.65 Hacking Modbus Slaves using Metasploit 72
3.66 Hacking PLC using modbus-cli 73
3.67 Gaining Remote Access using DNP3 74
3.68 How to Defend Against OT Hacking 75
3.69 OT Vulnerabilities and Solutions 76

3.70 How to Secure an IT/OT Environment (Security Controls based on Purdue Model) 77
3.71 Implementing a Zero-Trust Model for ICS/SCADA 78
3.72 International OT Security Organizations and Module Summary 79

4. Tool Encyclopedia 80

4.1 IoT Information Gathering Tools 81
4.2 IoT Vulnerability Scanning Tools 82
4.3 IoT Gaining/Maintaining Access Tools 82
4.4 IoT Device Management (Defender) Platforms 83
4.5 OT/ICS Information Gathering Tools 84
4.6 OT/ICS Vulnerability Scanning Tools 85
4.7 OT/ICS Exploitation and Hardware Tools 86

5. Command Sheet (I have no lab, so teach me to read commands) 87

5.1 Nmap (Scanning IoT Insecure HTTP Ports) 87
5.2 ifconfig, Airmon-ng, and Airodump-ng (Wireless Recon Setup) 88
5.3 Wireshark (Capturing and Reading MQTT IoT Traffic) 89
5.4 IoTSeeker (Default Credential Scanner) 90
5.5 RFCrack (SDR Rolling-Code, Replay, and Jamming Attacks) 91
5.6 HackRF One (hackrf_transfer and hackrf_info) 92
5.7 BUS Auditor (UART and I2C Pin Scanning) 93
5.8 CamOver (Camera Admin Password Disclosure) 94
5.9 Firmware Mod Kit (extract-firmware.sh) 96
5.10 Firmware Analysis Commands (file, strings, hexdump, binwalk, dd, mount, grep, find, qemu) 97
5.11 Fuxnet Deployment Pattern (xargs -P) 98
5.12 Shodan Search Filters (ICS/SCADA Recon) 100
5.13 Nmap ICS/SCADA Protocol Scripts 101
5.14 Wireshark (Analyzing Modbus/TCP Traffic) 102
5.15 Nessus (ICS/SCADA Vulnerability Scan Setup) 103
5.16 Fuzzowski (Fuzzing ICS Protocols) 104
5.17 Metasploit (modbus_findunitid and modbusclient) 105
5.18 modbus-cli (Reading and Writing PLC Registers/Coils) 107

6. Ports, Protocols, Standards and Acronyms 109

6.1 Ports named or shown in this module 109
6.2 Protocols and standards 110
6.3 Key acronyms used in this module 112

7. Attack vs Countermeasure and Look-alike Comparisons 114

7.1 Attack vs countermeasure 114
7.2 Look-alike comparisons 116

8. Exam Cheat Sheet 117
8.1 Keyword -> answer pairs 117
8.2 Must-memorize lists 119
8.3 Numbers and defaults to remember 120
8.4 "Pick the BEST option" rules for this module 121
9. Practice Questions 122
10. Flashcards 133
11. Glossary 135
12. Coverage Report 139
12.1 Page-to-section map 139
12.2 [UNCLEAR] items 140
12.3 Items to double check (marked "verify" in this guide) 141
12.4 Coverage confirmation 141

Source: 39 pages of screenshots from the official Module 18 theory videos (slides 1 to 87+). Content marked "ADDED CONTEXT" comes from outside the screenshots. Items marked "verify" should be checked against current official sources. Items marked [UNCLEAR on page N] were too small, low-resolution, or blurry in the source screenshot to read with full confidence; per the source material's own note, this mainly affects embedded terminal, browser, and console screenshots on pages 2, 6, 10, 11, 12, 13, 14, 15, 16, 17, 18, 20, 22, 26, 27, 29, 30, 31, 32, 33, 34, 35, 36, and 39.

1. Module at a Glance

Module: 18 - IoT and OT Hacking (EC-Council CEH v13 AI).

What this module is (5 lines):

1. It opens with IoT (Internet of Things) fundamentals: what IoT is, its 5-layer architecture, and the short-range, medium-range, and long-range wireless technologies, operating systems, and application protocols that make IoT devices talk to each other.
2. It lists OWASP's risk and attack-surface frameworks for IoT, then walks through 17 named IoT vulnerabilities and 21 named IoT attacks (DDoS, HVAC exploitation, rolling code, jamming, backdoors, SDR attacks, DNS rebinding, fault injection, and more), plus real IoT malware families.
3. It gives a full IoT hacking methodology with named tools for each step: information gathering (Shodan, sniffing), vulnerability scanning (IoTSeeker, Genzai), gaining access (RFCrack, HackRF One, Open Sniffer, CamOver, bus auditing, NAND glitching), and maintaining access (firmware extraction and reverse engineering).
4. It then switches to OT (Operational Technology): ICS/SCADA concepts, the Purdue Model, OT-specific protocols, OT threats and vulnerabilities, MITRE ATT&CK for ICS, and named OT attacks (HMI-based attacks, side-channel attacks, PLC rootkits, the Evil PLC attack, RF remote-controller attacks, supply chain attacks, and OT malware such as Fuxnet).
5. It closes with a full OT hacking methodology (Shodan/CIRT.net/Nmap/ Wireshark/ Malcolm recon, Nessus/Skybox/Fuzzowski vulnerability work, Metasploit/modbus-cli/ DNP3 exploitation) and countermeasures for both IoT and OT, including the Purdue-Model security-control table and a Zero Trust model for ICS/SCADA.

Why it matters: IoT devices (cameras, routers, smart locks, smart TVs) and OT systems (power grids, water treatment, manufacturing lines) are now everywhere, and both were built for function first, security second. Unlike a laptop, many of these devices cannot run antivirus, cannot easily be patched, and often sit on networks without a firewall between them and the internet. CEH treats IoT and OT as two

related but distinct attack surfaces, each with its own recon tools, its own named attacks, and its own protocols that rarely existed in earlier, IT-only modules (Modbus, BACnet, Zigbee, DNP3, S7).

What the exam usually asks from it:

- Match a named IoT or OT attack to its one-line description: rolling code attack, jamming attack, DNS rebinding, fault injection, Sybil attack, side-channel attack, HMI-based attack, PLC rootkit attack, Evil PLC attack, RF remote-controller attacks, OT supply chain attacks.
- Match a tool to its job and its domain (IoT vs. OT): Shodan, IoTSeeker, Genzai, RFCrack, HackRF One, Open Sniffer, CamOver, Firmware Mod Kit (IoT); Nessus, Skybox, Malcolm, Fuzzowski, Metasploit modbus modules, modbus-cli (OT).
- Read an Nmap command and identify which ICS/SCADA protocol or vendor it targets (Modbus, S7, BACnet, EtherNet/IP, Niagara Fox, ProConOS, Omron, PCWorx).
- Identify the 5 layers of IoT architecture and the Purdue Model's levels and zones for OT.
- Tell apart look-alike attacks: rolling code vs. replay; side-channel (shared hardware) vs. jamming (RF interference); Evil PLC attack vs. PLC rootkit attack; HMI attack sub-types (memory corruption, credential management, lack of authentication, code injection).
- OWASP's IoT Top 10 and its matching solutions; OWASP's IoT attack surface areas.
- Default/weak credential attacks against Telnet (port 23) and DNP3 on exposed ICS/IoT devices found via Shodan.
- Countermeasures for IoT devices and for OT networks, including the Purdue-Model-based security control table.

v13 AI note: this module's own screenshots do not name any AI-specific tool (no ChatGPT, WormGPT, or LLM content appears in the source material). Any AI-related content in this guide is explicitly marked ADDED CONTEXT and reflects how AI realistically intersects with IoT/OT security (AI-assisted phishing used to deliver IoT/OT backdoors, AI-assisted firmware/malware analysis, and AI-driven anomaly detection in commercial OT security tools), not a claim that the screenshots covered it.

The 10 things to remember most:

1. IoT architecture has 5 layers: Edge Technology -> Access Gateway -> Internet -> Middleware -> Application (bottom to top).
2. A rolling code (hopping code) attack needs a jammer plus a sniffer: jam the first unlock code, capture the second, replay it later.
3. IMDS-style default-credential exposure happens in both domains: Telnet (port 23) for IoT devices and DNP3 for ICS, both commonly found wide open on Shodan.
4. The Evil PLC attack turns a FOUND vulnerable PLC into a weapon against the engineering WORKSTATIONS that connect to it; the PLC rootkit attack instead directly manipulates the PLC's own I/O and physical-process logic.

5. Modbus/TCP (port 502) has no built-in authentication or encryption, which is why Metasploit's `modbus_findunitid`/`modbusclient` and the `modbus-cli` gem can read and write live PLC registers/coils with no login.
6. The Purdue Model has 6 levels (0-5) in two zones (Manufacturing/OT, Enterprise/IT) separated by an Industrial DMZ (IDMZ).
7. MITRE ATT&CK for ICS organizes attacker techniques into tactics: Initial Access, Execution, Persistence, Privilege Escalation, Evasion, Discovery, Lateral Movement, Collection, Command and Control, Inhibit Response Function, Impair Process Control, and Impact.
8. Side-channel attacks appear in both domains: IoT side-channel attacks leak encryption keys through emitted signals; OT side-channel attacks use timing analysis or power analysis to guess a password character by character.
9. Firmware analysis follows a fixed order: obtain -> analyze (file, md5sum, strings, hexdump) -> extract filesystem (binwalk, dd) -> mount -> analyze filesystem -> emulate (QEMU).
10. OWASP publishes three separate IoT lists used throughout this module: the IoT Top 10 (threats/vulnerabilities), the IoT Attack Surface Areas, and matching Top 10 Solutions.

2. Big Picture Map

The module has two parallel halves that share one shape: concepts and architecture first, then threats/vulnerabilities, then a hacking methodology (info gathering -> vulnerability scanning -> gaining access -> maintaining access), then countermeasures.

Diagram part 1: the IoT half.

```
IoT CONCEPTS
5-layer architecture | wireless tech | protocols
               |
               v
IoT THREATS AND VULNERABILITIES
OWASP Top 10 | Attack Surface Areas
17 vulnerabilities | 21 named attacks | malware
               |
               v
IoT HACKING METHODOLOGY
Info Gathering (Shodan, sniffing)
Vuln Scanning (IoTSeeker, Genzai)
Gaining Access (RFCrack, HackRF, Open Sniffer,
CamOver, bus auditing, NAND glitching)
Maintaining Access (firmware extraction/reverse
engineering)
               |
               v
IoT COUNTERMEASURES
Device hardening | manufacturer guidelines
OWASP solutions | hardware best practices
```

Diagram part 2: the OT half, same shape, different tools.

```
OT CONCEPTS
ICS/SCADA/PLC/RTU/DCS/HMI | IT-OT convergence
Purdue Model (6 levels, 2 zones, 1 DMZ)
OT-specific protocols by Purdue level
               |
               v
OT THREATS AND VULNERABILITIES
Challenges of OT | 10 vulnerabilities
MITRE ATT&CK for ICS | 14 named threats
HMI attacks | side-channel | PLC/Evil PLC
RF controller attacks | supply chain | malware
               |
               v
OT HACKING METHODOLOGY
Info Gathering (Shodan, CIRT.net, Nmap,
Wireshark, Malcolm)
Vuln Scanning (Nessus, Skybox, Fuzzowski)
Gaining Access (hardware tools, Metasploit,
modbus-cli, DNP3)
               |
               v
OT COUNTERMEASURES
Defense checklist | vulnerability solutions
Purdue security controls | Zero Trust | bodies
```

How each layer maps to its tools and defenses:

Layer	Attacker tools/techniques	Defender tools/guidance
IoT recon	Shodan, Censys, FOFA, Nmap, Wireshark	Reduce exposure, monitor Shodan listings
IoT vuln scan	IoTSeeker, Genzai, beSTORM, IoTSploit	IoTVAS, patch firmware, change defaults
IoT gain access	RFCrack, HackRF One, Open Sniffer, CamOver	Disable Telnet/UPnP, strong auth, lockout
IoT maintain access	Firmware Mod Kit, binwalk, QEMU	Firmware signing, secure boot, encryption
OT recon	Shodan, CIRT.net, Kamerka-GUI, ZoomEye	Remove from internet, enterprise firewall
OT vuln scan	Nessus, Skybox, Fuzzowski	Scan and patch regularly, segment network
OT exploit	Metasploit modbus modules, modbus-cli, DNP3	MFA, VPN, RBAC, disable unused services
OT hardware	GDB, OpenOCD, Binwalk, Ghidra, JTAG rigs	Physical hardening, tamper protection
Program-wide	MITRE ATT&CK for ICS (attacker playbook)	Purdue security controls, Zero Trust, IDS

3. Topic-by-Topic Guide

3.1 Learning Objectives of Module 18

What it is: The module's own roadmap, in two groups of 3: IoT concepts/attacks, IoT hacking methodology, IoT countermeasures; then the same 3-part pattern repeated for OT.

How it works (6 objectives):

1. Explain IoT concepts and attacks.
2. Explain IoT hacking methodology.
3. Explain IoT attack countermeasures.
4. Explain OT concepts and attacks.
5. Explain OT hacking methodology.
6. Explain OT attack countermeasures.

EXAM FOCUS: This 3-plus-3 pattern (concepts/attacks -> methodology -> countermeasures) is the skeleton CEH uses for almost every hacking module; recognizing it helps you predict what section a fact belongs to even mid-exam.

Memory trick: "Know it, break it, fix it" - twice, once for IoT and once for OT.

3.2 What is the IoT? How IoT Works

What it is: The Internet of Things (IoT), also called the Internet of Everything (IoE), is a network of devices that each have an IP address and can sense, collect, and send data using embedded sensors, communication hardware, and processors. In IoT language, a "thing" is any device implanted on a natural, human-made, or machine-made object that can communicate over a network.

Analogy: A traditional appliance is deaf and mute: a thermostat just reacts to a dial. An IoT thermostat is the same box with ears, a mouth, and a phone line: it can report the room temperature and receive a new target temperature from your phone, anywhere in the world.

How it works (data flow, 5 steps):

1. IoT devices (industrial devices, wearables, healthcare devices, home devices) sense and collect data.
2. Devices send data to an IoT Gateway, which can also receive commands from the internet and forward them back to devices.
3. The gateway sends data onward over the internet to a server or cloud storage.
4. The server/cloud performs analysis on the stored data.
5. A mobile app lets the user send remote-control commands back down through the same chain to the device.

Real-world use: A hospital's remote patient monitor (a healthcare IoT device) continuously streams heart-rate data through a gateway to a cloud server; a doctor's mobile app pulls the analyzed data and can also send a command back to adjust an infusion pump's settings remotely.

Tools and commands used here: none (concept-only slide); IoT hacking tools are introduced from section 3.19 onward.

Countermeasures and defense: treat every "thing" as a tiny computer with its own attack surface; apply the same discipline (patching, strong credentials, network segmentation) used for full-size servers, covered fully in sections 3.34-3.38.

EXAM FOCUS: The exam likes testing the exact definition: IoT devices are defined by having an IP address plus sense/collect/send capability, not just by being "smart" or "wireless."

Memory trick: "Sense it, send it, store it, see it, steer it" - the 5-step IoT data loop.

3.3 IoT Architecture (5 Layers)

What it is: The standard 5-layer stack that describes how data moves from a physical sensor up to a user-facing application.

Analogy: Like a postal system: a letter (sensor data) starts at a mailbox (edge technology), goes through a local sorting office (access gateway), travels across the country (internet layer), gets sorted again at a regional hub (middleware), and finally lands on the recipient's desk in a format they can read (application layer).

How it works (5 layers, bottom to top):

1. **Edge Technology Layer:** sensors, devices, machines, and intelligent edge nodes of various types; this is where physical data is first captured.
2. **Access Gateway Layer:** handles protocol translation and messaging, converting sensor-native protocols into something the rest of the network understands.
3. **Internet Layer:** provides the connection between endpoints across the wider network.
4. **Middleware Layer:** handles device management and information management.
5. **Application Layer:** delivers the finished application to different categories of users.

Real-world use: A smart factory's temperature sensor (edge layer) talks Zigbee to a local gateway (access gateway layer) that translates Zigbee into IP packets sent over the plant's internet connection (internet layer); a middleware service logs and manages thousands of such devices (middleware layer); a manager's dashboard app shows the final chart (application layer).

Tools and commands used here: none (architecture concept).

Countermeasures and defense: apply layer-appropriate controls: physical tamper protection at the edge layer, protocol validation at the gateway layer, encryption at the internet layer, access control at the middleware layer, and authentication at the application layer.

EXAM FOCUS: A question may describe a function ("translates protocols and handles messaging") and ask which layer that is - match it to Access Gateway, not Middleware (which is about device/ information MANAGEMENT, not translation).

Memory trick: "EAIMA: Edge, Access gateway, Internet, Middleware, Application" - read bottom to top like climbing a ladder.

3.4 IoT Technologies and Protocols

What it is: The full menu of wireless/wired communication technologies, IoT operating systems, and IoT application protocols that the exam expects you to recognize by category.

How it works (5 categories):

1. **Short-range wireless:** Bluetooth Low Energy (BLE), Light-Fidelity (Li-Fi), Near Field Communication (NFC), QR codes and barcodes, Radio Frequency Identification (RFID), Wi-Fi, Wi-Fi Direct, Z-Wave, ZigBee, ANT.
2. **Medium-range wireless:** HaLow, LTE-Advanced, 6LoWPAN, QUIC, Neul.
3. **Long-range wireless and wired:** Low-Power Wide-Area Networking (LPWAN), LoRaWAN, Sigfox, Very Small Aperture Terminal (VSAT), Thread, cellular, Ethernet, Multimedia over Coax Alliance (MoCA), NB-IoT, Power-Line Communication (PLC).
4. **IoT operating systems:** Windows 10 IoT, Amazon FreeRTOS, Fuchsia, RIOT, Ubuntu Core, ARM Mbed OS, Zephyr, Embedded Linux, NuttX RTOS, Integrity RTOS, Tizen.
5. **IoT application protocols:** CoAP, Edge, LWM2M, Physical Web, XMPP, Mihini/M3DA, MQTT.

Real-world use: A smart agriculture sensor in a remote field uses LoRaWAN (long-range, low-power) to send soil-moisture readings for kilometers back to a single gateway, while a smartwatch on your wrist uses BLE (short-range) to sync with your phone a few feet away; both are "IoT," but the right wireless technology depends entirely on the distance and power budget needed.

Tools and commands used here: none directly (concept list); MQTT traffic is captured live with Wireshark in section 3.20.

AI angle: ADDED CONTEXT: AI-based network traffic classifiers in modern IoT security platforms (such as Microsoft Defender for IoT, covered in section 3.63) are trained to automatically recognize which of these protocols (Modbus, BACnet, Zigbee, MQTT, and more) is present on a network segment, without a human manually matching port numbers.

Countermeasures and defense: know which protocols on your network have no built-in encryption (MQTT, CoAP, and most of the short-range list by default) and add a security layer (TLS, DTLS, VPN tunnel) where the protocol itself does not provide one.

EXAM FOCUS: The exam frequently asks "which category does X belong to" (is NFC short-range or long-range? is Sigfox for IoT device operating systems or long-range wireless?). Memorize the category by the device's realistic range, not just the name.

Memory trick: "BLE and NFC barely reach your pocket; LoRaWAN and Sigfox reach the next town; everything else is somewhere in between."

3.5 OWASP Top 10 IoT Threats

What it is: OWASP's (Open Worldwide Application Security Project) standard list of the 10 most common IoT security threats, used as a baseline checklist across the whole IoT industry.

How it works (the 10 threats):

1. Weak, Guessable, or Hardcoded Passwords.
2. Insecure Network Services.
3. Insecure Ecosystem Interfaces.
4. Lack of Secure Update Mechanisms.
5. Use of Insecure or Outdated Components.
6. Insufficient Privacy Protection.
7. Insecure Data Transfer and Storage.
8. Lack of Device Management.
9. Insecure Default Settings.
10. Lack of Physical Hardening.

Real-world use: A home router shipped with the hardcoded password "admin/admin" (threat 1) is found by an attacker's Shodan search within minutes of being plugged in, because the manufacturer never forced a password change on first boot (threat 9, insecure default settings).

Tools and commands used here: referenced throughout sections 3.6- 3.38; matching solutions for each threat are in section 3.36.

Countermeasures and defense: see section 3.36 (OWASP Top 10 IoT Vulnerabilities Solutions) for the matched fix to each numbered threat.

EXAM FOCUS: The exam may give you a scenario and ask "which OWASP IoT Top 10 item does this match?" Learn the 10 labels by number; many questions test recall of the exact list order and wording.

Memory trick: "WIILUPIDL" is unpronounceable, so instead group them in pairs: Passwords+Network (1-2), Interfaces+Updates (3-4), Components+Privacy (5-6), Data+Management (7-8), Defaults+Hardening (9-10).

3.6 OWASP IoT Attack Surface Areas

What it is: OWASP's companion list of the 18 physical and digital places on an IoT ecosystem where an attack can actually land, grouped differently from the Top 10 threats list (which describes WHAT goes wrong; this list describes WHERE it can happen).

How it works (18 attack surface areas):

1. Ecosystem (general).
2. Device Memory.
3. Device Physical Interfaces.
4. Device Web Interface.
5. Device Firmware.

6. Device Network Services.
7. Administrative Interface.
8. Local Data Storage.
9. Cloud Web Interface.
10. Third-party Backend APIs.
11. Update Mechanism.
12. Mobile Application.
13. Vendor Backend APIs.
14. Ecosystem Communication.
15. Network Traffic.
16. Authentication/Authorization.
17. Privacy.
18. Hardware (Sensors).

Real-world use: A penetration tester scoping an IoT assessment uses this exact 18-item list as their test plan, checking the device's firmware (5), its local web interface (4), the vendor's cloud API (13), and the companion mobile app (12) as four separate, distinct test targets instead of treating "the device" as one single thing to test.

Tools and commands used here: none directly; this list is a planning framework, not a tool.

Countermeasures and defense: use this list as an audit checklist; a device can be secure on network traffic (15) but still fail completely on physical interfaces (3) if, for example, a debug UART port is left exposed (see section 3.27).

EXAM FOCUS: Do not confuse this list (18 attack SURFACE AREAS, i.e., WHERE) with the OWASP Top 10 IoT THREATS (section 3.5, WHAT). The exam may ask you to classify a scenario as a specific surface area rather than a threat.

Memory trick: "Every room in the house is a surface: memory, wires, web page, firmware, network, admin panel, storage, cloud, APIs, update channel, app, vendor API, chatter between devices, traffic, login, privacy, and the sensors themselves."

3.7 IoT Vulnerabilities (17 Named Items)

What it is: A detailed, 17-item list of specific technical weaknesses found in real IoT devices, going beyond the OWASP Top 10's high-level categories into concrete, testable flaws.

How it works (17 vulnerabilities):

1. **Username Enumeration:** collecting valid usernames by watching how the login mechanism responds.
2. **Weak Passwords:** passwords like "1234" or pre-programmed defaults.

3. **Account Lockout:** no lockout after 3-5 failed login attempts, so brute forcing can continue forever.
4. **Unencrypted Services:** network services not encrypted, open to eavesdropping/tampering.
5. **Two-Factor Authentication (lack of):** no security token or fingerprint scanner as a second factor.
6. **Poorly Implemented Encryption:** encryption exists but is misconfigured or outdated (such as SSL v2).
7. **Update Sent Without Encryption:** updates transmitted over the network without TLS, or the update file itself is unencrypted.
8. **Update Location Writable:** the storage location for update files is world-writable, letting anyone plant malicious firmware.
9. **Denial of Service:** the service (or the whole device) can be knocked offline.
10. **Removal of Storage Media:** an attacker can physically remove the storage media from the device.
11. **No Manual Update Mechanism:** no way to manually force an update check.
12. **Missing Update Mechanism:** no way to update the device at all.
13. **Firmware Version/Update Date Not Displayed:** the current firmware version and/or last update date is hidden from the user.
14. **Firmware and Storage Extraction:** firmware contains source code, binaries, pre-set passwords, and SSH keys, all extractable.
15. **Manipulating the Code Execution Flow:** using a JTAG adapter and GNU debugger (GDB) to modify firmware execution and bypass almost all software-based security controls; side-channel attacks can also leak information or alter execution flow.
16. **Obtaining Console Access:** connecting to a serial interface gives full console access; custom bootloaders that try to block single-user mode can usually still be bypassed.
17. **Insecure Third-Party Components:** out-of-date versions of BusyBox, OpenSSL, SSH, web servers, and similar bundled software.

Real-world use: A security researcher extracts a smart camera's firmware (vulnerability 14), finds a hardcoded root SSH key inside it (also 14), and uses that key to get console access on every camera of that model worldwide (vulnerability 16), because the vendor reused one key across its entire product line.

Tools and commands used here: JTAG adapter + GDB (vulnerability 15); serial/UART tools (vulnerability 16); firmware extraction tools covered fully in sections 3.27-3.32.

Countermeasures and defense: unique per-device keys (never reuse one key fleet-wide); enforce account lockout; encrypt both update transport and update files; make the update storage location read-only to unauthorized users; display firmware version/update date to the user.

EXAM FOCUS: Vulnerabilities 7 and 8 (update sent without encryption vs. update location world-writable) are easy to confuse; 7 is about the TRANSPORT of the update, 8 is about WHO CAN WRITE to where it is stored.

Memory trick: Group them by theme: Login weaknesses (1-3), Encryption/update weaknesses (4-9), Physical/firmware weaknesses (10-17).

3.8 IoT Threats Overview (21 Named Attacks)

What it is: The module's master list of 21 named attack techniques against IoT devices and networks, most of which get their own detailed topic section later (cross-referenced below).

How it works (the 21 threats):

1. DDoS Attack (section 3.9).
2. Attack on HVAC Systems (section 3.10).
3. Rolling Code Attack (section 3.11).
4. BlueBorne Attack (section 3.25).
5. Jamming Attack (section 3.12).
6. Remote Access using Backdoor (section 3.13).
7. Remote Access using Telnet (section 3.30).
8. Sybil Attack (section 3.17).
9. Exploit Kits (section 3.17).
10. Man-in-the-Middle Attack (section 3.17).
11. Replay Attack (section 3.17).
12. Forged Malicious Device (section 3.17).
13. Side Channel Attack (section 3.17).
14. Ransomware (section 3.17).
15. Client Impersonation.
16. SQL Injection Attack.
17. SDR-Based Attack (section 3.14).
18. Fault Injection Attack (section 3.16).
19. Network Pivoting.
20. DNS Rebinding Attack (section 3.15).
21. Firmware Update (FOTA) Attack.

Real-world use: A security awareness briefing for an organization deploying IP cameras walks through this 21-item list and marks which ones already have a mitigating control in place (such as disabling Telnet) versus which remain open risks (such as no FOTA signature checking), turning the list into a live risk register.

Tools and commands used here: see each attack's dedicated section for its specific tools.

Countermeasures and defense: see sections 3.34-3.38 for the full IoT defense checklist, manufacturer guidelines, and OWASP solutions.

EXAM FOCUS: Client Impersonation, SQL Injection Attack, Network Pivoting, and Firmware Update (FOTA) Attack are named here but not expanded with their own diagram slide in this module; know their one-line meaning: Client Impersonation (pretending to be a legitimate IoT client to the server), SQL Injection (injecting malicious SQL into a device/backend's database query), Network Pivoting (using a compromised IoT device as a jump point into the rest of the network), FOTA Attack (abusing Firmware-Over-The-Air update channels to push malicious firmware).

Memory trick: "DHARB JEM FCS RSSF NDF" is unreadable, so just remember the 4 clusters: physical/RF attacks (HVAC, rolling code, BlueBorne, jamming, SDR, fault injection), access attacks (backdoor, Telnet, client impersonation), network attacks (DDoS, Sybil, MITM, replay, DNS rebinding, pivoting), and data/update attacks (SQL injection, ransomware, FOTA, exploit kits, forged device, side channel).

3.9 DDoS Attack (IoT Botnets)

What it is: A Distributed Denial-of-Service attack where an attacker infects many IoT devices with malware, turning them into an army of bots (a botnet), then commands them all at once to flood a target server with traffic until it goes offline.

Analogy: Instead of one person trying to block a shop's doorway, the attacker secretly convinces thousands of strangers scattered across a city to all show up and jam the doorway at the exact same moment.

How it works (5 steps):

1. The attacker exploits vulnerabilities in IoT devices and installs malicious software (malware) on their operating systems.
2. The attacker gains remote access to the now-vulnerable devices.
3. The attacker injects malware into the IoT devices to turn them into bots; multiple infected devices together are called an army of botnets.
4. The attacker uses a command-and-control (C&C) center to instruct the botnets to launch the attack.
5. The target server is hit with a large volume of requests from multiple IoT devices in different locations at once, and goes offline, unable to process further requests.

Real-world use: Thousands of compromised home routers and DVRs, infected because they still used factory-default Telnet passwords, are commanded by a single C&C server to simultaneously flood a gaming company's servers, taking the service offline for hours (this is the real-world pattern behind the historical Mirai botnet, ADDED CONTEXT).

Tools and commands used here: no specific attacker tool named on this slide; see section 3.18 for real IoT botnet malware families.

Countermeasures and defense: change default credentials immediately; disable Telnet (port 23); segment IoT devices onto their own network; monitor for unusual outbound traffic spikes from IoT devices.

EXAM FOCUS: The giveaway phrase "army of botnets" or "multiple infected IoT devices" plus "large volume of requests from different locations" always points to DDoS, not a single-source DoS.

Memory trick: "Infect many, command as one, flood together."

3.10 Exploit HVAC

What it is: An attack path where an organization's internet- connected Heating, Ventilation, and Air Conditioning (HVAC) system, running without proper security, becomes the attacker's gateway into the rest of the corporate network.

Analogy: The HVAC system is like an unlocked side door to a bank: nobody worries about guarding the thermostat, but once an attacker is inside through that door, they are standing in the same building as the vault.

How it works (4 steps):

1. The attacker uses Shodan to search for vulnerable Industrial Control System (ICS) devices connected to the internet.
2. The attacker tries default user credentials against the ICS login (seen in the slide's screenshot as an iRidium server login page).
3. Once logged in, the attacker gains access to the HVAC system through the ICS interface.
4. The attacker controls the temperature from the HVAC system, or pivots to carry out further attacks on the organization's local network.

Real-world use: An attacker finds an exposed HVAC control panel on Shodan, logs in with the default "admin/admin" credentials, and uses that foothold to pivot and attack other systems on the same corporate network the HVAC unit was never supposed to be able to reach.

Tools and commands used here: Shodan (full details in section 3.19 and section 3.56 for the OT-side equivalent).

Countermeasures and defense: change all default ICS/HVAC credentials; isolate HVAC and building-management systems on their own network segment, with no direct path to business-critical systems; remove HVAC panels from direct internet exposure.

EXAM FOCUS: This is a frequently tested real-world case: HVAC was the attacker's foothold in the widely-cited 2013-2014 Target department store breach (ADDED CONTEXT, verify exact year/details against current sources); the exam may describe this scenario without naming "Target" directly and expect you to recognize "HVAC as initial access vector."

Memory trick: "The thermostat is a door, not just a dial."

3.11 Rolling Code Attack

What it is: An attack against smart vehicle key fobs and garage door openers that use a "rolling code" (also called a hopping code): a code that changes every single time you press the unlock button, so that simply recording and replaying an old code should not work.

Analogy: Imagine a lock that only accepts a new secret word every time, and the word is whispered between the key and the lock so quietly that nobody else should be able to hear it. The attacker's trick is to cover one ear of the lock (jam it) so it never hears the first whispered word, while the attacker's own microphone (sniffer) secretly records both words.

How it works (6 steps):

1. The victim presses the car remote button to unlock the car, which transmits an RF signal containing the current rolling code.
2. An attacker with a jamming device jams the car's reception of that first code, while simultaneously sniffing and recording it.
3. Because the car never received the first code cleanly, it does not unlock.
4. The victim, assuming a simple glitch, presses the button again, sending a second code.
5. On this second attempt, the attacker forwards the recorded FIRST code to the car (which the car still accepts, since it was never used), unlocking the car normally for the victim, while the attacker separately records the second code.
6. The attacker later uses the recorded SECOND code (which the car has still never consumed) to unlock and steal the vehicle at will.

Real-world use: A car thief parks near a shopping mall with a cheap RF jammer and receiver, waits for a shopper to "double-click" their key fob (a common user behavior when a car does not unlock on the first press), and walks away with a stored code that will open that exact car later that night.

Tools and commands used here: jamming device, RF sniffer/receiver (concept slide); the matching offensive tool, RFCrack, is detailed in section 3.23.

Countermeasures and defense: modern vehicles increasingly use rolling codes tied to encrypted, time-synchronized sequences that expire quickly; users should manually confirm their vehicle actually locked (listen/look for the physical lock action) rather than trusting a single button press blindly.

EXAM FOCUS: The key exam detail is WHY this works: the attacker needs BOTH a jammer (to stop the first code from being consumed) AND a sniffer (to capture it), not just one or the other; a question may ask what equipment combination is required.

Memory trick: "Jam the first, save the second, steal it later."

3.12 Jamming Attack

What it is: An attack that disrupts communication between wireless IoT devices by flooding the airwaves with radio noise on the same frequency the devices use, so legitimate messages can no longer get through.

Analogy: Like shouting loudly over someone at the exact same pitch as their voice so nobody else in the room can make out what they are saying.

How it works (2 steps):

1. The attacker transmits radio signals randomly, matching the same frequency used by the sensor nodes for their communication.
2. The network becomes jammed as a result, disabling every endpoint from sending or receiving messages on that frequency.

Real-world use: An attacker uses a cheap jamming device to block a wireless home security sensor's communication to its base station right before attempting a physical break-in, since the alarm sensor cannot report "motion detected" if its signal cannot reach the hub.

Tools and commands used here: a jamming device (concept slide); HackRF One (section 3.25-3.26) and RFCrack (section 3.23) both include jamming functionality in this module.

Countermeasures and defense: use frequency-hopping spread spectrum (FHSS) communication where possible; monitor for sudden, total signal-loss events as a possible jamming indicator rather than assuming "device offline" means a dead battery.

EXAM FOCUS: Jamming is a pure AVAILABILITY attack at the RF layer; distinguish it from a DDoS attack (which floods a network/ server with legitimate-looking TRAFFIC, not radio noise).

Memory trick: "Same frequency, same moment, drowned out signal."

3.13 Hacking Smart Grid/Industrial Devices: Remote Access using Backdoor

What it is: A social-engineering-driven attack chain where a phishing email delivers a malicious attachment that silently installs a backdoor, which the attacker then uses to reach the organization's private network and ultimately smart-grid/industrial systems like HVAC and SCADA.

Analogy: The phishing email is a Trojan horse gift; once it is inside the city walls (the employee's computer), soldiers (the backdoor) quietly open the main gate (the network) for the rest of the attacking army.

How it works (5 steps):

1. The attacker gathers basic information about the target organization using social engineering techniques.
2. The attacker sends phishing emails to employees with a malicious attachment.
3. An employee opens the legitimate-looking email and downloads the attachment.
4. If the employee opens/runs the malicious file, it installs a backdoor for the attacker.
5. Using the backdoor, the attacker gains access to the organization's private network; from there, the slide shows the attacker entering the SCADA system and disabling the UPS (Uninterruptible Power Supply), then disabling power supplies, leaving a city in the dark.

Real-world use: Utility-company engineers receive what looks like a routine vendor invoice; one opens the attached file, silently installing a backdoor that the attacker later uses to reach the plant's SCADA network and disrupt power delivery, exactly the kind of attack chain behind real-world grid-disruption incidents (ADDED CONTEXT: this mirrors publicly documented attacks on Ukrainian power grid infrastructure; verify specific incident details against current sources).

Tools and commands used here: phishing email with malicious attachment (concept-level); no specific named tool on this slide.

AI angle: ADDED CONTEXT: attackers increasingly use generative AI to draft highly convincing, grammatically perfect phishing emails tailored to a specific employee's role (for example, an email written to sound exactly like a real vendor's invoice format), making this exact attack chain harder for employees to visually spot; defenders counter with AI-based email security filters that flag anomalous sender/content patterns.

Countermeasures and defense: security awareness training against phishing; email attachment sandboxing; strict network segmentation between IT and SCADA/OT networks (see the Purdue Model, section 3.41); disable or tightly restrict macro/attachment execution.

EXAM FOCUS: This is the classic 5-phase hacking cycle in miniature: reconnaissance (step 1), the actual exploit delivery (steps 2-4), and gaining access plus a pivot toward "maintaining access"- style control over OT systems (step 5).

Memory trick: "One click on a fake invoice, and the lights can go out."

3.14 SDR-Based Attacks on IoT

What it is: A family of three attacks that all use Software Defined Radio (SDR), a radio system whose signal processing is done in software instead of fixed hardware, letting an attacker examine and manipulate IoT communication signals flexibly.

Analogy: A traditional radio is like a single, fixed musical instrument that can only play one kind of note; an SDR is like a synthesizer that can imitate almost any instrument, just by changing its software settings.

How it works (3 attack types):

1. **Replay Attack:** the attacker obtains the specific frequency used between connected devices, captures the original data when a command is initiated, segregates that command sequence, and injects it back into the IoT network later.
2. **Cryptanalysis Attack:** the attacker follows the same capture process as a replay attack, but also reverse-engineers the protocol to fully understand the captured signal; this requires skill in cryptography, communication theory, and modulation schemes.
3. **Reconnaissance Attack:** the attacker gathers information about the target device from its own specifications, then uses a multimeter to investigate the chipset and identify pins (such as ground pins) to discover the product ID and other details.

Real-world use: An attacker examines a wireless door sensor using an SDR, captures its "door opened" signal (replay attack setup), and later replays that exact signal to make a security system falsely believe the door is still closed while physically opening it.

Tools and commands used here: SDR hardware/software (concept slide); HackRF One is this module's specific SDR hardware example (sections 3.25-3.26); RFCrack (section 3.23) performs SDR-based rolling-code and jamming attacks specifically.

Countermeasures and defense: use devices with proper encryption and rolling/ one-time codes rather than static, replayable signals; shield or limit the range of sensitive RF-based control signals.

EXAM FOCUS: Tell the three types apart by GOAL: Replay = re-send a captured command; Cryptanalysis = break the protocol's math/encoding; Reconnaissance = learn about the hardware itself (no signal replay involved).

Memory trick: "Capture and resend (replay), capture and crack (cryptanalysis), or just look and learn (recon)."

3.15 Identifying and Accessing Local IoT Devices (DNS Rebinding)

What it is: An attack where a victim's browser is tricked into visiting a malicious web page (via an ad or other lure), and that page's embedded code scans the victim's own LOCAL network for IoT devices, then uses DNS rebinding to actually command and control those local devices from the attacker's remote server.

Analogy: Normally a website can only "look" at the internet, not reach into your home network, like a stranger who can see your house from the street but can't walk inside. DNS rebinding tricks your browser into believing the stranger's address has changed to be your own house's address, so it lets them walk right in.

How it works (2 phases):

1. **Discovering/identifying local IoT devices:** malicious code in the victim's browser obtains the victim's local IP address, then sends requests to every address in that local range. Active devices reply with a TCP reset (RST) packet; inactive devices time out instead. The code also gathers private information like UIDs and BSSIDs of nearby access points, then classifies all devices by their response pattern.
2. **Accessing local devices via DNS rebinding:** the attacker checks whether the malicious code can perform DNS rebinding across all discovered devices, using tools such as Singularity of Origin. Once DNS rebinding is working, the attacker can command and control the victim's local IoT devices directly from their own remote server, because the browser now believes the attacker's web server and the local device share the same origin.

Real-world use: A victim clicks an ordinary-looking online ad; in the background, the ad's code silently scans the victim's home network, finds a smart thermostat, and uses DNS rebinding to send it unauthorized commands, all without the victim ever installing anything or noticing a thing.

Tools and commands used here: Singularity of Origin (DNS rebinding testing framework).

Countermeasures and defense: IoT devices and home routers should validate the `Host` header on incoming requests and reject requests claiming an unexpected hostname; browsers and DNS resolvers can also block replies that rebind a public hostname to a private/internal IP address.

EXAM FOCUS: The giveaway phrase is "a malicious ad/page" plus "no direct network access needed" plus "controls devices on the VICTIM'S OWN local network" - this combination points specifically to DNS rebinding, not a classic remote exploit.

Memory trick: "The browser is tricked into thinking 'over there' is actually 'right here.'"

3.16 Fault Injection Attacks

What it is: Also called Perturbation attacks, these attacks inject a fault, glitch, or malicious program into a device to compromise its security; they can be invasive (physically altering the chip) or non-invasive (altering its operating environment from outside).

Analogy: Instead of picking a lock the normal way, the attacker briefly messes with the lock's power or temperature at just the right instant, making it fail open by accident.

How it works (4 types):

1. **Optical, Electro-Magnetic Fault Injection (EMFI), Body Bias Injection (BBI):** attackers inject faults by projecting lasers or electromagnetic pulses at the chip.
2. **Power/Clock/Reset Glitching:** attackers inject faults or glitches into the chip's power supply or clock network.
3. **Frequency/Voltage Tampering:** attackers tamper with the chip's operating conditions by modifying the power supply level and/or altering the clock frequency.
4. **Temperature Attacks:** attackers alter the operating temperature of the chip, affecting its whole operating environment.

Real-world use: A hardware security researcher uses a precisely timed voltage glitch (type 2 or 3) on a smart lock's microcontroller at the exact moment it checks a PIN code, causing the chip to skip the comparison instruction entirely and treat any PIN as correct.

Tools and commands used here: lasers/EM pulse rigs, glitching hardware (concept-level, specialized hardware lab equipment); see section 3.28 for the related NAND glitching technique and section 3.64 for general ICS hardware-hacking tools.

Countermeasures and defense: add sensors that detect abnormal voltage, clock, temperature, or light exposure and respond by locking or wiping the chip; use tamper-resistant chip packaging.

EXAM FOCUS: Fault injection targets the HARDWARE's physical operating conditions to cause a security check to fail, unlike a software exploit that targets a coding bug; the exam may ask you to identify fault injection from a description involving voltage, clock, temperature, or lasers rather than code.

Memory trick: "Light, power, clock, or heat - mess with the chip's environment, not its code."

3.17 Other IoT Attacks (Sybil, Exploit Kits, MITM, Replay, Forged Device, Side-Channel, Ransomware)

What it is: Seven additional named IoT attacks, each described in one line in the source material, rounding out the 21-item list from section 3.8.

How it works (7 attacks):

1. **Sybil Attack:** the attacker uses multiple forged identities to create a strong illusion of traffic congestion, disrupting communication between neighboring nodes and networks.
2. **Exploit Kits:** the attacker uses malicious scripts to exploit poorly patched vulnerabilities in an IoT device.
3. **Man-in-the-Middle (MITM) Attack:** the attacker pretends to be a legitimate sender, intercepts all communication between the real sender and receiver, and hijacks it.
4. **Replay Attack:** the attacker intercepts legitimate messages from a valid communication and continuously resends the intercepted message to the target device, causing a denial-of-service or crashing the device.
5. **Forged Malicious Device:** the attacker physically replaces an authentic IoT device with a malicious one, if they have physical access to the network.
6. **Side-Channel Attack:** the attacker extracts information about encryption keys by observing emitted signals ("side channels") from IoT devices.
7. **Ransomware Attack:** malware that uses encryption to block a user's access to their own device, either by locking the screen or locking their files.

Real-world use: An attacker with brief physical access to a smart building swaps a legitimate door-access card reader for a visually identical but malicious one (forged malicious device), which silently forwards every employee's card credentials to the attacker while still unlocking doors normally so nobody notices.

Tools and commands used here: no specific named tool on this slide; these are conceptual attack definitions.

Countermeasures and defense: physically secure device locations and use tamper-evident seals (against forged devices); use signed, timestamped/nonce-based messages (against replay attacks); monitor for duplicate or impossible node identities (against Sybil attacks); maintain offline, versioned backups (against ransomware).

EXAM FOCUS: Side-channel attack (item 6 here) targets ENCRYPTION KEYS via emitted signals; this is the IoT-domain version of the same concept that appears again for OT in section 3.49 (timing/power analysis) - same underlying idea, different domain and technique.

Memory trick: "Fake identities (Sybil), fake devices (forged), fake senders (MITM), repeated old messages (replay), broken patches (exploit kits), leaked signals (side-channel), locked files (ransomware)."

3.18 IoT Malware

What it is: Real, named malware families that specifically target IoT devices, most often by exploiting default credentials and open management ports (such as SSH and Telnet) to recruit devices into botnets.

How it works (KmsdBot example, 3 steps):

1. The latest version (called Kmsdx) adds new features like Telnet scanning and authentication, widening the range of IoT devices it can compromise.
2. It exploits SSH ports and default credentials, and supports a broader range of CPU architectures commonly found in IoT devices, showing adaptability in how it infiltrates targets.
3. Because many IoT devices still run their factory-default credentials (never changed by users), the risk of compromise and botnet recruitment stays high.

Real-world use: KmsdBot silently scans the internet for devices still answering on SSH or Telnet with default usernames/passwords, logs in, and drops itself onto the device to use its CPU/bandwidth for further attacks or cryptomining, all without the device's owner ever noticing a thing changed.

Tools and commands used here: none (this is malware the attacker deploys, not a hacking tool the attacker runs manually); the source material also shows a code-analysis screenshot (file names like `main.go`, `ssh.go`, `telnet.go`, `util.go` and stored credentials in `telnet.txt`) illustrating how researchers reverse engineer such malware's source.

Other named IoT malware families: WailingCrab, P2PInfect, NKAbuse, IoTroop, XorDdos (named in this module as additional current IoT malware families; each follows the same general pattern of exploiting weak credentials/unpatched services to spread across IoT devices).

AI angle: ADDED CONTEXT: security researchers increasingly use AI- assisted reverse engineering (automatically summarizing what a function like `sshcheck` or `scantelnet` does from its code, similar to the manual line-count table EC-Council shows for KmsdBot) to speed up malware triage when thousands of new IoT malware samples appear each month.

Countermeasures and defense: never leave factory-default credentials in place; close or firewall off SSH/Telnet on IoT devices that do not need remote management; keep firmware patched; monitor for devices suddenly scanning other IPs (a sign of botnet behavior).

EXAM FOCUS: The exam may ask which weakness most IoT botnet malware (KmsdBot and similar) relies on first: the answer is almost always default/unchanged credentials on exposed management services (SSH/Telnet), not a sophisticated zero-day exploit.

Memory trick: "Same old password, same old port, same old takeover."

3.19 Information Gathering using Shodan

What it is: Shodan is a search engine that indexes internet- connected devices themselves (not web pages), such as routers, traffic lights, CCTV cameras, servers, and smart home devices, by continuously scanning the internet and recording each device's banner response.

Analogy: Google indexes web PAGES; Shodan indexes DEVICES. It is like a phone book not for people, but for every machine that ever answered a knock on its door.

How it works (search filters):

1. Attackers search by device type and country, for example `webcamxp country:"US"`, to list every matching webcam Shodan has indexed in that country.
2. Attackers search by city, for example `webcamxp city:"paris"`, to narrow results geographically.
3. Attackers search by exact coordinates, for example `webcamxp geo:-50.81,201.80`, to find webcams near a precise location.
4. Each result includes gathered information such as IP address, hostname, ISP, device location, and the device's own banner text (which can leak the exact software/version running).

Real-world use: An attacker runs `webcamxp country:"US"` and gets a results page showing total result counts, top cities, and top ports for every matching device; clicking into one result shows a raw HTTP banner response (server type, headers, cache-control, and more) that often reveals exploitable details without the attacker ever touching the device directly.

Tools and commands used here: Shodan search syntax: `webcamxp country:"US"`, `webcamxp city:"paris"`, `webcamxp geo:<lat>,<long>` (full breakdown in section 5.1). Other information- gathering tools named alongside Shodan: MultiPing, FCC ID Search, Censys, FOFA.

Countermeasures and defense: minimize what a device's banner reveals (hide version strings where possible); do not expose management/viewing interfaces directly to the internet; periodically search Shodan for your own organization's devices to catch unintentional exposure early.

EXAM FOCUS: Shodan search filters (`country:`, `city:`, `geo:`) are a classic exam target; expect a question asking you to read a Shodan query string and identify exactly what it searches for.

Memory trick: "Shodan is Google for things that blink, not things that write."

3.20 Information Gathering through Sniffing

What it is: A workflow combining Nmap (to spot insecure open ports), wireless tools (Airmon-ng, Airodump-ng) to find and monitor nearby wireless networks, and Wireshark to capture and read the raw traffic, including IoT-specific protocols like MQTT.

How it works (6 steps):

1. Run Nmap against the target IP range, checking common insecure HTTP ports used by IoT devices: `nmap -p 80,81,8080,8081 <Target IP address range>`.
2. Run `ifconfig` to identify your own wireless card (the slide's example interface is named `wlan0`).
3. Run Airmon-ng to put that wireless card into monitor mode: `airmon-ng start wlan0`.
4. Run Airodump-ng to scan all nearby wireless networks: `airodump-ng start wlan0mon`.
5. Discover the target wireless network, note its channel, and set your wireless card to listen on that same channel using Airmon-ng again: `airmon-ng start wlan0mon 11` (channel 11 in this example).
6. Launch Wireshark, double-click the interface that is in monitor mode (`wlan0mon`), and start capturing traffic; the slide's capture shows MQTT (MQ Telemetry Transport Protocol) ping requests/ responses and publish messages flowing between an IoT device (10.10.1.22) and a broker (10.10.1.19) on TCP port 1883.

Real-world use: A penetration tester captures MQTT traffic between a smart-home hub and its devices, reading unencrypted "publish" and "ping" messages in Wireshark, confirming the deployment uses no TLS and leaks device state (such as "door unlocked") in plaintext over the air.

Tools and commands used here: `nmap -p 80,81,8080,8081 <range>`, `ifconfig`, `airmon-ng start <interface>`, `airodump-ng start <interface>mon`, Wireshark (full breakdown in section 5.2-5.3). Other sniffing/IoT tools named alongside: Suphammer (`suphammer.net`) and a second partially-legible IoT sniffing tool reference [UNCLEAR on page 11: the exact second tool name/URL after "Suphammer" was not fully legible in the source capture].

Countermeasures and defense: encrypt MQTT with TLS (and use MQTT authentication, not anonymous connections); use WPA2/WPA3 on the wireless network itself so Airodump-ng-style monitoring cannot capture readable traffic; avoid using insecure HTTP ports (80/8080) for any device management interface.

EXAM FOCUS: MQTT's default port, 1883, with no built-in encryption, is a frequently tested fact; expect a question asking what port MQTT uses and/or whether it is encrypted by default (it is not; TLS-secured MQTT uses port 8883 instead, ADDED CONTEXT, verify).

Memory trick: "Scan the ports, find the channel, tune in, then read the mail" (Nmap -> Airmon-ng/Airodump-ng -> same channel -> Wireshark).

3.21 Vulnerability Scanning using IoTSeeker

What it is: IoTSeeker is a tool that scans a network for specific types of IoT devices and reports whether each one is still using its factory-default credentials or has had its password changed.

How it works:

1. The attacker (or a defending organization auditing its own network) runs IoTSeeker against a list or range of target IP addresses, using the Perl script: `perl iotScanner.pl <IP1>,<IP2>,<IP3>,...`.
2. For each address, IoTSeeker identifies the device type (examples from the slide: Stardot, Arecont, American Dynamics, W-Box) and reports one of three states: "has default passwd" (still vulnerable), "has changed passwd" (safe), or (implicitly) not a matching device type at all.

Real-world use: An organization's security team runs IoTSeeker across its own camera subnet and finds that two specific IP cameras (of type W-Box and American Dynamics) still report "default passwd," giving them a prioritized, exact list of devices to immediately reconfigure, rather than treating every device as equally risky.

Tools and commands used here: `perl iotScanner.pl <ip_list>` (full breakdown in section 5.4).

Countermeasures and defense: change every device's default password at setup time, before it is ever connected to a network; run IoTSeeker or an equivalent tool periodically as a self-audit, not just once.

EXAM FOCUS: IoTSeeker's output format ("is of type X has/still has default passwd") is distinctive; a question may show sample output and ask which specific device still needs a password change.

Memory trick: "Seek first, then speak: which ones still answer to their factory name?"

3.22 Vulnerability Scanning using Genzai

What it is: Genzai is an IoT security toolkit that detects and scans IoT devices (wireless routers, surveillance cameras, Human Machine Interfaces/HMIs) for default passwords and known vulnerabilities, matched by URL path and software version.

How it works:

1. Genzai is pointed at a target device or network range and fingerprints the device type and running software version.
2. It checks the fingerprinted device against a database of known default-password and vulnerability signatures (the slide's example shows a "TP-Link Router Default Password" finding and a "TP-LINK Wireless N Router WAN Firmware Vulnerable to Buffer Overflow - CVE-2016-8423" finding, tagged as output in JSON-style results).
3. Results are logged (the slide shows output saved to `scan.json`) for later review or reporting.

Real-world use: A penetration tester runs Genzai across a client's office Wi-Fi network and discovers an old TP-Link router still exposed with a known, specific CVE-tagged buffer-overflow vulnerability, giving the client exact, actionable remediation guidance instead of a vague "update your router" recommendation.

Tools and commands used here: Genzai (GUI/CLI toolkit, exact syntax not fully legible in the source capture [UNCLEAR on page 12]). Other named vulnerability-scanning tools: beSTORM, IoTSploit, IoTSeeker (section 3.21), IoTVAS.

Countermeasures and defense: patch router/HMI firmware against known CVEs promptly; change default credentials; restrict management interfaces to trusted internal networks only.

EXAM FOCUS: Genzai's defining feature versus IoTSeeker is that it matches against named CVEs (specific, numbered vulnerabilities), not just "default password: yes/no."

Memory trick: "Genzai names the exact CVE; IoTSeeker just checks the lock."

3.23 Rolling Code Attack using RFCrack

What it is: RFCrack is a Python-based Software Defined Radio (SDR) attack tool for testing RF communications between any physical device that communicates over sub-GHz frequencies (IoT devices, cars, alarm systems), built for a Yardstick One SDR device.

How it works (4 attack commands):

1. **Live Replay (scan and capture mode):** `python RFCrack.py -i` starts RFCrack's interactive scanning mode to find and capture signals live.

2. **Rolling Code attack:** `python RFCrack.py -I -M MOD_2FSK -F 314350000` listens using 2-FSK modulation at a specific frequency (314.35 MHz in this example) to capture the rolling codes described in section 3.11.
3. **Adjust RSSI Range:** `python RFCrack.py -I -M MOD_2FSK -F 314350000 -U -100 -L -10` fine-tunes the signal-strength (RSSI) capture window using upper (`-U`) and lower (`-L`) bounds.
4. **Jamming:** `python RFCrack.py -j -F 314000000` actively jams the given frequency.

Real-world use: A car-theft investigator reproduces the rolling code attack from section 3.11 in a lab using RFCrack and a Yardstick One device, running the rolling-code command at the target vehicle's known frequency to demonstrate exactly how a thief could capture and later replay an unused unlock code.

Tools and commands used here: `python RFCrack.py -i`, `python RFCrack.py -I -M MOD_2FSK -F <freq>`, `python RFCrack.py -I -M MOD_2FSK -F <freq> -U <upper> -L <lower>`, `python RFCrack.py -j -F <freq>` (full breakdown in section 5.5); requires a Yardstick One SDR device (2 needed for rolling-code attacks specifically, one to jam and one to capture).

Countermeasures and defense: see section 3.11's countermeasures; additionally, defenders can use their own SDR hardware to monitor for abnormal RF jamming activity near sensitive facilities or vehicles.

EXAM FOCUS: RFCrack's flags map directly onto SDR concepts: `-M` is modulation scheme, `-F` is frequency in Hz, `-U`/`-L` are RSSI (signal strength) bounds, `-j` is jam mode; the exam may ask you to read one of these flags.

Memory trick: "i for info/scan, M for modulation, F for frequency, j for jam."

3.24 Hacking Zigbee Devices with Open Sniffer

What it is: Open Sniffer is a hardware/software tool that captures all IEEE 802.15.4 frames (the radio standard Zigbee is built on) within its physical range, and visualizes the captured frames on a chosen channel directly inside Wireshark.

How it works (3 capabilities):

1. Open Sniffer passively captures every 802.15.4 frame in range, including Zigbee traffic, and feeds it into Wireshark for analysis (the slide's example `zigbee-join-authenticate.pcap` shows beacon requests, beacon responses with EPID "MiniwareTech," and command frames).
2. Open Sniffer can continuously emit (transmit) packets, which can be used to create a Denial-of-Service attack on the target Zigbee device or network by flooding it.
3. Open Sniffer can send user-defined, custom frames, letting an attacker replay previously captured frames to perform a replay attack against the Zigbee network.

Real-world use: An attacker captures a Zigbee smart-lock's "join network" and "authenticate" exchange using Open Sniffer, studies the beacon and command frame structure in Wireshark, and later uses the same tool's custom-frame feature to replay a captured unlock command.

Tools and commands used here: Open Sniffer hardware + Wireshark (802.15.4/ Zigbee dissector); no CLI command shown in the source material beyond opening the resulting `.pcap` file in Wireshark.

Countermeasures and defense: use Zigbee's network-layer encryption (Zigbee supports AES-128) rather than relying on frequency obscurity alone; monitor for unexpected beacon request floods (possible DoS precursor); rotate network keys periodically.

EXAM FOCUS: Open Sniffer targets 802.15.4-based protocols broadly (Zigbee is the named example); the exam may describe "capturing beacon frames with an EPID field" as the giveaway for Zigbee/802.15.4 traffic specifically, distinct from Wi-Fi (802.11) capture.

Memory trick: "802.15.4 is Zigbee's native tongue; Open Sniffer is the interpreter Wireshark reads it through."

3.25 BlueBorne Attack Using HackRF One

What it is: HackRF One is an advanced, half-duplex software- defined radio covering 1 MHz to 6 GHz, used by attackers to perform attacks such as BlueBorne (a Bluetooth attack) or "AirBorne"-style attacks, including replay, fuzzing, and jamming, against a wide range of wireless protocols.

Analogy: HackRF One is a universal radio translator: instead of being built for just one wireless standard, it can be reprogrammed in software to speak almost any radio "language," from GSM to Z-Wave.

How it works (3 facts):

1. HackRF One operates over a very wide frequency range, 1 MHz to 6 GHz, covering most IoT and consumer wireless bands in one device.
2. It transmits and receives in half-duplex mode (one direction at a time), which makes it straightforward for attackers to use it for replay, fuzzing, and jamming attacks.
3. It can sniff a wide range of wireless protocols, from GSM (cellular) to Z-Wave (smart home).

Real-world use: A researcher studying BlueBorne-style Bluetooth vulnerabilities uses HackRF One to capture and analyze Bluetooth Low Energy traffic from a smart lock, looking for implementation flaws that could let an attacker take over the connection without ever pairing.

Tools and commands used here: HackRF One hardware (`hackrf_info`, `hackrf_transfer`, detailed in section 3.26 and section 5.6).

Countermeasures and defense: keep Bluetooth/wireless firmware patched against known vulnerability classes like BlueBorne; disable Bluetooth when not actively in use; use devices that support modern, encrypted pairing methods.

EXAM FOCUS: HackRF One's defining exam fact is its frequency range (1 MHz-6 GHz) and half-duplex operation; a question may ask which SDR hardware this describes, distinguishing it from narrower-range tools.

Memory trick: "One radio, one direction at a time, almost every frequency."

3.26 Replay Attack using HackRF One

What it is: A concrete walkthrough of using HackRF One to replay a captured RF signal against a target IoT device, after first determining the device's operating frequency.

How it works (4 steps):

1. The attacker determines the target device's frequency using online resources such as the FCC database, or using a tool like RTL-SDR to detect the frequency in the vicinity directly.
2. The attacker captures (records) the target signal with HackRF One: `hackrf_transfer -r <file>.raw -f 433900000 -l 20 -g 20` (receive, save to a raw file, at 433.9 MHz, with specified IF/VGA gain values).
3. The attacker replays the captured signal back at the target: `hackrf_transfer -t <file>.raw -f 433900000 -x 40` (transmit, from the saved raw file, at the same frequency, with a specified TX gain).
4. The attacker verifies the HackRF One hardware is connected and working using `hackrf_info`, which reports the board ID, firmware version, part ID, and serial number.

Real-world use: An attacker captures a wireless doorbell's "button pressed" signal at 433.9 MHz using `hackrf_transfer -r`, then later replays the exact same raw capture using `hackrf_transfer -t` to repeatedly trigger the doorbell remotely, confirming the device has no replay protection.

Tools and commands used here: `hackrf_transfer -r <file> -f <freq> -l <if_gain> -g <vga_gain>`, `hackrf_transfer -t <file> -f <freq> -x <tx_gain>`, `hackrf_info` (full breakdown in section 5.6).

Countermeasures and defense: use rolling/one-time codes (section 3.11) instead of static RF signals for any security-relevant command; add timestamp or sequence-number validation so a replayed old signal is rejected.

EXAM FOCUS: `-r` (receive/capture) versus `-t` (transmit/replay) is the core flag pair the exam may test; also note `-f` always sets frequency in Hz for `hackrf_transfer`.

Memory trick: "r for record, t for transmit, f for frequency - record it, then throw it back."

3.27 Identifying IoT Communication Buses and Interfaces

What it is: Attackers identify serial and parallel hardware interfaces on an IoT device's circuit board, such as UART, SPI, JTAG, and I2C, to gain shell access, extract firmware, or otherwise interact with the device at a low level.

How it works (using BUS Auditor, 2 examples):

1. **UART scan:** `busauditor.generic.uartscan -v 3.3 -p /dev/ttyACM0` scans for UART pins at a 3.3V target voltage over the given serial port, and reports the discovered RX/TX pin combination and baud rate (the slide's result: RX on pin 1, TX on pin 0, baud rate 9600).
2. **I2C scan:** `busauditor.generic.i2cscan -v 3.3 -p /dev/ttyACM0 -s 0 -e 10` scans pins 0 through 10 for an I2C bus at 3.3V, and reports the discovered device address and SCL/SDA pin assignments (the slide's result: device address 0x48, SCL on pin 1, SDA on pin 8).

Real-world use: A hardware hacker opens an IoT device's case, connects BUS Auditor probes to unlabeled header pins on the circuit board, runs a UART scan, and within seconds identifies which two pins carry serial console traffic, letting them attach a USB-serial adapter and get a login prompt without any prior documentation of the board.

Tools and commands used here: BUS Auditor (`busauditor.generic.uartscan`, `busauditor.generic.i2cscan`), DIVA (Damn Insecure and Vulnerable Application), and a PCB (printed circuit board) itself as the physical target (full breakdown in section 5.7).

Countermeasures and defense: disable or physically remove unused debug headers (UART/JTAG/I2C) before shipping a production device; require authentication even over console/serial access, not just over the network.

EXAM FOCUS: Know the 4 named interfaces and their one-line purpose: UART (asynchronous serial console), SPI (synchronous serial, often flash memory), JTAG (chip debugging/programming), I2C (synchronous serial bus for multiple low-speed peripherals).

Memory trick: "UART talks, SPI feeds flash, JTAG debugs, I2C chats with many chips on one line."

3.28 NAND Glitching

What it is: NAND glitching is the process of gaining privileged root access while a device is booting, achieved by making a physical ground connection to the serial I/O pin of the device's flash memory (NAND) chip at a precise moment during boot.

Analogy: Like tripping a hidden switch at the exact instant a vault door is swinging open, so it jams half-open instead of fully locking, letting you slip inside before the normal boot/security sequence finishes.

How it works (2 steps):

1. The attacker interrupts the ongoing booting process of the device by grounding the serial I/O pin of the flash memory chip at a specific point during startup.
2. This interruption causes the device's boot sequence to behave unexpectedly (the slide's captured console output shows boot errors like "bad PEB handling," MMC/DMA errors, and a filesystem mount failure), which the attacker leverages to attain root access on the device that normal boot security would not have granted.

Real-world use: A researcher reverse-engineering a locked-down IoT set-top box uses NAND glitching on its flash chip during power-on, causing the bootloader's security checks to be skipped and dropping into a root shell instead of the normal locked user interface.

Tools and commands used here: a ground wire/probe applied to the flash chip's serial I/O pin at boot time (specialized hardware technique; no software tool named in the source material).

Countermeasures and defense: use flash memory chips and bootloaders with tamper-detection and secure-boot chains that verify signatures at every boot stage, not just the first; physically shield or epoxy-cover flash chip pins on production hardware.

EXAM FOCUS: NAND glitching is a PHYSICAL, boot-time attack targeting the flash memory chip specifically; distinguish it from the general fault-injection categories in section 3.16 (which target a chip's power/clock/voltage/temperature more broadly, not specifically the NAND flash boot sequence).

Memory trick: "Ground the flash chip mid-boot, and the front door jams open."

3.29 Exploiting Cameras using CamOver

What it is: CamOver is a camera exploitation tool that discloses a network camera's admin password, letting an attacker log in with full administrative control.

How it works (3 usage modes):

1. Run `camover` with no arguments to see its help/usage output, confirming the tool is installed and listing its options (`-t` for threads, `-o` output file, `-i` input file of addresses, `-a` single address, `--shodan`/`--zoomeye` with an API key, `-p` pages to retrieve from ZoomEye).
2. Exploit a single camera by IP address: `camover -a <Camera IP Address>`.
3. Exploit many internet-connected cameras at once by combining multi-threading with a Shodan search: `camover -t --shodan <Shodan API Key>`.

Real-world use: An attacker runs `camover -t --shodan <api_key>` and within minutes receives disclosed admin passwords for dozens of publicly exposed network cameras across different networks, all pulled automatically through Shodan's own search results.

Tools and commands used here: `camover -a <ip>`, `camover -t --shodan <key>`, `camover --zoomeye <key> -p <pages>` (full breakdown in section 5.8).

Countermeasures and defense: change every camera's default admin password immediately after installation; never expose camera admin interfaces directly to the internet; disable remote/cloud access features not actually needed.

EXAM FOCUS: CamOver's defining exam detail is that it combines with Shodan/ZoomEye search results to mass-target cameras, not just one at a time; expect a question contrasting the single-IP flag (`-a`) with the mass-scan flags (`--shodan`, `--zoomeye`).

Memory trick: "CamOver: one camera with `-a`, or an entire Shodan page of them at once."

3.30 Gaining Remote Access using Telnet

What it is: An attack path where an attacker scans for IoT devices with an open Telnet port (port 23), then uses that unencrypted remote-access protocol to log in, often with default credentials, gaining full remote access to the device.

How it works (3 steps):

1. The attacker performs port scanning to learn which ports and services are open on the target IoT device.
2. Many embedded-system IoT devices (industrial control systems, routers, VoIP phones, televisions) implement remote access using Telnet by default.
3. If the attacker finds the Telnet port open, they exploit it to gain remote access to the device, often using tools such as Shodan and Censys to find such exposed

devices in bulk (the slide's example shows a Shodan host page for a Dahua DVR with Telnet open on port 23, showing a login banner).

Real-world use: An attacker searches Shodan for devices with port 23 open, finds a Dahua DVR login banner, tries the device's known factory-default Telnet credentials, and gains a remote shell on the DVR, which can then be used as a foothold or recruited into a botnet (section 3.9).

Tools and commands used here: Shodan, Censys (search for open port 23); a standard `telnet <ip> 23` client connection to actually log in (concept-level; exact login syntax not shown as a separate command in the source material).

Countermeasures and defense: disable Telnet entirely (port 23) on any device that supports a more secure alternative (SSH); if Telnet absolutely must remain for legacy reasons, restrict it to an isolated management network, never the open internet.

EXAM FOCUS: "Disable Telnet (port 23)" is one of the most frequently repeated, single most memorable countermeasures across this entire module (it reappears in section 3.34's defense checklist); remember Telnet is unencrypted, unlike SSH.

Memory trick: "Telnet types in the clear; anyone listening reads your password."

3.31 Maintain Access by Exploiting Firmware

What it is: After gaining remote access to an IoT device, an attacker explores its file system to access the device's firmware, then uses a tool like Firmware Mod Kit to deconstruct and reconstruct that firmware (potentially inserting malicious changes) to maintain long-term access.

How it works (using Firmware Mod Kit):

1. Run the extraction script against the target firmware image: `./extract-firmware.sh /root/docs/<path>/dd-wrt.v24_micro_generic.bin`.
2. Firmware Mod Kit scans the firmware for known signatures (the slide's example found a TRX firmware header, gzip-compressed data, LZMA-compressed data, and a SquashFS filesystem with a DD-WRT signature) and reports each one's offset, size, and checksum information.
3. It then extracts the TRX header image and the SquashFS filesystem at their respective offsets, giving the attacker direct access to the device's actual filesystem contents.
4. Firmware Mod Kit allows both deconstruction (extraction, as above) and reconstruction (rebuilding a modified firmware image), letting an attacker insert a backdoor and repackage the firmware to look unmodified.

Real-world use: An attacker extracts a router's stock firmware using Firmware Mod Kit, adds a hidden backdoor script into the extracted filesystem, repackages it into a new firmware image that looks identical to the original, and tricks the device owner into "updating" to this malicious version, achieving long-term persistence.

Tools and commands used here: `extract-firmware.sh <firmware>.bin` (full breakdown in section 5.9).

Countermeasures and defense: require cryptographically signed firmware updates, verified by the device before installation, so a repackaged/modified image is rejected; never allow unsigned firmware to be side-loaded.

EXAM FOCUS: Firmware Mod Kit's two-way capability (extract AND rebuild) is the key exam fact; a question may ask what makes this tool different from a simple firmware viewer.

Memory trick: "Take it apart, put a backdoor in, put it back together, looking exactly the same."

3.32 Firmware Analysis and Reverse Engineering

What it is: A structured, 6-stage process for analyzing an IoT device's firmware to find passwords, API tokens, endpoints, vulnerable services, backdoor accounts, configuration files, private keys, and other stored data.

How it works (6 stages):

1. **Obtain Firmware:** after gaining access to the target IoT device, extract the firmware from it.
2. **Analyze Firmware:** run `file` on the `.bin` file to identify its type; verify the firmware's MD5 signature; run `strings` against the `.bin` file to pull out readable text; run `hexdump` against the `.bin` file to inspect raw bytes.
3. **Extract the Filesystem:** run `binwalk` to analyze, reverse engineer, and extract data from the firmware image; then extract the actual filesystem using `dd`.
4. **Mount the Filesystem:** create a mount directory (for example, `mkdir rootfs`), then mount the extracted filesystem image, for example `sudo mount -t ext2 <filename> rootfs`.
5. **Analyze the Filesystem:** check key files and folders such as `/etc/passwd`, `/etc/shadow`, and `/etc/ssl`; search recursively for sensitive keywords with `grep -rnw /path/to/somewhere/ -e "pattern"` (for words like "password," "admin," "root"); search for sensitive file types with `find . -name "*.conf"` and similar patterns for `.pem`, `.crt`, and other file extensions.
6. **Emulate Firmware:** dynamically test the device's web interface using emulation software such as QEMU, for example `qemu-mipsel -L`

`<prefix> <binary>`, `qemu-arm -L <prefix> <binary>`, or `qemu-<arch> -L <prefix> <binary>` for other CPU architectures.

Real-world use: A researcher follows all 6 stages against a smart plug's firmware: extracts it, runs `strings` and finds a hardcoded cloud API token in plaintext, mounts the filesystem to confirm `/etc/shadow` contains a weak hashed root password, and finally emulates the device's web server under QEMU to test its login page for injection flaws without needing the physical hardware at all.

AI angle: ADDED CONTEXT: AI-assisted static analysis tools (building on tools like Ghidra and Binwalk, covered again in section 3.64) can now automatically summarize what a decompiled firmware function likely does in plain English, speeding up stage 2 (Analyze Firmware) for large or obfuscated codebases.

Tools and commands used here: `file`, `md5sum`, `strings`, `hexdump`, `binwalk`, `dd`, `mkdir`, `mount`, `grep -rnw`, `find -name`, `qemu-mipsel`/`qemu-arm`/`qemu-<arch>` (full breakdown in section 5.10).

Countermeasures and defense: never hardcode passwords, API tokens, or private keys inside firmware images; encrypt sensitive configuration data at rest within the firmware; strip debug symbols and unnecessary strings from production firmware builds.

EXAM FOCUS: Memorize the ORDER of the 6 stages; the exam may give a scenario describing one stage's output (such as "extracting the SquashFS filesystem with dd") and ask which stage of firmware analysis this represents.

Memory trick: "Obtain, Analyze, Extract, Mount, Analyze (again, the filesystem this time), Emulate" - OAEMAE, or simply "get it, read it, crack it open, step inside, search it, run it."

3.33 IoT Hacking Tools (CatSniffer and Others)

What it is: A roundup slide naming several additional IoT hacking tools beyond those already detailed, each with its own narrow specialty.

How it works (tool list):

1. **CatSniffer:** passively monitors IoT traffic to gather information about devices, communication protocols, and data exchanges, helping identify potential targets and vulnerabilities on an IoT network; built around a CC1352 radio chip with general-purpose and antenna reset switches and status LEDs.
2. **KillerBee:** a framework for Zigbee/802.15.4 security research and attack (packet capture, injection, and exploitation).
3. **JTAGulator:** a hardware tool that identifies on-chip debug/ programming interfaces (such as JTAG and UART) from unknown header pins on a circuit board.
4. **PENIOT:** a penetration testing tool specifically built for IoT devices.

5. **RouterSploit:** an exploitation framework specifically targeting embedded devices such as routers.

Real-world use: A hardware security consultant uses JTAGulator to quickly map which pins on an unfamiliar IoT device's circuit board belong to a JTAG debug interface, then uses that interface to extract firmware for further analysis using the methods from section 3.32.

Tools and commands used here: CatSniffer, KillerBee, JTAGulator, PENIOT, RouterSploit (all GitHub-hosted tools; see section 4 for the consolidated tool encyclopedia).

Countermeasures and defense: physically disable or remove JTAG/ debug headers in production hardware; monitor Zigbee/802.15.4 networks for unexpected packet injection.

EXAM FOCUS: JTAGulator specifically identifies UNKNOWN pinouts (it is a discovery tool for header pins), distinct from BUS Auditor (section 3.27), which runs a scan once you already have an idea of which pins to probe; RouterSploit targets embedded/ router devices specifically, similar in spirit to Metasploit but router-focused.

Memory trick: "Cat sniffs, Killer stings Zigbee, JTAGulator maps unknown pins, PENIOT pokes at IoT, RouterSploit breaks routers."

3.34 How to Defend Against IoT Hacking

What it is: A 14-item defense checklist for securing IoT devices and the networks they sit on.

How it works (14 defenses):

1. Disable "guest" and "demo" user accounts if enabled.
2. Use the "Lock Out" feature to lock out accounts after excessive invalid login attempts.
3. Implement strong authentication mechanisms.
4. Locate control system networks/devices behind firewalls, isolated from the business network.
5. Implement IPS and IDS in the network.
6. Implement end-to-end encryption and use Public Key Infrastructure (PKI).
7. Use VPN architecture for secure communication.
8. Deploy security as a unified, integrated system.
9. Allow only trusted IP addresses to access the device from the internet.
10. Disable Telnet (port 23).
11. Disable the UPnP port on routers.
12. Protect the devices against physical tampering.

13. Patch vulnerabilities and update device firmware regularly.
14. Monitor traffic on port 48101, since infected devices attempt to spread malicious files using this port.

Real-world use: An IT team rolling out 200 new IoT sensors across a factory floor uses this exact 14-item list as their pre-deployment checklist, disabling guest accounts, enforcing lockouts, disabling Telnet and UPnP, and placing the whole sensor network behind a dedicated firewall before a single device goes live.

Tools and commands used here: firewalls, IDS/IPS, VPN, PKI (concept-level controls, not single named tools).

Countermeasures and defense: this list IS the countermeasure set; apply all 14 items as a baseline, then layer on device-specific hardening from sections 3.35-3.38.

EXAM FOCUS: Item 14 (port 48101) is a specific, memorable number the exam may test directly, since it is tied to a real malware propagation behavior rather than a general best practice.

Memory trick: "Lock accounts, wall off the network, encrypt everything, trust only known IPs, kill Telnet and UPnP, patch often, and watch port 48101."

3.35 General Guidelines for IoT Device Manufacturers

What it is: A 7-item set of basic security measures aimed specifically at the people who BUILD IoT devices, not the people who deploy them.

How it works (7 guidelines):

1. Use SSL/TLS for all communication.
2. Maintain a mutual check on SSL certificates and the certificate revocation list.
3. Encourage the use of strong passwords.
4. Make the device's update process simple and secure, with a chain of trust.
5. Implement account lockout mechanisms after a certain number of wrong login attempts, to prevent brute-force attacks.
6. Lock devices down whenever and wherever possible to prevent attacks, and periodically check for unused tools, using whitelisting to allow only trusted tools/applications to run.
7. Use a secure boot chain to verify all software executed on the device.

Real-world use: A smart-lock manufacturer builds its product with a secure boot chain (guideline 7) that refuses to run any firmware not signed by the company's own key, directly preventing the Firmware Mod Kit-style persistence attack described in section 3.31.

Tools and commands used here: none (manufacturer-level design guidance, not an attacker/defender tool).

Countermeasures and defense: this entire list is countermeasure guidance; it complements the deployment-focused checklist in section 3.34 by addressing security built INTO the device itself rather than how it is configured after purchase.

EXAM FOCUS: Distinguish this manufacturer-facing list from section 3.34's deployment-facing list; a question describing "secure boot chain" or "chain of trust for updates" points to manufacturer guidelines, not deployment checklist items.

Memory trick: "Build it locked, signed, and whitelisted from day one."

3.36 OWASP Top 10 IoT Vulnerabilities Solutions

What it is: The matched solution set for each of the 10 OWASP IoT threats first listed in section 3.5, giving a concrete fix for every numbered problem.

How it works (10 matched solutions):

1. **Weak/Guessable/Hardcoded Passwords:** use Automated Password Management (APM); use strong, complex passwords; avoid hardcoded passwords.
2. **Insecure Network Services:** close open network ports; disable UPnP; encrypt data prior to TLS communication.
3. **Insecure Ecosystem Interfaces:** conduct periodic assessment of interfaces; perform sanity checking and output filtering; use strong passwords with two-factor authentication.
4. **Lack of Secure Update Mechanisms:** verify the source and integrity of updates; encrypt communication between endpoints; notify end users about security updates.
5. **Use of Insecure/Outdated Components:** monitor regularly for unmaintained components; remove unused dependencies/unnecessary features; avoid third-party software from a compromised supply chain.
6. **Insufficient Privacy Protection:** minimize data collection; anonymize collected data; let end users decide what data is collected.
7. **Insecure Data Transfer and Storage:** encrypt communication between endpoints; maintain proper SSL/TLS implementations; avoid proprietary encryption solutions.
8. **Lack of Device Management:** enable account lockout; blacklist malicious devices from suspicious sources; validate all asset attributes; securely decommission retired devices.
9. **Insecure Default Settings:** change default usernames and passwords; let users customize privacy/security settings; disable remote access to IoT devices when not in use.
10. **Lack of Physical Hardening:** set a unique password for BIOS/firmware; configure boot order to prevent unauthorized booting; minimize external ports such as USB.

Real-world use: A product security review for a new smart speaker checks each of the 10 solutions against the actual device: confirms unique per-unit passwords are enforced (solution 1), that unused USB debug ports have been removed from the production board (solution 10), and that firmware updates are both signed and delivered over TLS (solution 4).

Tools and commands used here: none directly (policy/design solutions); Automated Password Management (APM) systems and TLS/PKI tooling support several items.

Countermeasures and defense: this entire section IS the countermeasure list; use it as the direct answer key to section 3.5's threat list.

EXAM FOCUS: The exam may give a threat from section 3.5 and ask for its solution from this section, or vice versa; practice matching both directions.

Memory trick: Each solution echoes its threat's own wording: "weak password -> strong password," "insecure update -> verify/ encrypt update," "insecure default -> change the default."

3.37 IoT Hardware Security Best Practices

What it is: A 12-item checklist of hardware-level (physical and firmware-adjacent) security practices for IoT devices, complementing the network/software-focused lists in sections 3.34-3.36.

How it works (12 practices):

1. Limit the entry points (physical ports/interfaces).
2. Employ a hardware tamper-protection mechanism.
3. Monitor secure booting.
4. Implement security patches.
5. Maintain a proper interface management system.
6. Avoid open access to the hardware unit.
7. Secure authentication keys.
8. Maintain a proper event-logging mechanism.
9. Maintain a proper anti-malware protection system.
10. Protect device access credentials.
11. Isolate devices from regular supply units (power).
12. Implement a root-of-trust mechanism.

Real-world use: A medical-device manufacturer builds a root-of- trust chip (practice 12) into its infusion pump, tamper-evident casing (practice 2), and a dedicated, isolated power supply (practice 11) so that tampering with the device's power cannot be used to trigger a fault-injection attack (section 3.16) against its security logic.

Tools and commands used here: none (hardware design/operations checklist).

Countermeasures and defense: this list IS the countermeasure set; treat it as the hardware-layer companion to section 3.34's network- layer checklist and section 3.35's manufacturer design guidelines.

EXAM FOCUS: "Root-of-trust" (item 12) is a specific, frequently tested security architecture term: a hardware-anchored starting point that every later boot/software check can trust, because it cannot itself be modified.

Memory trick: "Limit, protect, monitor, patch, manage, lock, secure, log, scan, guard, isolate, anchor trust in hardware."

3.38 IoT Device Management

What it is: IoT device management is the use of software tools and processes to onboard, track, monitor, manage, and remotely update large fleets of physical IoT devices securely and promptly, while also enforcing permissions and security capabilities.

How it works (3 functions):

1. Supports IoT solutions through software tools/processes, helping onboard new devices securely and promptly.
2. Lets users track, monitor, and manage physical IoT devices, and forces remote firmware updates across the fleet.
3. Provides permissions and security capabilities to protect devices against vulnerabilities.

Real-world use: A retail chain with thousands of smart checkout terminals uses an IoT device management platform to push a critical security patch to every terminal overnight, confirm which terminals successfully updated, and automatically flag any terminal still running outdated, vulnerable firmware the next morning.

Tools and commands used here: named IoT device management solutions: SeaCat.io, Armis Centrix, Oracle Fusion Cloud Internet of Things (IoT), Golioth, AWS IoT Device Management, IBM Watson IoT Platform, openBalena, and Azure IoT Central (shown in the slide's "Northwind Traders checkout dashboard" example).

Countermeasures and defense: centralizing device management through one of these platforms is itself the countermeasure: it turns scattered, hard-to-patch devices into a fleet that can be audited and updated consistently.

EXAM FOCUS: The exam may ask you to recognize that IoT device management platforms are primarily a DEFENDER tool (for inventory, patching, and permissions at scale), not an attacker tool, which distinguishes this section from the hacking tools in sections 3.19- 3.33.

Memory trick: "One dashboard, every device, patched on time."

3.39 What is OT?

What it is: Operational Technology (OT) is the software and hardware designed to detect or cause changes in industrial operations through direct monitoring and/or controlling of physical industrial devices. OT consists of Industrial Control Systems (ICS), which include Supervisory Control and Data Acquisition (SCADA), Remote Terminal Units (RTU), Programmable Logic Controllers (PLC), and Distributed Control Systems (DCS), used to monitor and control industrial operations. ICS itself is an umbrella term covering all of these control-system types plus their associated equipment: systems, devices, networks, and controls used to automate industrial processes, including SCADA, DCS, Basic Process Control Systems (BPCS), Safety Instrumented Systems (SIS), Human Machine Interfaces (HMI), PLCs, RTUs, and Intelligent Electronic Devices (IEDs).

Analogy: If IT is the office building's computer network, OT is the factory floor's nervous system: it does not process emails, it directly opens valves, spins motors, and reads physical sensors in real time.

How it works (component roles):

1. **SCADA:** supervises and gathers data from an entire industrial process across potentially many sites.
2. **DCS:** controls a single, typically large and complex production process from one dedicated control location.
3. **PLC:** a ruggedized, purpose-built industrial computer that directly runs the control logic for a specific machine or process.
4. **RTU:** a field device that collects data from sensors and relays it back to a SCADA system, often over long distances.
5. **HMI:** the screen-based interface that lets a human operator see and interact with the industrial process.

Real-world use: A water treatment plant uses a SCADA system to supervise the entire plant, DCS-style controllers to manage the chemical dosing process precisely, PLCs to directly control individual pumps and valves, RTUs to report tank levels from remote sites miles away, and an HMI screen in the control room for operators to watch it all happen.

Tools and commands used here: none (concept-only slide).

Countermeasures and defense: treat every ICS component type (SCADA, DCS, PLC, RTU, HMI) as a distinct, named asset in any security inventory, since each has different attack surfaces (see sections 3.44, 3.48, 3.50-3.51).

EXAM FOCUS: ICS is the UMBRELLA term; SCADA, DCS, PLC, RTU, and HMI are each a SPECIFIC TYPE of ICS component. The exam often asks you to identify which specific component a scenario describes.

Memory trick: "ICS is the family name; SCADA, DCS, PLC, RTU, and HMI are the family members."

3.40 IT/OT Convergence (IIoT)

What it is: IT/OT convergence is the integration of IT computing systems with OT operation-monitoring systems, bridging the traditional gap between them to improve overall security, efficiency, and productivity. When IoT is applied specifically to industrial operations (monitoring supply chains, manufacturing, and management systems), it is called the Industrial Internet of Things (IIoT); this convergence is also what enables "smart manufacturing," known as Industry 4.0.

Analogy: IT and OT used to be two separate countries that rarely spoke to each other; convergence builds a bridge between them so data from the factory floor (OT) can flow straight into cloud analytics and business systems (IT), and vice versa.

How it works (data flow):

1. Mechanical devices, SCADA, DCS, PLCs, and RTUs (the OT side) generate operational data.
2. That data flows across OT networks into storage and data processing systems, often written using SQL, Java, Python, and similar IT-side languages and tools.
3. The processed data continues across IT networks up to the cloud and the internet, improving security, quality, and productivity across the whole organization.

Real-world use: A manufacturing plant feeds real-time machine sensor data (OT) directly into a cloud-based analytics dashboard (IT), letting managers predict equipment failures before they happen, exactly the Industry 4.0 promise of IT/OT convergence.

Tools and commands used here: none (concept-only slide).

Countermeasures and defense: convergence also means OT's attack surface grows to include IT-style threats (and vice versa); apply the Purdue Model's zoning (section 3.41) to keep this bridge controlled rather than wide open.

EXAM FOCUS: Know the term IIoT specifically: it is IoT applied TO industrial operations, not a separate technology from IoT itself; the exam may test whether you can distinguish IIoT (industrial use case) from general consumer IoT.

Memory trick: "Converge IT and OT, and you get Industry 4.0 and IIoT."

3.41 The Purdue Model

What it is: The Purdue Model, derived from the Purdue Enterprise Reference Architecture (PERA), is a widely used reference model describing the internal connections and dependencies between important components in ICS networks. It splits the network into three zones (Manufacturing/OT zone, an Industrial DMZ, and an Enterprise/IT zone), themselves divided into levels.

Analogy: Think of a tall building with strict floor access rules: the ground floor (physical process) only talks to the floor directly above it, and nobody from the top-floor executive offices (enterprise IT) can walk straight down to the factory floor without passing through a guarded lobby (the DMZ) in between.

How it works (6 levels in 3 zones, bottom to top):

1. **Level 0 - Physical Process (Manufacturing/OT zone):** the actual physical equipment and process.
2. **Level 1 - Basic Controls/Intelligent Devices (Manufacturing/OT zone):** the lowest-level controllers directly touching the process.
3. **Level 2 - Control Systems/Area Supervisory Controls (Manufacturing/OT zone):** supervisory control over a specific area.
4. **Level 3 - Operation Systems/Site Operations (Manufacturing/OT zone):** site-wide operational management.
5. **Industrial Demilitarized Zone (IDMZ):** a buffer zone separating OT from IT, so neither side connects directly to the other.
6. **Level 4 - Business Logistics Systems (Enterprise/IT zone):** business-side planning and logistics.
7. **Level 5 - Enterprise Network (Enterprise/IT zone):** the top-level corporate IT network.

Real-world use: A security architect reviewing a chemical plant's network diagram uses the Purdue Model to confirm that Level 1 PLCs never talk directly to the Level 5 enterprise network; all traffic between the two zones must pass through and be inspected at the IDMZ.

Tools and commands used here: none (reference architecture model); section 3.70 maps specific security controls onto each Purdue level.

Countermeasures and defense: this model IS the foundational countermeasure for OT network design; strict level-to-level segmentation, with no direct Level 0/1 to Level 4/5 connections, is the single most repeated OT security principle in this module.

EXAM FOCUS: Memorize the exact level numbers and what each one means; the exam commonly asks "which Purdue level is this" given a scenario description (such as "the business planning system" = Level 4).

Memory trick: "0 is the machine, 1 is its controller, 2 supervises the area, 3 runs the site, the DMZ guards the middle, 4 runs the business, 5 is the whole enterprise."

3.42 OT Technologies and Protocols (by Purdue Level)

What it is: The specific network protocols and technologies commonly found at each Purdue Model level, since OT networks do not use the same protocol stack as a typical IT network.

How it works (protocols grouped by level):

1. **Levels 4-5 (Enterprise/IT):** DCOM, FTP/SFTP, GE-SRTP, IPv4/ IPv6, OPC UA, TCP/IP, Wi-Fi, SMTP, HTTP/HTTPS.
2. **Level 3 (Operations/Site):** CC-Link, GE-SRTP, HSCP, ICCP (IEC 60870-6), IEC 61850, IEC 60870-5-104, ISA/IEC 62443, Modbus, NTP, Profinet, SuiteLink, Tase-2, TCP/IP, ControlNet, Profibus PA/DP.
3. **Level 2 (Area Supervisory Control):** 6LoWPAN, CC-Link, DNP3, DNS/DNSSEC, FTE, HART-IP, IEC 60870-5-101/104, IPv4/IPv6, ISA/IEC 62443, OPC, NTP, SOAP, TCP/IP, DeviceNet, AS-Interface (AS-i).
4. **Levels 0-1 (Basic Controls/Physical Process):** BACnet, EtherCAT, CANopen, Crimson v3, GE-SRTP, Zigbee, ISA/IEC 62443, ISA SP100, MELSEC-Q, Modbus, Niagara Fox, Omron FINS, PCWorx, Profibus, Profinet, Sercos II, S7 Communications, WiMax, FOUNDATION Fieldbus.

Real-world use: A protocol analyst inspecting traffic at Level 1 of a manufacturing network expects to see Modbus or Profinet frames between a PLC and its field devices, not SMTP or HTTPS, which would only be normal much higher up at Levels 4-5; seeing enterprise protocols appear at Level 0-1 is itself a red flag for a segmentation failure.

Tools and commands used here: full protocol breakdowns and known weaknesses are in section 6.2 (Ports, Protocols, Standards, and Acronyms); Wireshark/Nmap usage against several of these protocols (Modbus, S7, BACnet, EtherNet/IP) is covered in sections 3.58-3.59.

Countermeasures and defense: monitor for protocol anomalies per Purdue level (a protocol appearing where it should not is a strong signal of lateral movement or misconfiguration).

EXAM FOCUS: The exam often names a specific protocol (BACnet, DNP3, Modbus, S7 Communications) and asks which Purdue level or zone it is typically associated with; use this list to anchor Modbus/S7/ Profinet/BACnet at Levels 0-2, and OPC UA/HTTP(S) at Levels 3-5.

Memory trick: "The closer to the machine, the more specialized the protocol; the closer to the business, the more it looks like ordinary IT traffic."

3.43 Challenges of OT

What it is: A 22-item list of structural, organizational, and technical challenges that make OT networks harder to secure than typical IT networks.

How it works (22 challenges, grouped):

1. Lack of visibility.
2. Plain-text passwords.
3. Network complexity.
4. Legacy technology.
5. Lack of anti-virus protection.
6. Lack of skilled security professionals.
7. Rapid pace of change.
8. Outdated systems.
9. Haphazard modernization.
10. Insecure connections.
11. Usage of rogue devices.
12. Convergence with IT.
13. Organizational challenges.
14. Unique production networks/proprietary software.
15. Vulnerable communication protocols.
16. Remote management protocols.
17. Insufficient segmentation.
18. Physical security.
19. Vendor dependencies.
20. Resource constraints.
21. Lack of encryption.
22. Data integrity issues.

Real-world use: A newly hired OT security lead quickly discovers that the plant runs 15-year-old PLCs (legacy technology, challenge 4) that cannot run any endpoint security software (lack of anti-virus, challenge 5), communicate in plaintext (challenge 2), and sit on a flat network with no segmentation between the office and the factory floor (challenge 17) - a realistic combination of several of these 22 challenges at once.

Tools and commands used here: none (organizational/technical challenge list).

Countermeasures and defense: prioritize fixing network segmentation (challenge 17) and visibility (challenge 1) first, since most other OT attacks in this module (sections 3.48-3.67) depend on an attacker being able to reach and see the OT network directly.

EXAM FOCUS: This list explains WHY OT security differs fundamentally from IT security; a question may ask "which of the following is a reason OT is harder to secure than IT," and any of these 22 items (especially legacy tech, lack of antivirus, and lack of segmentation) is a valid answer.

Memory trick: Group into 4 clusters: people/process challenges (skills, organization, pace of change), technology-age challenges (legacy, outdated, unique/proprietary), network challenges (complexity, segmentation, rogue devices, convergence), and security hygiene challenges (passwords, encryption, visibility, antivirus, physical security).

3.44 OT Vulnerabilities (10 Named Items)

What it is: A detailed, 10-item list of specific technical vulnerabilities commonly found in real OT networks.

How it works (10 vulnerabilities):

1. **Publicly Accessible OT Systems:** ability to password-brute- force or probe OT systems directly, to disable or disrupt their functions.
2. **Insecure Remote Connections:** ability to exploit vulnerabilities in jump boxes to gain remote access to OT systems.
3. **Missing Security Updates:** outdated software versions increase risk and pave the way to compromise OT systems.
4. **Weak Passwords:** ability to access OT systems if default vendor credentials on embedded devices/management interfaces are never changed.
5. **Insecure Firewall Configuration:** insecure firewalls propagate security threats into the OT network, making it vulnerable.
6. **OT Systems Placed within the Corporate IT Network:** ability to use a compromised IT system to reach the OT network.
7. **Insufficient Security for Corporate IT Network from OT Systems:** ability to gain unauthorized access to corporate IT systems THROUGH insecure OT devices (the reverse direction of item 6).
8. **Lack of Segmentation within OT Networks:** a flat, unsegmented OT network assumes all systems are equally important, so compromising just one device can expose the entire OT network.
9. **Lack of Encryption and Authentication for Wireless OT Networks:** ability to perform sniffing and authentication-bypass attacks.
10. **Unrestricted Outbound Internet Access from OT Networks:** susceptibility to malware and command-and-control attacks.

Real-world use: An attacker who phishes a single corporate laptop (an IT-side compromise) discovers the OT network was never properly segmented from IT (vulnerability 6/8 combined), and simply pivots straight from that laptop into the plant's control systems with no further exploit needed.

Tools and commands used here: none directly named; recon/ exploitation tools for finding these specific vulnerabilities appear in sections 3.56-3.67.

Countermeasures and defense: see section 3.69 for the matched solution to each numbered vulnerability.

EXAM FOCUS: Vulnerabilities 6 and 7 look similar but point in OPPOSITE directions: 6 is IT-compromise-reaches-OT; 7 is OT- compromise-reaches-IT. The exam may test which direction a scenario describes.

Memory trick: "Open to the internet, weakly guarded, weakly walled off, weakly encrypted, and too trusting of outbound traffic."

3.45 MITRE ATT&CK for ICS (Part 1)

What it is: MITRE ATT&CK for ICS is a published, structured framework cataloging the real-world methods and techniques an attacker uses to establish and maintain a foothold inside a targeted ICS environment, organized into named tactics. This first half covers 6 of the framework's tactics.

How it works (6 tactics):

1. **Initial Access:** methods to first get into the ICS environment, such as drive-by compromise, exploiting a public-facing software application, or exploiting remote services.
2. **Execution:** techniques to run malicious code, manipulate data, or trigger other system functions illegitimately, such as changing the operating mode, using the command-line interface (CLI), or executing through APIs.
3. **Persistence:** procedures that let an attacker retain access even if the compromised device restarts or communication is interrupted, such as modifying a program, inserting module firmware, executing through APIs, or modifying project files.
4. **Privilege Escalation:** gaining higher-level access/authorization to perform further malicious activity, such as through software exploitation or hooking.
5. **Evasion:** techniques to dodge traditional defense mechanisms throughout an operation, such as removing indicators, deploying rootkits, or changing the operator mode.
6. **Discovery:** the process of gathering information about an ICS environment to assess and identify target assets, such as removing indicators, enumerating network connections, sniffing the network, or identifying remote systems.

Real-world use: An attacker exploits a public-facing engineering workstation's remote service (Initial Access), changes a PLC's operating mode via its API (Execution), modifies the PLC's own project file so the change survives a restart (Persistence), exploits a software bug to gain admin rights on the engineering workstation (Privilege

Escalation), removes log entries of their activity (Evasion), and sniffs the network to map out every other connected device (Discovery) - working through all 6 tactics in sequence.

Tools and commands used here: none named directly on this slide; this is the attacker PLAYBOOK, not a single tool; many module tools (Shodan, Metasploit, modbus-cli) map onto specific techniques within these tactics.

AI angle: ADDED CONTEXT: defenders increasingly feed MITRE ATT&CK for ICS technique IDs into AI-assisted SOC (Security Operations Center) correlation tools, which can automatically flag when a sequence of OT network events matches a known ATT&CK tactic chain (such as Discovery immediately followed by Execution).

Countermeasures and defense: map your own detection capability against each of these 6 tactics to find coverage gaps; this framework is also referenced by Stratus Red Team in other modules as a way to safely test exactly these techniques.

EXAM FOCUS: Memorize the tactic NAMES and their one-line meaning; the exam may describe a specific technique (such as "modifying a program so it survives a reboot") and ask which TACTIC it belongs to (here, Persistence).

Memory trick: "Get in, do something, stay in, climb higher, hide, and look around" - Initial Access, Execution, Persistence, Privilege Escalation, Evasion, Discovery.

3.46 MITRE ATT&CK for ICS (Part 2)

What it is: The second half of the MITRE ATT&CK for ICS framework, covering the remaining tactics focused on moving through the environment and ultimately causing real-world physical impact.

How it works (6 more tactics):

1. **Lateral Movement:** additional movement across the target ICS environment by leveraging existing access, such as using default credentials, program downloads, or remote services.
2. **Collection:** methods to gather information and knowledge about the ICS infrastructure's data and domains, such as automated collection, information repositories, or I/O images.
3. **Command and Control (C2):** techniques to deactivate, control, or exploit physical control processes using command and control, such as frequently used ports, connection proxies, or standard application-layer protocols.
4. **Inhibit Response Function:** ways to thwart reactions to a security event such as a hazard or failure, such as activating firmware update mode, or blocking command/reporting messages.

5. **Impair Process Control:** tactics to disable, exploit, or control physical control processes in the target environment, such as I/O brute-forcing, altering parameters, or injecting module firmware.
6. **Impact:** techniques to damage, disrupt, or gain control of the target ICS environment's data, systems, and surroundings, such as damage to property, loss of availability, or denial of control.

Real-world use: After gaining initial access (section 3.45), an attacker uses default credentials to move laterally to a second PLC (Lateral Movement), reads I/O images to understand the physical process better (Collection), sets up command-and-control traffic over a commonly allowed port (Command and Control), blocks the plant's alarm reporting messages right before acting (Inhibit Response Function), alters a critical process parameter (Impair Process Control), and ultimately causes physical equipment damage (Impact).

Tools and commands used here: none named directly; see sections 3.65-3.67 for specific exploitation tools (Metasploit modbus modules, modbus-cli, DNP3 access) that implement techniques within these tactics.

Countermeasures and defense: "Inhibit Response Function" and "Impair Process Control" are specifically OT/physical-process tactics with no direct IT equivalent; prioritize detecting these two, since by the time they occur, physical safety may already be at risk.

EXAM FOCUS: The exam may ask which MITRE ATT&CK for ICS tactic is UNIQUE to OT and has no close equivalent in the general enterprise MITRE ATT&CK framework; "Inhibit Response Function" and "Impair Process Control" are the best answers, since they describe tampering with physical safety/process responses, not generic IT behavior.

Memory trick: "Move sideways, collect, command from afar, silence the alarms, twist the dials, and finally, break something."

3.47 OT Threats Overview (14 Named Threats)

What it is: The module's master list of 14 named threat categories against OT systems, most of which get their own detailed topic section later (cross-referenced below). Most OT systems run legacy, outdated software with no built-in security protection, leaving a potential gateway for cyber criminals to reach both the corporate IT network and the OT infrastructure itself.

How it works (the 14 threats):

1. Maintenance and Administrative Threat.
2. Data Leakage.
3. Protocol Abuse.

4. Potential Destruction of ICS Resources.
5. Reconnaissance Attacks (section 3.56-3.60).
6. Denial-of-Service Attacks.
7. HMI-based Attacks (section 3.48).
8. Exploiting Enterprise Specific Systems and Tools.
9. Spear Phishing.
10. Malware Attacks (section 3.55).
11. Exploiting Unpatched Vulnerabilities.
12. Side-Channel Attacks (section 3.49).
13. Buffer Overflow Attacks.
14. Exploiting RF Remote Controllers (sections 3.52-3.53).

Real-world use: A threat-modeling workshop for a power substation walks through this 14-item list and identifies that Protocol Abuse (item 3, exploiting the fact that Modbus/DNP3 has no authentication) and HMI-based Attacks (item 7) are the two most realistic near-term threats, given the substation's current unencrypted SCADA setup.

Tools and commands used here: see each threat's dedicated section for its specific tools where one exists.

Countermeasures and defense: see sections 3.68-3.71 for the full OT defense checklist, matched vulnerability solutions, Purdue-based security controls, and the Zero Trust model.

EXAM FOCUS: "Maintenance and Administrative Threat," "Data Leakage," "Protocol Abuse," and "Exploiting Enterprise Specific Systems and Tools" are named here but not expanded with their own dedicated diagram slide in this module; know their one-line meaning: Maintenance and Administrative Threat (risk introduced by maintenance staff/contractors with excess access), Data Leakage (sensitive process data unintentionally exposed), Protocol Abuse (exploiting that OT protocols like Modbus/DNP3 lack authentication), Exploiting Enterprise Specific Systems and Tools (abusing legitimate IT/business tools that also touch the OT environment).

Memory trick: Group into 4 clusters: human/process threats (maintenance, phishing, data leakage), technical exploitation threats (protocol abuse, unpatched vulnerabilities, buffer overflow), disruption threats (DoS, destruction of resources, malware), and targeted attacks (recon, HMI, side-channel, RF controllers).

3.48 HMI-based Attacks

What it is: Attacks that specifically target the Human Machine Interface (HMI), the software/screen that lets operators monitor and control an industrial process, because the HMI is the core hub controlling critical infrastructure. Attackers target HMIs either to cause physical damage to SCADA devices or to collect sensitive information about the critical architecture.

Analogy: The HMI is the cockpit of an airplane: you don't need to touch the engines directly if you can just alter what the pilot sees and controls from the cockpit dashboard.

How it works (4 SCADA vulnerability categories exploited):

1. **Memory Corruption:** attackers exploit code security issues including out-of-bounds read/write vulnerabilities, plus heap- and stack-based buffer overflows.
2. **Credential Management:** attackers abuse hard-coded passwords and credentials stored in cleartext to gain administrative privileges.
3. **Lack of Authorization/Authentication and Insecure Defaults:** attackers exploit confidential information transmitted in cleartext, insecure default settings, and unsafe ActiveX controls.
4. **Code Injection:** attackers exploit critical information transmitted in cleartext, insecure defaults, missing encryption, and insecure ActiveX controls to gain illegal access to the target system.

Real-world use: The slide's own example shows a SIELCO Sistemi "Winlog" HMI screen monitoring 4 tanks (temperature, flow rate, level, and mixer speed); an attacker exploiting a known HMI software vulnerability in this exact kind of product could silently change a tank's displayed temperature reading, masking a real, dangerous overheating condition from the operator watching the screen.

AI angle: ADDED CONTEXT: attackers can use AI-generated deepfake audio or video to impersonate a plant manager instructing an HMI operator to make an unsafe change over the phone or a video call, combining this technical HMI attack surface with AI-powered social engineering.

Tools and commands used here: no specific named exploitation tool on this slide; vulnerabilities map to general categories (memory corruption, credential management, missing auth, code injection) rather than one single product.

Countermeasures and defense: patch HMI/SCADA software regularly against known memory-corruption CVEs; never store credentials in cleartext; enforce authentication on every HMI session; disable or replace legacy ActiveX controls with modern, sandboxed alternatives.

EXAM FOCUS: Tell the 4 categories apart by root cause: Memory Corruption = a CODING bug; Credential Management = a cleartext/ hardcoded SECRET; Lack of Auth/ Insecure Defaults = a MISSING CHECK; Code Injection = UNVALIDATED INPUT reaching executable logic.

Memory trick: "Broken code, exposed secrets, missing locks, and poisoned input - four ways into the cockpit."

3.49 Side-Channel Attacks (OT)

What it is: The OT-domain version of a side-channel attack: monitoring the physical implementation of a target OT system (rather than attacking its logic directly) to obtain critical information such as a password, using two techniques: timing analysis and power analysis.

Analogy: Instead of picking a safe's lock, you listen to the clicks the dial makes and time how long each click takes, slowly figuring out the combination without ever touching the lock mechanism itself.

How it works (2 techniques):

1. **Timing Analysis:** the attacker monitors how long the device takes to finish one complete password-authentication attempt, and uses that timing to determine how many characters of the attempted password were correct (a correct prefix typically takes measurably longer to reject than a totally wrong one).
2. **Power Analysis:** the attacker observes changes in the power consumption of the device's semiconductors during each clock cycle, using an oscilloscope or measuring probe connected to an analysis computer; by comparing the power profile of a correct character attempt against a wrong one, the attacker retrieves one character of the password at a time.

Real-world use: A researcher connects an oscilloscope probe to a SCADA system's power rail while repeatedly submitting password guesses; by watching for a subtly different power-consumption pattern on the correct first character versus incorrect ones, they reconstruct the entire password one character at a time, without ever triggering a failed-login lockout counter that monitors network traffic alone.

Tools and commands used here: oscilloscope or measuring-device probe, attacker's computer with analysis software (concept-level specialized hardware; no single named software tool on this slide).

Countermeasures and defense: use constant-time password comparison logic (so correct and incorrect attempts always take identical time/power); add physical shielding against power/EM analysis on sensitive control hardware; rate-limit and lock out authentication attempts regardless of timing.

EXAM FOCUS: This is the OT-domain twin of the IoT side-channel attack in section 3.17 (which leaks ENCRYPTION KEYS via emitted signals); here the specific target is a PASSWORD, extracted via TIMING or POWER patterns. A question may ask you to name the two specific techniques (timing analysis, power analysis).

Memory trick: "Listen to how long it thinks, or how hard it breathes, while it checks each letter of your password."

3.50 Hacking Programmable Logic Controller (PLC)

What it is: PLCs are especially attractive targets because they directly control the physical processes of critical infrastructure. Attackers find internet-exposed PLCs using tools like Shodan, then can tamper with their integrity and availability by exploiting pin control operations, including through a PLC rootkit attack.

Analogy: A PLC is the hand that actually flips the physical switch; a rootkit planted on that hand can make it flip switches on its own, regardless of what the control room thinks it told the hand to do.

How it works (PLC Rootkit Attack, pre-condition + 3 steps):

Pre-condition: the attacker identifies the physical process controlled by the PLC, and identifies the mapping between the PLC's I/O pins and its internal logic.

1. **Gaining Access (step 1):** the attacker gains authorized access to the PLC by injecting a rootkit (1.1), launches a control-flow attack against the PLC runtime (1.2), or gains access by simply guessing the PLC's default password (1.3).
2. **Attack Step 2:** the attacker maps the I/O for debugging the register, and intercepts read/write (R/W) operations.
3. **Attack Step 3:** the attacker modifies the I/O initialization sequence, which leads to the final consequence: the attacker gets root access to the PLC and manipulates its real-world operations.

Real-world use: An attacker guesses a water treatment plant's PLC's default password (step 1.3), maps which I/O pin controls a chlorine dosing valve (step 2), and modifies the valve's initialization sequence (step 3) so that it silently opens further than the operator's display shows, all while the HMI continues to report a normal reading.

Tools and commands used here: no single named exploitation tool on this slide; recon via Shodan (section 3.56); exploitation building blocks via Metasploit's Modbus modules and modbus-cli (sections 3.65- 3.66) implement the underlying read/write operations described here.

Countermeasures and defense: change default PLC passwords immediately; restrict who can physically or remotely reconfigure I/O mappings; monitor PLC logic/ configuration for unexpected changes (configuration drift detection).

EXAM FOCUS: This PLC Rootkit Attack directly manipulates the PLC's own I/O/logic; contrast it carefully with the Evil PLC Attack (section 3.51), which instead uses a compromised PLC as a weapon against connecting WORKSTATIONS.

Memory trick: "Guess the password, map the pins, rewrite the startup sequence - now the hand moves on its own."

3.51 Evil PLC Attack

What it is: The Evil PLC attack flips the usual direction of a PLC compromise: instead of just controlling the PLC itself, the attacker turns a found, vulnerable PLC into a weapon that attacks the engineering WORKSTATIONS that connect to program or monitor it.

Analogy: Instead of a hacked ATM machine just stealing cash, imagine the ATM is reprogrammed so that the NEXT bank technician who plugs their laptop in to service it gets infected instead.

How it works (4 steps):

1. The attacker identifies a vulnerable PLC exposed online using Shodan.
2. If a vulnerable PLC is found, the attacker turns that ordinary PLC into an "Evil PLC" by modifying its configuration settings, changing its behavior and logic, through the PLC's own normal download/upload procedures.
3. The now-malicious PLC's configuration change is designed to trigger when connected workstations initiate upload procedures against it (pulling the PLC's program back to the engineer's machine).
4. Each victim workstation that connects to the compromised PLC for normal engineering work is itself attacked/compromised as a result.

Real-world use: A plant engineer plugs their laptop into what looks like a normal, working PLC to pull its current program for routine maintenance; because the PLC was already turned "Evil" by a prior attacker, the engineer's own workstation is compromised the moment the upload procedure runs, giving the attacker a fresh foothold on the IT-adjacent engineering network.

Tools and commands used here: Shodan (identifying the vulnerable PLC); the PLC's own native configuration/download procedures (abused, not a separate named attacker tool).

Countermeasures and defense: never connect an engineering workstation to an untrusted or previously-unmonitored PLC without verification; use workstation endpoint protection capable of detecting malicious payloads delivered through engineering software upload/download procedures; keep PLCs off the public internet.

EXAM FOCUS: The exam's favorite trap here is confusing direction: Evil PLC attack = PLC attacks the WORKSTATION; PLC Rootkit Attack (section 3.50) = attacker attacks the PLC directly. If a scenario describes an engineer's laptop getting infected FROM a PLC, it is the Evil PLC attack.

Memory trick: "An Evil PLC waits patiently for someone trustworthy to plug in, then bites back."

3.52 Hacking Industrial Systems through RF Remote Controllers: Replay and Command Injection

What it is: Many industrial machines (manufacturing, logistics, mining, construction) are operated by RF remote controllers; improper security in these controllers exposes the machines to serious risk. This section covers the first two of four named attack techniques against them.

How it works (2 techniques):

1. **Replay Attack:** the attacker records the RF commands transmitted by a legitimate operator while within transmission (TX) range of the target machine, captures that data, and later transmits the recorded commands back to the machine to gain basic control over it.
2. **Command Injection:** the attacker captures RF data from the operator's commands, then uses offline reverse engineering of the captured packets to understand their structure, and either alters existing RF packets or injects entirely new, attacker-crafted packets to gain complete access over the target machine (going beyond simply replaying what was captured).

Real-world use: An attacker near a construction site records a crane operator's "lower boom" RF command (Replay Attack setup), then later transmits that exact recording to make the crane lower its boom on command; a more advanced attacker instead reverse engineers the crane's command packet format (Command Injection) to craft entirely new commands the operator never actually sent, such as "raise boom past its safety limit."

Tools and commands used here: SDR-class hardware/software (concept-level, similar in spirit to HackRF One/RFCrack from sections 3.23-3.26, though no single named tool is specified for this exact industrial-RF slide).

Countermeasures and defense: use rolling/encrypted codes for industrial RF controllers (the same principle as section 3.11's rolling code defense); add command authentication so the machine rejects unsigned or out-of-sequence commands.

EXAM FOCUS: Tell these two apart: Replay Attack = re-sends an UNMODIFIED captured command; Command Injection = CRAFTS new or ALTERED commands after reverse engineering the protocol. Command Injection requires deeper protocol understanding and grants more complete control.

Memory trick: "Replay plays back the same tune; command injection writes a brand new one."

3.53 Hacking Industrial Systems through RF Remote Controllers: Re-pairing and Malicious Reprogramming

What it is: The second pair of named attacks against industrial RF remote controllers, both focused on taking over the control RELATIONSHIP itself rather than just capturing individual commands.

How it works (2 techniques):

1. **Re-pairing with a Malicious RF Controller:** the attacker hijacks the machine's pairing sequence and pairs their OWN malicious RF controller with the target machine, disguising it as the legitimate original controller; after the attack, the attacker's controller (TX1, paired) can command the machine, while the original operator's controller (TX1, unpaired) no longer works.
2. **Malicious Reprogramming Attack:** the attacker injects malware directly into the firmware of the remote controller itself, maintaining persistent, fully remote access to the system from then on, without needing to be physically nearby for future attacks.

Real-world use: A warehouse worker's forklift remote control suddenly stops responding; unknown to them, an attacker nearby just completed a re-pairing attack, replacing the operator's legitimate pairing with their own malicious controller, and can now operate the forklift themselves while the real operator stands there with a dead remote.

Tools and commands used here: specialized RF pairing-sequence capture hardware (concept-level; no single named tool specified on this slide).

Countermeasures and defense: use RF pairing protocols that require physical confirmation (such as a button press on the machine itself) for any new pairing, and that alert operators when an existing pairing is dropped unexpectedly; digitally sign remote controller firmware updates so malicious reprogramming is rejected.

EXAM FOCUS: Malicious Reprogramming Attack is the ONLY one of the 4 RF-controller techniques (across sections 3.52-3.53) that gives the attacker PERSISTENT remote access without needing to stay physically nearby; the other three (Replay, Command Injection, Re-pairing) all require the attacker's RF hardware to be in range at the time of the attack.

Memory trick: "Steal the pairing and the operator goes dark; poison the firmware and the attacker never has to come back."

3.54 OT Supply Chain Attacks

What it is: Attacks that compromise the hardware, software, or services of an organization's SUPPLIERS first, then use that compromised supplier relationship to infiltrate the actual target organization's OT environment.

Analogy: Instead of breaking into a castle's front gate directly, the attacker poisons the flour delivered by the castle's trusted bread supplier.

How it works (5 attack types):

1. **Third-Party Software Compromise:** attackers inject malicious code into trusted software updates, creating backdoors or malicious functionality that activates when installed.
2. **Hardware Manipulation:** attackers alter hardware components during manufacturing or distribution, embedding malicious firmware or chips that can be activated later, after deployment.
3. **Service Provider Breach:** attackers compromise service providers (maintenance or support contractors) to infiltrate the target's OT network, using stolen credentials, remote access tools, or insider access.
4. **Injection of Malicious Components:** attackers introduce malicious components or firmware into the supply chain by tampering during shipping, or by substituting legitimate parts with compromised ones.
5. **Exploitation of Trusted Relationships:** attackers exploit the trust and access levels already granted to suppliers, subcontractors, or partners, to move laterally within the target's network.

Real-world use: A maintenance contractor with standing remote access to a power plant's OT network is itself breached by an attacker (Service Provider Breach); the attacker then uses that contractor's trusted, already-whitelisted remote access path to reach the plant's actual control systems, bypassing the plant's own perimeter defenses entirely.

Tools and commands used here: none named directly; this is an attack-pattern category rather than a tool-driven technique.

Countermeasures and defense: audit and limit supplier/contractor remote access to the minimum needed; require suppliers to meet the same security standards as internal systems; verify hardware/firmware integrity (checksums, signed components) on receipt; monitor for unexpected behavior from vendor-supplied software immediately after updates.

EXAM FOCUS: The exam's giveaway phrase for this category is any mention of "vendor," "supplier," "contractor," or "third-party update" as the attack's ENTRY POINT, rather than a direct exploit against the target's own systems.

Memory trick: "Poison the supply line, and the target welcomes the attacker in through its own front door."

3.55 OT Malware: Fuxnet

What it is: Fuxnet is a destructive industrial control system (ICS) malware purpose-built to disrupt OT environment operations; attackers use it to modify crucial data, block access to sensor gateways, and attempt to physically damage sensors within an OT environment.

How it works (deployment pattern shown in the slide):

1. The attacker builds a target list of sensor router IP addresses (the slide's example file is named `all_sensor_router_ips-ex-civilian.txt`).
2. The attacker pipes that target list into a parallel deployment command, for example `cat all_sensor_router_ips...txt | xargs -P10 -I{} ./fuxnet.sh {}`, running the Fuxnet deployment script against up to 10 targets at once (`-P10`).
3. The attacker deletes the deployment script after use (`rm fuxnet.sh`) to reduce forensic evidence.
4. Each successful deployment is logged by target identifier (the slide shows output like "Deploying Fuxnet to [location/router name]"), confirming which sensor routers/gateways were hit.
5. The malware interacts with each target's own JSON-based configuration data (object UUIDs, complex/region names, object types such as "collector" or "control_room"), letting it map and disrupt specific physical sensor infrastructure precisely.

Real-world use: Fuxnet was reported as targeting Russian sensor- gateway infrastructure as part of the Russia-Ukraine conflict (ADDED CONTEXT, verify current reporting/attribution), illustrating that OT malware in live geopolitical conflicts can target physical sensor networks (such as those monitoring urban infrastructure) directly, not just generic IT systems.

Tools and commands used here: `fuxnet.sh` (deployment script), `xargs -P<n> -I{}` (parallel execution pattern) (full breakdown in section 5.11).

Other named OT malware families: Kapeka, Abyss Locker, AvosLocker, COSMICENERGY, INDUSTROYER.V2, Pipedream (named in this module as additional current OT/ICS-targeting malware families).

Countermeasures and defense: segment sensor/gateway networks so a single compromised router cannot reach the entire sensor fleet; monitor for unusual parallel connection patterns (a hallmark of scripted, multi-target deployment like Fuxnet's `xargs -P10` pattern); maintain offline backups of sensor gateway configuration data.

EXAM FOCUS: Fuxnet's defining exam fact is that it targets PHYSICAL SENSOR infrastructure specifically (sensor gateways and the sensors themselves), not just generic servers; also recognize INDUSTROYER.V2 and Pipedream as other real, named ICS-specific malware the exam may ask you to identify by name alone.

Memory trick: "Fuxnet hunts sensors, not servers - and cleans up its own deployment script behind it."

3.56 Information Gathering: Identifying ICS/SCADA Systems using Shodan

What it is: The OT-domain equivalent of section 3.19's IoT Shodan recon: using Shodan to find ICS/SCADA systems connected to the internet, including systems running water treatment plants, nuclear power plants, HVAC systems, electrical transmission systems, and home heating systems.

How it works (3 search filter examples):

1. Search for Modbus-enabled ICS/SCADA systems by their standard port: `port:502`.
2. Search for SCADA systems by a specific PLC vendor name, for example: `"Schneider Electric"`.
3. Search for SCADA systems by geolocation and country, for example: `SCADA country:"US"`.

Real-world use: A researcher runs `port:502` on Shodan and receives thousands of results worldwide, each showing the device's IP, port, and banner; cross-referencing the top countries and top organizations columns quickly reveals which regions have the most exposed, unauthenticated Modbus devices.

Tools and commands used here: Shodan search syntax: `port:502`, `"Schneider Electric"`, `SCADA country:"<code>"` (full breakdown in section 5.12).

Countermeasures and defense: never expose ICS/SCADA devices directly to the internet; if remote access is required, use a VPN with strong authentication instead; periodically search Shodan for your own organization's ICS footprint.

EXAM FOCUS: `port:502` as a Shodan filter is a direct, memorable fact (502 = Modbus/TCP's default port, confirmed again in section 6.1); expect a question connecting this exact port number to both Shodan recon and Modbus itself.

Memory trick: "If Modbus answers on 502 to the whole internet, Shodan already knows about it."

3.57 Information Gathering: Gathering Default Passwords using CIRT.net

What it is: CIRT.net hosts an online default-password database that stores default credentials for a very wide range of devices, including those used in critical infrastructure, routers, switches, and ICS equipment.

How it works:

1. An attacker (or auditor) searches the CIRT.net default-password database for a specific vendor/product, for example "Siemens Corp - Simatic WinCC SCADA."
2. The database returns the device's documented default username and password (the slide's example shows entries for WinCC Administrator-level credentials).
3. The attacker tries these known defaults against a real, discovered device (found via Shodan or Nmap) to attempt login.

Real-world use: After finding an exposed Siemens WinCC SCADA system via Shodan (section 3.56), an attacker looks up "Siemens WinCC" on CIRT.net, finds its documented factory-default administrator password, and successfully logs in because the operator never changed it.

Tools and commands used here: CIRT.net web-based default- password lookup (no CLI syntax; browser-based search).

Countermeasures and defense: this entire attack class disappears the moment default credentials are changed at installation; treat "default credentials never changed" as the single highest- priority OT finding to remediate.

EXAM FOCUS: CIRT.net is specifically a DEFAULT PASSWORD database covering 531+ vendors and 2117+ passwords (per the slide); contrast it with vulnerability-scanning tools (Nessus, section 3.61) which find a much broader range of issues, not just default credentials. Other named information-gathering tools alongside CIRT.net: Kamerka-GUI, SearchDiggity, Zeek, Criminal IP, ZoomEye.

Memory trick: "Why guess a default password, when a public database already lists it for you?"

3.58 Information Gathering: Scanning ICS/SCADA Systems using Nmap

What it is: A set of 12 specialized Nmap commands, each tuned to detect and gather information from a specific ICS/SCADA protocol, vendor, or device family, going well beyond Nmap's generic port scanning.

How it works (12 scan types):

1. **Identify open ports/services (generic, careful/slow scan):** `nmap -Pn -sT --scan-delay 1s --max-parallelism 1 -p <port_list> <target>` (slowed down deliberately to avoid disrupting fragile ICS devices).
2. **Identify HMI systems:** `nmap -Pn -sT -p 46824 <target>`.
3. **Scan Siemens SIMATIC S7 PLCs:** `nmap -Pn -sT -p 102 --script=s7-info <target>`.
4. **Scan Modbus devices:** `nmap -Pn -sT -p 502 --script modbus-discover <target>`.
5. **Scan BACnet devices:** `nmap -Pn -sU -p 47808 --script bacnet-info <target>`.
6. **Scan Ethernet/IP devices:** `nmap -Pn -sU -p 44818 --script enip-info <target>`.
7. **Scan Niagara Fox devices:** `nmap -Pn -sT -p 1911,4911 --script fox-info <target>`.
8. **Scan ProConOS devices:** `nmap -Pn -sT -p 20547 --script proconos-info <target>`.
9. **Scan Omron PLC devices:** `nmap -Pn -sT -p 9600 --script omcon-info <target>`.
10. **Scan PCWorx devices:** `nmap -Pn -sT -p 1962 --script pcworx-info <target>`.

Real-world use: A penetration tester given authorized scope on a plant network runs the slow, careful generic scan first (`--scan-delay 1s --max-parallelism 1`) to avoid crashing fragile legacy PLCs, then follows up with the Modbus-specific script (`modbus-discover` on port 502) once a live host is confirmed, rather than blasting a full default Nmap scan at ICS equipment.

Tools and commands used here: all 12 Nmap command variants above (full breakdown, with flag tables, in section 5.13).

Countermeasures and defense: monitor for unusual scan traffic on ICS-specific ports (502, 102, 47808, 44818, 1911/4911, 20547, 9600, 1962); rate-limit or block scanning behavior at the network perimeter, since ICS devices often cannot handle aggressive scans safely even when not maliciously intended.

EXAM FOCUS: The exam loves matching an Nmap `--script` name or port number to its protocol/vendor: `s7-info`=Siemens S7 (port 102), `modbus-discover`=Modbus (port 502), `bacnet-info`=BACnet (port 47808), `enip-info`=EtherNet/IP (port 44818), `fox-info`=Niagara Fox (ports 1911/4911), `proconos-info`=ProConOS (port 20547), `omcon-info`=Omron (port 9600), `pcworx-info`=PCWorx (port 1962).

Memory trick: "Each ICS vendor speaks its own language on its own port; Nmap just needs the right script to translate it."

3.59 Information Gathering: Analyzing Modbus/TCP Traffic Using Wireshark

What it is: Using Wireshark to capture and analyze Modbus/TCP traffic on an industrial network, made possible by the fact that Modbus/TCP has no built-in encryption or security features at all.

How it works:

1. Capture traffic on the industrial network segment carrying Modbus/TCP (default port 502).
2. Apply Wireshark's `modbus` display filter to isolate just Modbus packets from everything else.
3. Read a Query/Response pair: the slide's example shows a Query (Transaction ID visible) from one host to another, followed by a Response; expanding the Modbus/TCP layer reveals the Transaction Identifier, Protocol Identifier, Length, Unit Identifier, Function Code (the slide shows "Read Input Registers"), Reference Number, and Word Count fields, all sent in plaintext.

Real-world use: An attacker with any network visibility into a plant's Modbus traffic (no credentials needed) can read every single register value, function code, and unit ID being exchanged between a PLC and its master, because nothing in the Modbus/TCP protocol itself is encrypted or authenticated.

Tools and commands used here: Wireshark with the `modbus` display filter (full breakdown in section 5.14).

Countermeasures and defense: wrap Modbus/TCP traffic in a VPN tunnel or other encrypted transport, since the protocol itself cannot be secured directly; segment Modbus traffic onto an isolated network with no outside visibility.

EXAM FOCUS: "Modbus/TCP does not have any built-in encryption or security features" is a direct, frequently tested exam fact; a question may ask why an attacker can read Modbus traffic so easily with just Wireshark, and the answer is this total lack of built-in protection, not a specific Wireshark exploit.

Memory trick: "Modbus speaks in the clear; Wireshark just has to listen."

3.60 Information Gathering: Discovering ICS/SCADA Network Protocols using Malcolm

What it is: Malcolm is a powerful network traffic analysis tool that gives attackers (and legitimate analysts) insight into exactly which protocols are in use across an ICS/IoT environment, through two intuitive web interfaces: an OpenSearch dashboard and an Arkime interface.

How it works:

1. Malcolm passively ingests network traffic (via Zeek logs) from the ICS/IoT environment.
2. Its dashboard breaks traffic down by protocol with live counts, for example in the slide: Ethernet (100,802), CIP (23,104), Bacnet (16,570), CoAP (12,657), DHCP/DNS/HTTP/TFTP under "Common Protocols," 7comm (10,924), Modbus (6,514), EtherNet/IP and DNP3 under "ICS/IoT Protocols" with smaller counts (such as EtherNet/IP at 3,829 and DNP3 at 224).
3. A source-country breakdown (the slide shows Ukraine-related IP entries) lets an analyst see where the traffic is geographically originating from.

Real-world use: A defender deploys Malcolm on a span port monitoring their OT network and immediately discovers unexpected CIP (Common Industrial Protocol) and DNP3 traffic on a segment that was assumed to be Modbus-only, revealing an undocumented, unmanaged device communicating over a different protocol than expected.

Tools and commands used here: Malcolm (web-based dashboard, no CLI syntax shown in the source material).

Countermeasures and defense: use a tool like Malcolm proactively, as a DEFENDER, to maintain an accurate, continuously updated inventory of which protocols are actually running on your OT network, rather than relying on outdated documentation.

EXAM FOCUS: Malcolm's defining exam fact is that it gives PROTOCOL-LEVEL VISIBILITY across an entire ICS/IoT environment through two specific named interfaces (OpenSearch dashboard, Arkime); contrast it with Wireshark (section 3.59), which analyzes ONE capture/session at a time rather than providing a live, aggregated dashboard.

Memory trick: "Malcolm counts every protocol's voice in the room, all at once."

3.61 Vulnerability Scanning Using Nessus

What it is: Nessus is a vulnerability assessment tool that lets attackers (and auditors) find vulnerabilities in ICS and SCADA systems, discovering and grouping them together to plan further attacks on the target OT network.

How it works (5 steps):

1. Log in to the Nessus web client, open the Policies tab, select "Create New Policy," and choose the "Basic Network Scan" template.
2. Modify the DISCOVERY node's settings for port scanning, providing a port range (the slide's example: `0-1000`).

3. Check whether SCADA-specific plugins are enabled in the Plugins tab; if not enabled, results will only cover non-SCADA ports.
4. Save the policy, open the "My Scans" folder, select "New Scan," choose the "User Defined" policies section, and pick the policy created in step 1.
5. Fill in the scan's required fields (including the target IP address) and click "Launch."

Real-world use: A plant security team runs a Nessus scan with SCADA plugins enabled against their control network and gets back specific findings such as "ICCP/COTP (ISO 8073) Protocol Detection" and "Modbus/TCP Coil Access," each tagged with a SCADA family label and severity, giving them a prioritized, documented list of exposed ICS services to fix.

Tools and commands used here: Nessus web client (GUI-driven workflow; no CLI syntax shown in the source material; full breakdown in section 5.15).

Countermeasures and defense: run Nessus (or an equivalent scanner) against your own OT network regularly, with SCADA plugins specifically enabled, since the default scan profile alone can miss ICS-specific issues entirely.

EXAM FOCUS: The critical exam detail is step 3: SCADA-specific plugins must be explicitly confirmed/enabled, or Nessus will only report on non-SCADA findings, giving a falsely reassuring result for an ICS-heavy network.

Memory trick: "No SCADA plugins, no SCADA findings - check the Plugins tab first."

3.62 Fuzzing ICS Protocols

What it is: Fuzzing ICS protocols such as Modbus, BACnet, and IPP means sending deliberately malformed or unexpected data at a target service to find crashes, errors, and exploitable vulnerabilities; Fuzzowski is the named tool for this in the module.

How it works (3 fuzzing command examples):

1. **Fuzzing BACnet:** `python -m fuzzowski 127.0.0.1 47808 -p udp -f bacnet -rt 0.5 -m BACnetMon` targets BACnet over UDP on its standard port, using the BACnet fuzzing profile (`-f bacnet`), a restart-timeout of 0.5 seconds (`-rt`), and a named monitor (`-m`).
2. **Fuzzing Modbus:** `python -m fuzzowski 127.0.0.1 502 -p tcp -f modbus -rt 1 -m modbusMon` targets Modbus over TCP on its standard port, with a 1-second restart timeout.
3. **Fuzzing IPP (Internet Printing Protocol):** `python -m fuzzowski printer1 631 -f ipp -l get_printer_attribs --restart smartplug` targets a printer's IPP service on port 631, using a specific fuzzing test case (`get_printer_attribs`) and a named restart target (`smartplug`).

Real-world use: A vendor's QA team fuzzes their own new BACnet building-controller firmware with Fuzzowski before release, and discovers that a specific malformed BACnet packet crashes the device entirely, letting them patch the parsing bug before it ships to customers.

Tools and commands used here: `python -m fuzzowski <target> <port> -p <protocol> -f <fuzz_profile> -rt <restart_timeout> -m <monitor_name>` (full breakdown in section 5.16).

Countermeasures and defense: fuzz-test ICS device firmware before deployment; apply input validation and bounds checking in protocol parsers; isolate fragile legacy devices from any untrusted network traffic that could resemble fuzzing input.

EXAM FOCUS: `-f` selects the protocol fuzzing profile (bacnet, modbus, ipp); `-rt` is the restart timeout (how long Fuzzowski waits before assuming the target crashed and needs restarting); the exam may ask you to read one of these flags.

Memory trick: "Throw garbage at the protocol, then watch what breaks."

3.63 Vulnerability Scanning using Skybox

What it is: Skybox's Vulnerability Control module performs detailed path analysis across combined OT and IT networks, giving insight into vulnerabilities and their related attack vectors, and can prioritize millions of vulnerabilities across both networks based on actual risk.

How it works:

1. Skybox ingests vulnerability data and network topology from both the OT and IT sides of the organization.
2. It performs path analysis to understand which vulnerabilities are actually reachable by an attacker, given the real network paths between zones (not just a flat vulnerability count).
3. It prioritizes the resulting (potentially millions of) vulnerabilities by risk, factoring in exposure and exploitability, and presents them on a dashboard (the slide shows categories like "Exposed & Exploitable Vulnerabilities by Threat Origin" and "Vulnerabilities by Business Line & Risk").

Real-world use: A large utility with both a corporate IT network and dozens of substations uses Skybox to cut through 266,000+ raw vulnerability findings and surface only the small subset that are BOTH exploitable AND reachable along a real network path from the internet, letting a small security team focus remediation effort where it actually matters.

Tools and commands used here: Skybox Vulnerability Control (dashboard-driven; no CLI syntax shown in the source material). Other named sniffing/vulnerability-scanning tools: SmartRF Packet Sniffer, Microsoft Defender for IoT.

Countermeasures and defense: adopt a risk-based (path-aware) prioritization approach rather than trying to patch every vulnerability equally; this is itself the value Skybox-style tools provide to defenders.

EXAM FOCUS: Skybox's defining exam fact is PATH ANALYSIS across COMBINED OT+IT networks at large scale (millions of vulnerabilities); contrast it with Nessus (section 3.61), which scans and reports per target but does not natively perform this kind of cross-network reachability/path analysis.

Memory trick: "Skybox doesn't just count the holes, it maps which ones an attacker could actually walk through."

3.64 Hacking ICS Hardware

What it is: A direct, physical approach to attacking ICS devices: using publicly available sources to learn about the exact hardware chips inside a specific ICS device, then performing static and dynamic analysis of the chip's functions to find arguments used and detect the presence (or absence) of input/output validation.

How it works:

1. Research the target device's hardware chips using public sources (datasheets, teardown reports, vendor documentation).
2. Perform static analysis (reading the chip's code/behavior without running it) and dynamic analysis (observing it while running) of the functions on the chip.
3. Analyze the integrated software inside the chip to retrieve information such as certificates, key generation algorithms, and encryption functions.

Real-world use: A hardware researcher identifies the exact microcontroller model inside a PLC from its datasheet, uses GDB and OpenOCD together over a JTAG connection to step through its firmware live, and extracts a hardcoded encryption key directly from the chip's running memory.

Tools and commands used here (software tools): GDB, OpenOCD, Binwalk, Fritzing, Radare2, Ghidra, IDA Pro.

Tools and commands used here (hardware tools): signal analyzer, multimeter, memory programmer and microcontrollers, oscilloscope, soldering equipment, magnifying glass or digital microscope, communication interface such as JTAG, screwdrivers and precision screwdrivers, precision tweezers for connection and converters.

Countermeasures and defense: use tamper-resistant chip packaging and secure enclosures; avoid storing encryption keys or certificates directly accessible in chip memory without hardware-backed protection (such as a secure element); limit public documentation detail about exact internal chip models where feasible.

EXAM FOCUS: This section's software tool list (GDB, OpenOCD, Binwalk, Fritzing, Radare2, Ghidra, IDA Pro) overlaps heavily with general reverse-engineering tools from other modules; the exam may simply ask you to recognize these as hardware/firmware analysis tools rather than network exploitation tools.

Memory trick: "Read the datasheet, probe the chip, debug it live, and the chip's own secrets come out."

3.65 Hacking Modbus Slaves using Metasploit

What it is: Because Modbus Master and Slave devices communicate in plaintext with no authentication at all, an attacker can use Metasploit to both scan for live Modbus Slave devices and directly manipulate their register/coil data.

How it works (2 Metasploit auxiliary modules):

1. **Scanning Modbus Slaves** (`auxiliary/scanner/scada/modbus_findunitid`): set `RHOST` to the target address, then `run`; the module tries each possible Modbus Unit Identifier (`UNIT_ID_FROM` through `UNIT_ID_TO`, default range 1-254) and reports whether each responds with valid Modbus/TCP data (a real, in-use station) or incorrect/no data (probably not in use).
2. **Manipulating Modbus Slave Data** (`auxiliary/scanner/scada/modbusclient`): set `data_address` (starting register address, for example `0`), `number` (how many registers to read, for example `5`), `rhost` (target IP), and `unit_number` (the target Unit Identifier, for example `2`), then `run`; the module reads (or, in write mode, writes) the specified register range directly.

Real-world use: An attacker runs `modbus_findunitid` against a water treatment PLC's IP and discovers Unit ID 2 responds correctly; they then use `modbusclient` with that Unit ID to read 5 registers starting at address 0, retrieving live process values (the slide's example result: `[1, 22, 33, 0, 0]`) with no credentials required at all.

Tools and commands used here: `set RHOST <ip>`, `run` (for `modbus_findunitid`); `set data_address <n>`, `set number <n>`, `set rhost <ip>`, `set unit_number <n>`, `run` (for `modbusclient`) (full breakdown in section 5.17).

Countermeasures and defense: Modbus/TCP has no native authentication, so the only effective defense is network-level: never expose Modbus devices directly to untrusted networks; use a VPN, firewall rule, or dedicated industrial protocol gateway with authentication in front of Modbus traffic.

EXAM FOCUS: `modbus_findunitid` finds WHICH Unit IDs exist and respond; `modbusclient` then READS or WRITES actual register/coil data at a known Unit ID; the exam may ask which module does which job, or ask you to read `set` command output to determine which parameter was configured.

Memory trick: "First find who's listening (findunitid), then talk directly to them (modbusclient)."

3.66 Hacking PLC using modbus-cli

What it is: `modbus-cli` is a Ruby gem (command-line tool) that lets an attacker read and write live Modbus register and coil values on a target PLC directly from the command line, after first finding the target using Shodan/Nmap.

How it works (8 steps):

1. **Identify internet-connected PLCs:** use Shodan and Nmap to find industrial facilities exposed online; for example, search Shodan for `TM221ME16R` to find internet-connected Schneider Electric TM221 PLCs.
2. **Install modbus-cli:** `gem install modbus-cli`.
3. **Understand datatypes:** modbus-cli supports multiple data representations at the same memory address, each with its own flag and addressing style: word (`--word`, default, unsigned, 16-bit, Schneider address `%MW100` = Modicon address `400101`), integer (`--int`, signed, 16-bit, same addresses), floating point (`--float`, 32-bit, Schneider `%MF100`), double word (`--dword`, 32-bit, Schneider `%MD100`), and Boolean/coils (1-bit, Schneider `%M100` = Modicon `101`, no type flag needed).
4. **Read register values:** `modbus read <target_ip> %MW100 10` (or the equivalent Modicon address `400101`) reads 10 consecutive register values starting at that address.
5. **Manipulate register values:** `modbus write <target_ip> %MW100 2 2 2 2 2 2 2 2` writes the value `2` into 8 consecutive registers starting at `%MW100`.
6. **Read coil values:** `modbus read <target_ip> %M100 10` (or Modicon address `101`) reads 10 consecutive coil (Boolean) values.
7. **Manipulate coil values:** `modbus write <target_ip> %M100 1 1 1 1 1 1 1 1` sets 8 consecutive coils starting at `%M100` to `1` (on).
8. **Capture data into an output file:** `modbus read --output SCADAregisters.txt <target_ip> %MW100 200` (or `SCADAcoils.txt` for coil values) saves the read results into a file instead of just printing them to the terminal, formatted as structured text (host, port, slave, offset, data).

Real-world use: Using only the Shodan search term `TM221ME16R`, an attacker finds a live, internet-exposed Schneider Electric PLC, installs `modbus-cli` locally, reads its first 10 registers to confirm live process data is flowing, then writes the value `2` into 8 of those registers directly, altering the PLC's behavior with a single command and no authentication whatsoever.

Tools and commands used here: `gem install modbus-cli`, `modbus read <ip> <address> <count>`, `modbus write <ip> <address> <values...>`, `modbus read --output <file> <ip> <address> <count>` (full flag table in section 5.18).

Countermeasures and defense: this is the same root cause as section 3.65 (Modbus has no authentication); the only real fix is network isolation, not a modbus-cli-specific control; also never expose exact PLC model numbers (like `TM221ME16R`) in ways that make them trivially searchable on Shodan.

EXAM FOCUS: The `%MW`/`%M` (Schneider-style) versus numeric (Modicon-style, like `400101`/`101`) addressing schemes both point to the SAME physical memory; the exam may give you one address format and ask for its Modicon/Schneider equivalent, or ask which data-type flag (`--word`, `--int`, `--float`, `--dword`) matches a given size (16-bit, 16-bit signed, 32-bit float, 32-bit).

Memory trick: "Find it on Shodan by model number, then read or write its memory like it's your own spreadsheet - no password asked."

3.67 Gaining Remote Access using DNP3

What it is: The OT-domain equivalent of section 3.30's IoT Telnet attack: industrial control systems are often configured with direct internet access, ignoring firewall protections, and accessed using default or weak credentials; if the DNP3 (Distributed Network Protocol 3) port is found open, an attacker exploits it to gain remote access.

How it works (4 steps):

1. The attacker performs port scanning to find open ports/services on target industrial systems.
2. If the DNP3 port is found open, the attacker exploits this to gain remote access to the system.
3. The attacker uses tools such as Shodan to find and gain remote access to such exposed targets at scale (the slide's example shows a Shodan host page with an open port 22, OpenSSH banner details, and general open-port information for a target system in Romania).
4. Because DNP3-based systems often sit with weak credentials and no real firewall protection, the attacker's access is often immediate once the port is confirmed open.

Real-world use: An attacker searches Shodan for devices exposing DNP3 alongside an open SSH port, finds a utility substation RTU with both ports open and a default SSH credential still in place, and gains full remote access to the device's control functions.

Tools and commands used here: Shodan (full breakdown in section 5.12); DNP3's standard port is TCP/UDP 20000 (ADDED CONTEXT, verify; not explicitly numbered in the source screenshot, which instead shows a general open-port/SSH example).

Countermeasures and defense: never expose DNP3-speaking devices directly to the internet; require VPN access with multi-factor authentication for any legitimate remote OT access; change every default credential on RTUs and other DNP3 devices.

EXAM FOCUS: DNP3 (OT) and Telnet (IoT, section 3.30) share the exact same exam pattern: "found open on Shodan, exploited via default/ weak credentials." If a question specifies DNP3, it is testing the OT/ICS protocol; if it specifies Telnet port 23, it is testing the IoT/general embedded-device angle.

Memory trick: "Open port, old password, instant access - whether it's Telnet on an IoT camera or DNP3 on a substation RTU."

3.68 How to Defend Against OT Hacking

What it is: A 14-item defense checklist for OT networks, paralleling section 3.34's IoT checklist but tuned to ICS/SCADA realities.

How it works (14 defenses):

1. Use purpose-built sensors to discover vulnerabilities in the network.
2. Update systems to the latest technologies and regularly patch systems.
3. Implement secure configuration and secure coding practices for OT applications.
4. Maintain an asset register for tracking and scrutinizing outdated systems.
5. Use strong passwords and change default factory-set passwords.
6. Secure remote access through multiple layers of defense, including VPNs.
7. Secure the network perimeter, filtering and preventing unauthorized inbound traffic.
8. Regularly scan systems and networks using anti-malware tools.
9. Harden systems by disabling unused services and functionalities.
10. Regularly patch vulnerabilities released by manufacturers.
11. Employ IDS and flow-measurement systems to detect attacks early.
12. Use only tested and familiar third-party web servers for serving ICS web applications.
13. Ensure ICS vendors add cryptographic signatures to application updates.
14. Perform periodic audits of industrial systems to validate security controls.

Real-world use: An OT security manager uses this checklist during an annual audit, confirming asset inventory is current (item 4), all default passwords have been changed (item 5), and that the vendor's latest firmware update was cryptographically signed before being applied (item 13).

Tools and commands used here: purpose-built OT sensors, IDS, flow-measurement systems, anti-malware tools (concept-level controls).

Countermeasures and defense: this list IS the countermeasure set; apply all 14 items as the OT-side baseline, paired with section 3.70's Purdue-based controls and section 3.71's Zero Trust model.

EXAM FOCUS: Item 13 (cryptographic signatures on vendor updates) directly counters the OT Supply Chain Attacks from section 3.54 (specifically "Third-Party Software Compromise"); the exam may link a specific attack to its specific matching defense from this list.

Memory trick: "Sense it, patch it, configure it safely, know your assets, lock the passwords, wall off remote access, filter the perimeter, scan for malware, trim unused services, patch again, watch the flow, trust only proven software, sign every update, and audit it all."

3.69 OT Vulnerabilities and Solutions

What it is: The matched solution set for each of the 10 OT vulnerabilities first listed in section 3.44.

How it works (10 matched solutions):

1. **Publicly Accessible OT Systems:** use an enterprise-grade firewall and remote access solution; develop and regularly test incident response plans.
2. **Insecure Remote Connections:** use strong multi-factor authentication and password policies; implement appropriate security patching practices; implement RBAC to manage remote access permissions.
3. **Missing Security Updates:** test applications in a sandbox environment before launching live; employ a firewall and perform device hardening.
4. **Weak Passwords:** use separate username conventions for corporate IT vs. OT networks; change default credentials at installation time; perform security audits for secure password policy compliance.
5. **Insecure Firewall Configuration:** implement secure firewall configuration; configure the access control list on the firewall.
6. **OT Systems Placed within the Corporate IT Network:** implement multi-factor authentication; segregate corporate IT and OT devices; establish a DMZ for all IT-OT connections.
7. **Insufficient Security for Corporate IT Network from OT Systems:** restrict access on the IT-OT network based on business need; establish a secure gateway between the two networks.
8. **Lack of Segmentation within OT Networks:** state clear separation between critical and non-critical systems; implement a defense-in-depth zoning model; adopt a zero-trust model that assumes no default trust.
9. **Lack of Encryption and Authentication for Wireless OT Networks:** use strong wireless encryption protocols; use industry-standard cryptographic algorithms; conduct regular security audits.

10. **Unrestricted Outbound Internet Access from OT Networks:** conduct a formal risk assessment; monitor and segregate OT systems from external access; download security updates to a separate repository outside the OT network.

Real-world use: Following a security audit flagging vulnerability 6 (OT systems inside the corporate IT network), a plant implements solution 6 in full: adds MFA for any cross-network access, physically segregates OT devices onto their own switches, and stands up a dedicated DMZ for the few systems that legitimately need to bridge both sides.

Tools and commands used here: firewalls, RBAC systems, MFA solutions, DMZ architecture (concept-level controls).

Countermeasures and defense: this entire section IS the countermeasure list; use it as the direct answer key to section 3.44's vulnerability list.

EXAM FOCUS: The exam may give an OT vulnerability from section 3.44 and ask for its solution here, or vice versa; practice matching both directions, the same way as the IoT OWASP pairing in sections 3.5/3.36.

Memory trick: Each solution echoes its vulnerability's own wording, just like the IoT pairing: "weak password -> change default and audit," "insufficient segmentation -> zone it and trust nothing by default."

3.70 How to Secure an IT/OT Environment (Security Controls based on Purdue Model)

What it is: A direct mapping of specific attack vectors, risks, and matching security controls onto each zone/level of the Purdue Model (first introduced in section 3.41).

How it works (5 zone rows):

1. **Enterprise (Levels 4 & 5):** attack vectors include spear phishing and ransomware, abusing infrastructure/network access; controls: firewalls, IPS, anti-bot, URL filtering, SSL inspection, antivirus, DLP (Data Loss Prevention).
2. **Industrial DMZ (Level 3.5/IDMZ):** attack vector is DoS attacks; risks include malware injection and network infection; controls: anti-DoS solutions, IPS, anti-bot, application control, ALF (Application Layer Firewall).
3. **Manufacturing (Level 3, Operational Systems):** attack vectors include ransomware, bot infection, industrial spying, unsecured USB ports; risks include altering the industrial process and unpatched monitoring systems; controls: anti-bot, IPS, sandboxing, application control, traffic encryption, port protection.
4. **Manufacturing (Levels 2 & 1, Control Systems & Basic Controls):** attack vectors include DoS exploitation, unencrypted protocols, default credentials, industrial spying; risks include application and OS vulnerabilities and altering the

industrial process; controls: IPS, firewall, communication encryption using IPsec, security gateways, use of authorized RTU/PLC commands only.

5. **Manufacturing (Level 0, Physical Process):** attack vector is physical security breach; risk is modification or disruption of the physical process; controls: point-to-point communication, MAC authentication, additional security gateways at Levels 0 and 1.

Real-world use: A security architecture review walks a plant's network diagram top-down through all 5 rows, confirming DLP is present at the Enterprise level (row 1), an Application Layer Firewall sits in the IDMZ (row 2), USB port protection is enforced on the manufacturing floor (row 3), RTU/PLC commands are restricted to an authorized allow-list (row 4), and MAC-based authentication secures the lowest physical-process level (row 5).

Tools and commands used here: firewalls, IPS, anti-bot, URL filtering, SSL inspection, antivirus, DLP, anti-DoS, application control, ALF, sandboxing, traffic encryption, port protection, IPsec, security gateways, MAC authentication (all concept-level controls, mapped per level).

Countermeasures and defense: this entire table IS the countermeasure framework; it is the single most complete, exam-ready reference connecting the Purdue Model (section 3.41) to concrete security products/controls.

EXAM FOCUS: The exam may describe a specific risk or control (such as "MAC authentication" or "Data Loss Prevention") and ask which Purdue zone/level it belongs to; use the pattern that controls get MORE IT-style (firewalls, DLP, antivirus) higher up, and MORE PHYSICAL/protocol-specific (MAC auth, authorized RTU/PLC commands) lower down.

Memory trick: "The higher the level, the more it looks like normal IT security; the lower the level, the more it's about physically and electrically trusting only the right commands."

3.71 Implementing a Zero-Trust Model for ICS/SCADA

What it is: Most ICS networks run on legacy systems or hardware with no modern security controls, making them vulnerable to sophisticated attacks; implementing a Zero-Trust model lets an organization apply robust access management to these legacy systems and the network as a whole, while also ensuring comprehensive visibility and validation of every application, user, and device.

How it works (5 steps):

1. **Defining the Network:** identify and document exactly what exists on the ICS network (assets, data flows, trust boundaries).

2. **Mapping the Traffic:** observe and record how traffic actually flows across the network, to understand real (not assumed) dependencies.
3. **Architecting the Network:** redesign the network architecture around the Zero Trust principle, building in explicit segmentation and control points based on the traffic mapped in step 2.
4. **Developing a ZT Policy:** write explicit Zero Trust policies defining exactly which users, applications, and devices are allowed to do what, with no default trust granted to anyone.
5. **Monitoring and Maintaining:** continuously monitor the network against the Zero Trust policy, and maintain/update it as the environment changes.

Real-world use: A utility company facing legacy RTUs that cannot be patched implements Zero Trust not by upgrading the RTUs themselves, but by wrapping them in a tightly controlled network architecture (steps 1-3) with explicit, continuously monitored access policies (steps 4-5), so even an unpatched, insecure RTU cannot be reached without passing multiple verification checkpoints first.

Tools and commands used here: none named directly on this slide (organizational/ architectural process); this complements the OWASP/ CASB-style Zero Trust concept covered more broadly in Module 19 (ADDED CONTEXT cross-reference).

Countermeasures and defense: this entire 5-step process IS the countermeasure; it is specifically valuable for OT because it does not require replacing legacy hardware to gain meaningful security improvement.

EXAM FOCUS: Memorize the exact ORDER of the 5 steps; the exam may describe an activity (such as "writing explicit access policies") and ask which step of implementing Zero Trust for ICS/SCADA it represents.

Memory trick: "Define it, map it, (re)build it, write the rules, then watch it forever" - Define, Map, Architect, Develop Policy, Monitor and Maintain.

3.72 International OT Security Organizations and Module Summary

What it is: Global cybersecurity organizations dedicated to improving the security resilience of critical infrastructure through policy, information sharing, and best practices, closing out the module.

How it works:

1. **OTCC (Operational Technology Cybersecurity Coalition):** works with industry and government stakeholders to promote interoperable, standards-based OT cybersecurity solutions and enhance the resilience of critical national infrastructure.

2. **OT-ISAC:** an Information Sharing and Analysis Center focused specifically on OT security, enabling organizations to share threat intelligence relevant to industrial environments.
3. **NERC (North American Electric Reliability Corporation):** sets and enforces reliability and security standards for the North American electric power grid.

Module summary recap (per the source material's own closing slide): this module covered IoT concepts and technologies/ protocols; IoT threats and attacks; the IoT hacking methodology (information gathering, vulnerability scanning, launching attacks, gaining and maintaining remote access) with its tools; countermeasures against IoT hacking; securing IoT devices/networks with IoT security tools; OT concepts, threats, and attacks; the OT hacking methodology and its tools; countermeasures against OT attacks; and OT security solutions/tools. The next module in the course covers cloud hacking (Module 19).

Real-world use: A utility's compliance team tracks NERC standards directly (since non-compliance carries real regulatory penalties for the power sector), while its security operations team separately monitors OT-ISAC threat-sharing bulletins for emerging, sector-specific attack techniques relevant to their own equipment.

Tools and commands used here: none (organizational bodies, not tools).

Countermeasures and defense: participating in these bodies (OTCC, OT-ISAC, NERC-equivalent regulators in your region) is itself a program-level countermeasure, giving an organization earlier warning of emerging OT threats than it could discover alone.

EXAM FOCUS: Know NERC specifically as the North American ELECTRIC GRID reliability/security standards body; a question mentioning "electric grid reliability standards" points to NERC by name.

Memory trick: "OTCC sets the direction, OT-ISAC shares the warnings, NERC enforces the rules for the grid."

4. Tool Encyclopedia

All tools below come from Module 18 unless marked "(added)" for a close exam alternative. Phase follows this module's hacking methodology: Info Gathering, Vuln Scanning, Gaining Access, Maintaining Access; IoT/OT is marked per tool's domain.

4.1 IoT Information Gathering Tools

Tool	What it does (simple)	Domain	Phase	Exam keyword
Shodan	Search engine indexing internet-connected device banners	IoT/OT	Info Gathering	"country:", "city:", "geo:" filters
Censys	Internet-wide device/banner search engine	IoT/OT	Info Gathering	alternative to Shodan
FOFA	Internet-wide device/banner search engine	IoT	Info Gathering	tool list item
MultiPing	Monitors/pings many hosts at once	IoT	Info Gathering	tool list item
FCC ID Search	Looks up a device's FCC ID for specs	IoT	Info Gathering	finds RF device specifications
Nmap	Port scanner; here used for ports 80/81/8080/8081	IoT	Info Gathering	insecure HTTP port discovery
Airmon-ng	Puts a wireless card into monitor mode	IoT	Info Gathering	"start "
Airodump-ng	Scans/lists nearby wireless networks	IoT	Info Gathering	"start mon"
Wireshark	Captures and analyzes network traffic	IoT/OT	Info Gathering	MQTT/Modbus dissectors
Suphammer (added)	IoT sniffing tool	IoT	Info Gathering	tool list item

4.2 IoT Vulnerability Scanning Tools

Tool	What it does (simple)	Domain	Phase	Exam keyword
IoTSeeker	Finds IoT devices still using default credentials	IoT	Vuln Scanning	"has/still has default passwd"
Genzai	IoT toolkit matching devices to default passwords/CVEs	IoT	Vuln Scanning	named CVE in output, e.g. CVE-2016-8423
beSTORM	IoT vulnerability/ fuzz scanning tool	IoT	Vuln Scanning	tool list item
IoTSploit	IoT vulnerability scanning tool	IoT	Vuln Scanning	tool list item
IoTVAS	IoT vulnerability assessment service	IoT	Vuln Scanning	tool list item

4.3 IoT Gaining/Maintaining Access Tools

Tool	What it does (simple)	Domain	Phase	Exam keyword
RFCrack	Python SDR tool for RF replay/ rolling-code/jam attacks	IoT	Gaining Access	"-M MOD_2FSK", "-F", "-j"
HackRF One	Wideband (1MHz-6GHz) half-duplex SDR hardware	IoT	Gaining Access	"hackrf_transfer", "hackrf_info"
Open Sniffer	Captures/replays 802.15.4 (Zigbee) frames	IoT	Gaining Access	EPID field, beacon frames
CamOver	Discloses network camera admin passwords	IoT	Gaining Access	"-a", "--shodan", "--zoomeye"
BUS Auditor	Scans for UART/ I2C pins on a circuit board	IoT	Gaining Access	"uartscan", "i2cscan"

Tool	What it does (simple)	Domain	Phase	Exam keyword
DIVA	Damn Insecure and Vulnerable Application (practice target)	IoT	Gaining Access	tool list item
JTAGulator	Identifies unknown JTAG/ UART header pinouts	IoT	Gaining Access	"unknown pinout discovery"
CatSniffer	Passively monitors IoT RF traffic (CC1352-based)	IoT	Info Gathering	hardware sniffer board
KillerBee	Zigbee/802.15.4 security research framework	IoT	Gaining Access	tool list item
PENIOT	IoT-focused penetration testing tool	IoT	Gaining Access	tool list item
RouterSploit	Exploitation framework for routers/ embedded devices	IoT	Gaining Access	router/embedded exploit framework
Firmware Mod Kit	Extracts and rebuilds firmware images	IoT	Maintaining Access	"extract-firmware.sh"
binwalk	Analyzes/extracts data from firmware images	IoT	Maintaining Access	finds TRX header, SquashFS
QEMU	Emulates firmware/CPU architectures for dynamic testing	IoT	Maintaining Access	"qemu-mipsel", "qemu-arm"

4.4 IoT Device Management (Defender) Platforms

Tool	What it does (simple)	Domain	Phase	Exam keyword
SeaCat.io	IoT device management platform	IoT (defense)	Maintaining	tool list item

Tool	What it does (simple)	Domain	Phase	Exam keyword
Armis Centrix	IoT device management/ asset visibility platform	IoT (defense)	Maintaining	tool list item
Oracle Fusion Cloud IoT	IoT device management platform	IoT (defense)	Maintaining	tool list item
Golioth	IoT device management platform	IoT (defense)	Maintaining	tool list item
AWS IoT Device Management	Fleet onboarding/ monitoring/ update platform	IoT (defense)	Maintaining	tool list item
IBM Watson IoT Platform	IoT device management platform	IoT (defense)	Maintaining	tool list item
openBalena	Open-source IoT fleet/device management platform	IoT (defense)	Maintaining	tool list item
Azure IoT Central	Microsoft's IoT device management platform	IoT (defense)	Maintaining	"checkout dashboard" example

4.5 OT/ICS Information Gathering Tools

Tool	What it does (simple)	Domain	Phase	Exam keyword
Shodan	Finds exposed ICS/SCADA systems by port/ vendor/geo	OT	Info Gathering	"port:502", "SCADA country:"
CIRT.net	Online default-password database (531+ vendors)	OT	Info Gathering	"default password database"
Kamerka-GUI	ICS/camera recon and mapping tool	OT	Info Gathering	tool list item

Tool	What it does (simple)	Domain	Phase	Exam keyword
SearchDiggity	Search-engine-based recon tool (Google dorking style)	OT	Info Gathering	tool list item
Zeek	Network security monitor/traffic analysis framework	OT	Info Gathering	feeds Malcolm's dashboard
Criminal IP	Internet-wide device/threat search engine	OT	Info Gathering	tool list item
ZoomEye	Internet-wide device search engine	OT/IoT	Info Gathering	alternative to Shodan
Nmap (ICS scripts)	Protocol-specific ICS/SCADA scanning scripts	OT	Info Gathering	"modbus-discover", "s7-info", etc.
Wireshark	Captures/analyzes Modbus/TCP and other ICS traffic	OT	Info Gathering	"modbus" display filter
Malcolm	Dashboard-based ICS/IoT protocol discovery tool	OT	Info Gathering	OpenSearch + Arkime dashboards

4.6 OT/ICS Vulnerability Scanning Tools

Tool	What it does (simple)	Domain	Phase	Exam keyword
Nessus	Vulnerability assessment scanner with SCADA plugins	OT	Vuln Scanning	"Basic Network Scan", SCADA plugins
Skybox Vulnerability Control	Path analysis + risk prioritization across OT/IT	OT	Vuln Scanning	"path analysis", millions of vulns
Fuzzowski	Fuzzes ICS protocols (Modbus, BACnet, IPP)	OT	Vuln Scanning	"-f bacnet/modbus/ipp"

Tool	What it does (simple)	Domain	Phase	Exam keyword
SmartRF Packet Sniffer	RF packet capture/analysis tool (Texas Instruments)	OT	Vuln Scanning	tool list item
Microsoft Defender for IoT	IoT/OT security monitoring platform	OT (defense)	Vuln Scanning	tool list item

4.7 OT/ICS Exploitation and Hardware Tools

Tool	What it does (simple)	Domain	Phase	Exam keyword
Metasploit (modbus_findunitid)	Finds live Modbus Slave Unit IDs	OT	Gaining Access	"UNIT_ID_FROM/TO"
Metasploit (modbusclient)	Reads/writes Modbus register data directly	OT	Gaining Access	"data_address", "unit_number"
modbus-cli	Ruby gem CLI for reading/writing Modbus registers/coils	OT	Gaining Access	"modbus read/write", "%MW", "%M"
GDB	Debugger used to step through chip firmware	OT/IoT	Gaining Access	JTAG + GDB combo
OpenOCD	On-chip debugging interface software	OT/IoT	Gaining Access	tool list item
Fritzing	Circuit design/documentation tool	OT/IoT	Gaining Access	tool list item
Radare2	Reverse engineering/binary analysis framework	OT/IoT	Gaining Access	tool list item
Ghidra	NSA-developed reverse engineering/decompiler suite	OT/IoT	Gaining Access	tool list item

Tool	What it does (simple)	Domain	Phase	Exam keyword
IDA Pro	Commercial disassembler/ decompiler	OT/IoT	Gaining Access	tool list item

> **EXAM TIP:** When two recon tools look similar, match the DOMAIN keyword in the question: "webcam, geo, city filters" -> Shodan (IoT camera context); "port:502, SCADA country" -> Shodan (OT/ICS context, same tool, different filter); "531 vendors, 2117 passwords" -> CIRT.net specifically; "path analysis across OT+IT" -> Skybox specifically; "OpenSearch + Arkime dashboards" -> Malcolm specifically.

5. Command Sheet (I have no lab, so teach me to read commands)

How to read any command below: the first word is the program; words starting with `-` or `--` are flags (options); a flag may take a value right after it; everything else is a target or argument. A `$` prompt means a normal user shell; a `#` prompt means a root shell. Commands are grouped by tool, in the order they first appear in section 3.

5.1 Nmap (Scanning IoT Insecure HTTP Ports)

Purpose: quickly check a target IP range for IoT devices exposing their management/ web interface over common insecure HTTP ports.

Full syntax:

```
nmap -p 80,81,8080,8081 <Target IP address range>
```

Breakdown:

- `-p 80,81,8080,8081` tells Nmap to check only these 4 ports instead of a default or full port range.
- `<Target IP address range>` can be a single IP or a CIDR block (such as `192.168.1.0/24`).

Flag table:

Flag	Meaning	When to use	Example
`-p`	Port(s) to scan	Narrowing scope to known IoT web ports	`-p 80,81,8080,8081`

Demo commands:

```
nmap -p 80,81,8080,8081 192.168.1.0/24
nmap -p 80,81,8080,8081 192.168.1.50
```

Sample output, line by line:

```
PORT     STATE SERVICE
80/tcp   open  http
8080/tcp open  http-proxy
```

Each `open` line is one exposed HTTP(-like) service; an IoT device answering here is a candidate for further manual/browser inspection or credential testing.

Common mistakes: assuming a closed/filtered port means "safe" - always also check other known IoT ports (23 Telnet, 502 Modbus) in a real assessment, not just these 4; forgetting this is UNENCRYPTED HTTP, so credentials entered on these pages travel in plaintext.

What the exam asks: recognizing WHY these 4 specific ports matter for IoT (common default web-management ports on routers/cameras/ IoT devices), not just generic Nmap syntax recall.

5.2 ifconfig, Airmon-ng, and Airodump-ng (Wireless Recon Setup)

Purpose: identify your own wireless network card, switch it into monitor mode (so it can see all nearby wireless traffic, not just traffic addressed to you), and then list every nearby wireless network along with its channel.

Full syntax:

```
ifconfig
airmon-ng start wlan0
airodump-ng start wlan0mon
airmon-ng start wlan0mon <channel_number>
```

Breakdown:

- `ifconfig` with no arguments lists all network interfaces; look for your wireless adapter's name (the module's example: `wlan0`).
- `airmon-ng start wlan0` puts that adapter into monitor mode, renaming it to `wlan0mon` in most setups.
- `airodump-ng start wlan0mon` scans and lists every nearby wireless network visible to the now-monitoring adapter, including each network's channel.
- `airmon-ng start wlan0mon <channel>` re-tunes the monitoring adapter to lock onto one specific channel, once the target network and its channel are known.

Flag table:

Flag/Argument	Meaning	When to use	Example
`start <iface>`	Puts/keeps an adapter in monitor mode	Before any wireless capture	`airmon-ng start wlan0`
`<channel_number>`	Locks the adapter to one Wi-Fi channel	After identifying target's channel	`airmon-ng start wlan0mon 11`

Demo commands:

```
ifconfig
airmon-ng start wlan0
airodump-ng start wlan0mon
airmon-ng start wlan0mon 11
```

Sample output, line by line:

```
wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>
       inet 192.168.1.20  netmask 255.255.255.0
```

This confirms `wlan0` exists, is up, and has an IP address, meaning it is a usable adapter before putting it into monitor mode.

Common mistakes: forgetting that the interface name often changes after enabling monitor mode (`wlan0` becomes `wlan0mon`); trying to capture traffic on the wrong channel (traffic on other channels will simply not appear).

What the exam asks: the correct ORDER of these 3 tools (`ifconfig` to identify, `airmon-ng` to enable monitor mode, `airodump-ng` to scan) and which tool does which job.

5.3 Wireshark (Capturing and Reading MQTT IoT Traffic)

Purpose: capture raw wireless traffic in monitor mode and read IoT application-layer protocol messages, specifically MQTT in this module's example.

Full syntax: Wireshark is a GUI tool; there is no single command line invocation shown in the source material beyond selecting the monitor-mode interface (`wlan0mon`) and clicking start capture.

Breakdown: once capturing, apply the display filter `mqtt` (or read the Protocol column directly) to isolate MQTT packets from everything else on the wire.

Flag table: not applicable (GUI tool); the relevant "input" is the display filter typed into Wireshark's filter bar.

Demo commands:

```
(Wireshark filter bar) mqtt
```

Sample output, line by line:

```
No. Source      Destination  Protocol  Info
39  10.10.1.22  10.10.1.19   MQTT      Ping Request
40  10.10.1.19  10.10.1.22   MQTT      Ping Response
```

Each row is one MQTT message; "Ping Request/Response" lines are simple keep-alive checks; "Publish Message" rows (seen elsewhere in the same capture) carry the actual IoT data being sent, all in plaintext since MQTT has no encryption by default.

Common mistakes: assuming MQTT traffic is safe to view just because it looks like simple "ping" messages; missing that expanding the Transmission Control Protocol layer reveals the destination port (1883, MQTT's default, unencrypted port).

What the exam asks: recognizing MQTT traffic by its port (1883) and message types (Ping Request/Response, Publish Message/Ack/ Release/Received/Complete) when shown a Wireshark capture.

5.4 IoTSeeker (Default Credential Scanner)

Purpose: scan a list of IoT device IP addresses to check whether each one is still using its factory-default password.

Full syntax:

```
perl iotScanner.pl <IP1>,<IP2>,<IP3>,...
```

Breakdown: `iotScanner.pl` is the Perl script itself; its single argument is a comma-separated list of target IP addresses to check.

Flag table: not applicable (no flags; a single comma-separated IP-list argument).

Demo commands:

```
perl iotScanner.pl 1.23.123.431,1.23.123.443,1.23.123.453
```

Sample output, line by line:

```
device 1.23.123.431 is of type Stardot has default passwd
device 1.23.123.443 is of type Arecont has changed passwd
```

Each line names the device's IP, its detected product type, and whether it "has default passwd" (still vulnerable) or "has changed passwd" (safe); lines with "has default passwd" are the attacker's (or auditor's) priority targets.

Common mistakes: feeding IoTSeeker IP addresses of device types it does not recognize (it will simply not report a meaningful result for those); assuming "has changed passwd" guarantees full security (it only confirms the DEFAULT password specifically was changed).

What the exam asks: reading IoTSeeker's exact output phrasing to determine which specific device(s) in a list remain vulnerable.

5.5 RFCrack (SDR Rolling-Code, Replay, and Jamming Attacks)

Purpose: capture, analyze, and replay or jam sub-GHz RF signals used by key fobs, alarms, and other IoT/vehicle devices, using a Yardstick One SDR device.

Full syntax:

```
python RFCrack.py -i
python RFCrack.py -I -M MOD_2FSK -F <frequency_Hz>
python RFCrack.py -I -M MOD_2FSK -F <frequency_Hz> \
    -U <upper_RSSI> -L <lower_RSSI>
python RFCrack.py -j -F <frequency_Hz>
```

Breakdown:

- `-i` starts interactive scan/capture mode.
- `-I` (capital i) starts a capture at a specific modulation and frequency.
- `-M MOD_2FSK` selects 2-FSK (Frequency-Shift Keying) modulation, common for key fob signals.
- `-F <frequency_Hz>` sets the target frequency in Hz (such as `314350000` for 314.35 MHz).
- `-U`/`-L` set the upper and lower RSSI (signal strength) capture bounds.
- `-j` switches to jamming mode instead of capture mode.

Flag table:

Flag	Meaning	When to use	Example
`-i` / `-I`	Interactive scan / targeted capture mode	Starting a capture session	`-i`
`-M`	Modulation scheme	Matching the target device's signal type	`-M MOD_2FSK`
`-F`	Frequency in Hz	Tuning to the target's known frequency	`-F 314350000`
`-U` / `-L`	Upper/lower RSSI bounds	Filtering by signal strength	`-U -100 -L -10`
`-j`	Jam the given frequency	Blocking a target's first transmission	`-j -F 314000000`

Demo commands:

```
python RFCrack.py -I -M MOD_2FSK -F 314350000
python RFCrack.py -j -F 314000000
```

Sample output, line by line:

```
Welcome to RFCrack - A Software Defined Radio Attack Tool
Release: 1.4
Hardware Needed: 1 Yardstick (2 for RollingCode)
```

This banner confirms the tool version and reminds the operator that a rolling-code attack specifically needs TWO Yardstick One devices (one to jam, one to capture), while other attacks need only one.

Common mistakes: using only one Yardstick One device when attempting a rolling-code attack (jamming and capturing at the same time requires two separate radios); setting the wrong modulation scheme for the target device, which will capture nothing useful.

What the exam asks: matching RFCrack's flags (`-M`, `-F`, `-U`/ `-L`, `-j`) to what they configure, and recalling that rolling-code attacks need 2 Yardstick One devices.

5.6 HackRF One (hackrf_transfer and hackrf_info)

Purpose: capture (receive) and replay (transmit) raw RF signals using the HackRF One SDR, and confirm the hardware is connected and working.

Full syntax:

```
hackrf_transfer -r <file>.raw -f <frequency_Hz> \
    -l <if_gain> -g <vga_gain>
hackrf_transfer -t <file>.raw -f <frequency_Hz> -x <tx_gain>
hackrf_info
```

Breakdown:

- `-r <file>` receives (captures) a signal and saves it to the named raw file.
- `-t <file>` transmits (replays) a signal from the named raw file.
- `-f <frequency_Hz>` sets the center frequency in Hz.
- `-l` sets the IF (baseband) receive gain; `-g` sets the VGA receive gain; `-x` sets the TX (transmit) gain.
- `hackrf_info` (no flags) queries and prints the connected HackRF device's board ID, firmware version, part ID, and serial number.

Flag table:

Flag	Meaning	When to use	Example
`-r`	Receive/capture to file	Recording a target signal	`-r capture.raw`
`-t`	Transmit/replay from file	Replaying a captured signal	`-t capture.raw`
`-f`	Center frequency in Hz	Tuning to the target device	`-f 433900000`
`-l` / `-g`	Receive IF/VGA gain	Tuning capture sensitivity	`-l 20 -g 20`
`-x`	Transmit gain	Tuning replay signal strength	`-x 40`

Demo commands:

```
hackrf_transfer -r doorbell.raw -f 433900000 -l 20 -g 20
hackrf_transfer -t doorbell.raw -f 433900000 -x 40
hackrf_info
```

Sample output, line by line:

```
Found HackRF board 0:
Board ID Number: 2 (HackRF One)
Firmware Version: 2015.07.2
```

This confirms exactly one HackRF One board is connected and responsive, with its firmware version, before attempting any capture/replay command.

Common mistakes: confusing `-r` (receive) with `-t` (transmit), which would attempt to transmit an empty/wrong file instead of capturing; forgetting `Ctrl-C` is needed to stop an ongoing capture/transmit session.

What the exam asks: reading a `hackrf_transfer` command and telling whether it is capturing (`-r`) or replaying (`-t`) a signal, and at what frequency.

5.7 BUS Auditor (UART and I2C Pin Scanning)

Purpose: automatically scan a circuit board's unlabeled header pins to identify which ones carry a UART serial console or an I2C bus, and at what voltage/baud rate.

Full syntax:

```
busauditor.generic.uartscan -v <voltage> -p <serial_port>
busauditor.generic.i2cscan -v <voltage> -p <serial_port> \
    -s <start_pin> -e <end_pin>
```

Breakdown:

- `-v <voltage>` sets the target voltage to test at (such as `3.3` for 3.3V logic, common on most IoT boards).
- `-p <serial_port>` is BUS Auditor's own connection to the host computer (such as `/dev/ttyACM0`), not the pin being tested.
- `-s`/`-e` (I2C scan only) set the start and end pin numbers to sweep across.

Flag table:

Flag	Meaning	When to use	Example
`-v`	Target voltage	Matching the board's logic level	`-v 3.3`
`-p`	BUS Auditor's host serial port	Always required	`-p /dev/ttyACM0`
`-s` / `-e`	I2C scan pin range (start/end)	Narrowing/widening the I2C sweep	`-s 0 -e 10`

Demo commands:

```
busauditor.generic.uartscan -v 3.3 -p /dev/ttyACM0
busauditor.generic.i2cscan -v 3.3 -p /dev/ttyACM0 -s 0 -e 10
```

Sample output, line by line:

```
UART Scan Result:
RX pin: 1
TX pin: 0
BaudRate: 9600
```

This identifies exactly which 2 pins carry serial RX/TX, and the baud rate needed to talk to them, letting the attacker (or researcher) wire up a USB-serial adapter correctly on the first try.

Common mistakes: testing at the wrong voltage (`-v`), which can either miss a valid UART/I2C bus or, worse, damage the board; forgetting that `-p` is the TOOL's connection to your PC, not the pin being scanned.

What the exam asks: distinguishing UART scan output (RX/TX pins, baud rate) from I2C scan output (device address, SCL/SDA pins).

5.8 CamOver (Camera Admin Password Disclosure)

Purpose: disclose a network camera's admin password, either for one specific camera or in bulk across every camera found via Shodan or ZoomEye.

Full syntax:

```
camover -a <Camera_IP_Address>
camover -t --shodan <Shodan_API_Key>
camover --zoomeye <ZoomEye_API_Key> -p <pages>
```

Breakdown:

- `-a <ip>` targets exactly one camera by IP address.
- `-t` enables multi-threading (faster bulk processing).
- `--shodan <key>` pulls a target list directly from a Shodan search, using the provided API key.
- `--zoomeye <key> -p <pages>` does the same using ZoomEye instead, pulling the specified number of result pages.

Flag table:

Flag	Meaning	When to use	Example
`-a`	Single target IP address	One known camera	`-a 203.0.113.5`
`-t`	Use threads for faster processing	Bulk/mass scanning	`-t --shodan <key>`
`--shodan`	Pull targets from Shodan	Mass exploitation over the internet	`--shodan <api_key>`
`--zoomeye`	Pull targets from ZoomEye	Alternative to Shodan	`--zoomeye <api_key>`
`-p`	Number of ZoomEye result pages	Controlling scan size	`-p 5`

Demo commands:

```
camover -a 203.0.113.5
camover -t --shodan abcd1234shodankey
```

Sample output, line by line:

```
camover: disclosure network camera admin password
[+] 203.0.113.5 -> admin:12345
```

The `[+]` line confirms a successful password disclosure, showing the exact admin username/password pair recovered for that camera's IP.

Common mistakes: running CamOver without a valid API key when using `--shodan`/`--zoomeye` (the scan will fail or return no targets); not combining `-t` with bulk modes, resulting in much slower scans than necessary.

What the exam asks: telling apart the single-target flag (`-a`) from the mass-scan flags (`--shodan`, `--zoomeye`).

5.9 Firmware Mod Kit (extract-firmware.sh)

Purpose: extract a firmware image's internal structure (bootloader header, compressed data, filesystem) for analysis, and later rebuild a modified image.

Full syntax:

```
./extract-firmware.sh <path_to_firmware>.bin
```

Breakdown: the single argument is the path to the firmware binary to extract; the script auto-detects known firmware signatures inside it.

Flag table: not applicable (single positional argument, no flags, in the version shown).

Demo commands:

```
./extract-firmware.sh /root/docs/dd-wrt.v24_micro_generic.bin
```

Sample output, line by line:

```
DECIMAL  DESCRIPTION
0x0      TRX firmware header, image size: 1769472 bytes
0x1C     gzip compressed data
0x9A8    LZMA compressed data
0x70728  Squashfs filesystem, DD-WRT signature
```

Each row is one identified section of the firmware image, with its byte offset (DECIMAL) and a description; the SquashFS row confirms where the actual filesystem begins, which the tool then extracts automatically.

Common mistakes: running the script against a corrupted or partial firmware download, which prevents signature detection; assuming extraction alone modifies the device (it does not; the device must later be reflashed with a rebuilt image for any change to take effect).

What the exam asks: recognizing that Firmware Mod Kit both EXTRACTS and REBUILDS firmware, which is what makes it useful for planting a backdoor and repackaging a seemingly-unmodified image.

5.10 Firmware Analysis Commands (file, strings, hexdump, binwalk, dd, mount, grep, find, qemu)

Purpose: a toolchain of standard Linux commands used together to identify, extract, mount, search, and emulate firmware, following the 6-stage process from section 3.32.

Full syntax:

```
file firmware.bin
md5sum firmware.bin
strings firmware.bin
hexdump firmware.bin
binwalk firmware.bin
dd if=firmware.bin of=rootfs.img bs=1 skip=<offset>
mkdir rootfs
sudo mount -t ext2 rootfs.img rootfs
grep -rnw /path/to/somewhere/ -e "password"
find . -name "*.conf"
qemu-mipsel -L <prefix> <binary>
qemu-arm -L <prefix> <binary>
```

Breakdown:

- `file` identifies the general file type of the `.bin` image.
- `md5sum` computes a hash to verify the firmware's integrity against a known-good value.
- `strings` extracts human-readable text embedded in the binary.
- `hexdump` shows the raw bytes of the file for manual inspection.
- `binwalk` automatically detects and can extract embedded file systems/compressed sections (similar to Firmware Mod Kit).
- `dd if=... of=... bs=1 skip=<offset>` copies out a byte range starting at a known offset, commonly used to pull out just the filesystem portion found by `binwalk`.
- `mount -t ext2 <image> <dir>` mounts the extracted filesystem image at a local directory so it can be browsed like a normal disk.
- `grep -rnw <dir> -e "<pattern>"` recursively (`-r`) searches with line numbers (`-n`) and whole-word matching (`-w`) for a text pattern across every file.
- `find . -name "*.conf"` finds every file matching a name pattern (such as all `.conf`, `.pem`, or `.crt` files).
- `qemu-<arch> -L <prefix> <binary>` emulates a binary built for a different CPU architecture (such as MIPS or ARM) so it can run and be tested on a normal x86 analysis machine.

Flag table:

Flag	Meaning	When to use	Example
`-r` (grep)	Recursive search	Searching an entire directory tree	`grep -r ... /path/`
`-n` (grep)	Show line numbers	Locating exact match location	`grep -n ...`
`-w` (grep)	Match whole words only	Avoiding partial-word false positives	`grep -w "root"`
`-name` (find)	Match by filename pattern	Finding files by extension	`find . -name "*.pem"`
`-L` (qemu)	Library/prefix path for emulation	Pointing QEMU at the right root filesystem	`-L ./rootfs`

Demo commands:

```
file firmware.bin
strings firmware.bin | grep -i password
binwalk firmware.bin
sudo mount -t ext2 rootfs.img rootfs
grep -rnw rootfs/ -e "password"
qemu-mipsel -L rootfs rootfs/usr/sbin/httpd
```

Sample output, line by line:

```
firmware.bin: data
DECIMAL   DESCRIPTION
670720    Squashfs filesystem, little endian
```

`file` returning just "data" means the format was not immediately obvious (common for proprietary firmware); `binwalk`'s output then pinpoints the exact byte offset (670720) where the real filesystem begins, which is what gets handed to `dd` next.

Common mistakes: skipping the MD5 verification step and analyzing a corrupted download; mounting a filesystem image with the wrong `-t` type (ext2 vs. squashfs vs. others); running `qemu-<wrong-arch>` against a binary built for a different CPU, which fails immediately.

What the exam asks: matching each command to its STAGE in the 6-stage firmware analysis process (obtain/analyze/extract/mount/ analyze filesystem/emulate) from section 3.32, and reading `binwalk` output to find an offset.

5.11 Fuxnet Deployment Pattern (xargs -P)

Purpose: deploy an OT-targeting malware script against a large list of target IP addresses in parallel, instead of one at a time.

Full syntax:

```
cat all_sensor_router_ips.txt | xargs -P10 -I{} ./fuxnet.sh {}
rm fuxnet.sh
```

Breakdown:

- `cat <file>` prints the target IP list, one per line.
- `xargs -P10` runs up to 10 instances of the following command in parallel at once.
- `-I{}` tells `xargs` to substitute each input line in place of `{}` in the command that follows.
- `./fuxnet.sh {}` runs the deployment script once per target IP, substituted via `{}`.
- `rm fuxnet.sh` deletes the script after use, to reduce forensic evidence left behind.

Flag table:

Flag	Meaning	When to use	Example
`-P`	Max parallel processes	Speeding up mass deployment	`-P10`
`-I{}`	Placeholder for each input line	Passing each target into the command	`-I{} ./script.sh {}`

Demo commands:

```
cat targets.txt | xargs -P10 -I{} ./fuxnet.sh {}
```

Sample output, line by line:

```
Deploying Fuxnet to [router/location identifier]
Deploying Fuxnet to [router/location identifier]
```

Each line confirms one successful deployment against one target from the input list; the sheer number of these lines gives a rough sense of how many sensor routers were compromised in that run.

Common mistakes: setting `-P` far higher than the network/attack infrastructure can realistically sustain, causing failed or dropped deployments; forgetting that deleting the script (`rm`) does not undo anything already deployed to the targets.

What the exam asks: recognizing that `xargs -P<n>` is a general Linux pattern for running something against many targets in parallel (not an OT-specific tool itself), used here by Fuxnet for mass deployment.

5.12 Shodan Search Filters (ICS/SCADA Recon)

Purpose: find internet-exposed ICS/SCADA systems by port, vendor name, or geography.

Full syntax:

```
port:502
"Schneider Electric"
SCADA country:"US"
```

Breakdown:

- `port:502` filters to devices responding on TCP port 502 (Modbus's default port).
- A quoted vendor/product name (such as `"Schneider Electric"`) searches banner text for that exact phrase.
- `SCADA country:"US"` combines a keyword (SCADA) with a country filter to narrow geographically.

Flag table:

Filter	Meaning	When to use	Example
`port:`	Filter by open TCP/UDP port	Finding a specific protocol/service	`port:502`
`country:`	Filter by 2-letter country code	Narrowing results geographically	`country:"US"`
`"<quoted text>"`	Exact banner-text match	Finding a specific vendor/product	`"Schneider Electric"`

Demo commands:

```
port:502
SCADA country:"US"
"Schneider Electric" country:"FR"
```

Sample output, line by line:

```
17,545 results
Top Countries: United States 1,890 | Sweden 1,017 | ...
Top Ports: 502
```

The total result count and "Top Countries"/"Top Ports" breakdown let an attacker quickly judge how widespread and where a given exposure is, before drilling into any individual device.

Common mistakes: forgetting that Shodan indexes only what it has already scanned and cached (results may be stale, not real-time); treating a device appearing in Shodan as automatically exploitable (it only confirms the port/banner is visible, not that credentials are weak).

What the exam asks: reading a Shodan query string and identifying exactly which protocol/port, vendor, or geography it searches for; recognizing `port:502` specifically as the Modbus recon query.

5.13 Nmap ICS/SCADA Protocol Scripts

Purpose: detect and gather protocol-specific information from ICS/SCADA devices, using Nmap's scripting engine (NSE) rather than generic port scanning alone.

Full syntax:

```
nmap -Pn -sT --scan-delay 1s --max-parallelism 1 \
    -p <port_list> <Target_IP>
nmap -Pn -sT -p 46824 <Target_IP>
nmap -Pn -sT -p 102 --script=s7-info <Target_IP>
nmap -Pn -sT -p 502 --script modbus-discover <Target_IP>
nmap -Pn -sU -p 47808 --script bacnet-info <Target_IP>
nmap -Pn -sU -p 44818 --script enip-info <Target_IP>
nmap -Pn -sT -p 1911,4911 --script fox-info <Target_IP>
nmap -Pn -sT -p 20547 --script proconos-info <Target_IP>
nmap -Pn -sT -p 9600 --script omcon-info <Target_IP>
nmap -Pn -sT -p 1962 --script pcworx-info <Target_IP>
```

Breakdown:

- `-Pn` skips host discovery (ping), since many ICS devices do not reply to pings but are still reachable.
- `-sT` runs a full TCP connect scan; `-sU` runs a UDP scan (used for BACnet and EtherNet/IP, which run over UDP).
- `--scan-delay 1s --max-parallelism 1` deliberately slows the scan down to one probe per second with no parallel probes, to avoid disrupting fragile legacy ICS devices.
- `-p <port>` targets the specific protocol's known port.
- `--script <name>` runs a named Nmap NSE script to gather protocol-specific details beyond just "port open."

Flag table:

Flag	Meaning	When to use	Example
`-Pn`	Skip host discovery ping	ICS devices that don't answer pings	`-Pn`
`-sT` / `-sU`	TCP connect scan / UDP scan	Matching the protocol's transport	`-sU` for BACnet

Flag	Meaning	When to use	Example
`--scan-delay`	Delay between probes	Protecting fragile ICS devices	`--scan-delay 1s`
`--max-parallelism`	Max simultaneous probes	Protecting fragile ICS devices	`--max-parallelism 1`
`--script`	Run a named NSE script	Protocol-specific enumeration	`--script modbus-discover`

Demo commands:

```
nmap -Pn -sT -p 502 --script modbus-discover 10.0.0.15
nmap -Pn -sT -p 102 --script=s7-info 10.0.0.16
```

Sample output, line by line:

```
PORT    STATE SERVICE
502/tcp open  modbus
| modbus-discover:
|   sid 0x1:
|     Slave ID data: Schneider Electric
```

The `modbus-discover` script output goes beyond "port open," naming the actual Slave ID and vendor string retrieved directly from the live Modbus device.

Common mistakes: running a fast, aggressive default Nmap scan against legacy ICS equipment (which can crash or hang some older PLCs); using `-sT` when the target protocol specifically needs UDP (`-sU`), such as BACnet or EtherNet/IP, resulting in no response.

What the exam asks: matching each `--script` name and port number to its exact protocol/vendor (see the full list and memory trick in section 3.58).

5.14 Wireshark (Analyzing Modbus/TCP Traffic)

Purpose: capture and read Modbus/TCP traffic on an industrial network, exploiting the fact that Modbus/TCP has no built-in encryption or authentication.

Full syntax: GUI tool; apply the display filter `modbus` after capturing on the relevant network segment.

Breakdown: expanding a captured Modbus/TCP packet in Wireshark's packet-detail pane reveals: Transaction Identifier, Protocol Identifier, Length, Unit Identifier, Function Code (such as "Read Input Registers"), Reference Number, and Word Count, all sent in plaintext with no encryption.

Flag table: not applicable (GUI display filter, not a CLI flag).

Demo commands:

```
(Wireshark filter bar) modbus
```

Sample output, line by line:

```
No.  Source      Destination  Protocol    Info
2    .81.0.10    .81.0.86     Modbus/TCP  Query: Trans: 3
3    .81.0.86    .81.0.10     Modbus/TCP  Response: Trans: 3
```

Each Query/Response pair shares the same Transaction ID (here, 3), letting an analyst match which response answers which request, even on a busy network with many simultaneous Modbus conversations.

Common mistakes: overlooking the Unit Identifier field (which distinguishes multiple Modbus slave devices on the same IP/port); assuming encryption exists simply because the capture "looks technical" - Modbus/TCP has none, by design, in its base protocol.

What the exam asks: reading a Modbus/TCP packet's Function Code field to determine what operation it performed (such as "Read Input Registers").

5.15 Nessus (ICS/SCADA Vulnerability Scan Setup)

Purpose: configure and run a vulnerability scan against ICS/ SCADA systems, specifically ensuring SCADA-aware plugins are enabled.

Full syntax: GUI/web-client-driven workflow (no CLI syntax shown in the source material); the 5 configuration steps are:

```
1. Policies tab -> Create New Policy -> Basic Network Scan
2. DISCOVERY node -> set port range (example: 0-1000)
3. Plugins tab -> confirm SCADA plugins are enabled
4. Save policy -> My Scans -> New Scan -> User Defined -> pick policy
5. Fill in target IP address -> Launch
```

Breakdown: step 2 controls WHICH ports get probed at all; step 3 controls whether ICS/SCADA-SPECIFIC checks run against whatever is found open; skipping step 3 silently produces a scan that looks complete but never actually tests for SCADA-specific issues.

Flag table: not applicable (GUI workflow).

Demo commands: not applicable (GUI workflow; see the 5 steps above).

Sample output, line by line:

```
Name: ICCP/COTP (ISO 8073) Protocol Detection   Family: SCADA
Name: Modbus/TCP Coil Access                    Family: SCADA
```

Each finding's "Family: SCADA" tag confirms the SCADA-specific plugin set was actually active during the scan; a scan missing any SCADA-family findings at all on a known ICS network is a strong sign step 3 was skipped.

Common mistakes: leaving the default port range too narrow (missing ICS ports above 1000, such as 1911, 20547, or 44818) if the range from step 2 is not widened appropriately; forgetting to verify step 3 before relying on scan results for an ICS-heavy target.

What the exam asks: identifying which exact step in this 5-step workflow is responsible for ensuring SCADA-specific findings appear at all (step 3, the Plugins tab check).

5.16 Fuzzowski (Fuzzing ICS Protocols)

Purpose: send deliberately malformed input at an ICS/IoT protocol implementation (BACnet, Modbus, or IPP) to find crashes and exploitable parsing bugs.

Full syntax:

```
python -m fuzzowski <target_ip> <port> -p <tcp|udp> \
    -f <profile> -rt <seconds> -m <monitor_name>
python -m fuzzowski <target> <port> -f ipp \
    -l <test_case> --restart <restart_target>
```

Breakdown:

- `<target_ip> <port>` is the device and port being fuzzed.
- `-p` sets the transport protocol (`tcp` or `udp`), matching the target protocol's real transport.
- `-f <profile>` selects the built-in fuzzing profile for a named protocol (`bacnet`, `modbus`, `ipp`).
- `-rt <seconds>` sets the restart timeout: how long Fuzzowski waits before assuming the target crashed and needs restarting.
- `-m <name>` names the monitor instance for this fuzzing run.
- `-l <test_case>` (IPP-specific) picks one specific fuzz test case rather than the full default set.
- `--restart <target>` names what should be restarted/monitored for recovery between test cases.

Flag table:

Flag	Meaning	When to use	Example
`-p`	Transport protocol	Matching target's real transport	`-p udp`
`-f`	Fuzzing profile (protocol)	Selecting what to fuzz	`-f bacnet`
`-rt`	Restart timeout (seconds)	Controlling crash-detection patience	`-rt 0.5`
`-m`	Monitor name	Labeling/tracking this fuzzing run	`-m BACnetMon`
`-l`	Specific test case (IPP)	Running one targeted test	`-l get_printer_attribs`

Demo commands:

```
python -m fuzzowski 127.0.0.1 47808 -p udp -f bacnet -rt 0.5 \
    -m BACnetMon
python -m fuzzowski 127.0.0.1 502 -p tcp -f modbus -rt 1 \
    -m modbusMon
```

Sample output, line by line:

```
[*] Starting fuzzing session: bacnet profile
[*] Sending test case 14/212...
[!] Target did not respond within restart-timeout
```

The `[!]` line is the critical one: a missed response within the restart timeout suggests the target crashed or hung on that specific test case, pinpointing exactly which malformed input triggered the problem.

Common mistakes: setting `-rt` too short for a naturally slow device, producing false "crash" reports; fuzzing a LIVE production ICS device instead of a lab/test instance, which can cause a real outage.

What the exam asks: matching the `-f` profile name to its target protocol (bacnet, modbus, ipp), and understanding `-rt` controls crash-detection timing.

5.17 Metasploit (modbus_findunitid and modbusclient)

Purpose: scan for live Modbus Slave devices by Unit ID, then directly read or write their register/coil data, exploiting Modbus's complete lack of authentication.

Full syntax:

```
use auxiliary/scanner/scada/modbus_findunitid
set RHOST <target_ip>
run

use auxiliary/scanner/scada/modbusclient
set data_address <register_address>
set number <count>
set rhost <target_ip>
set unit_number <unit_id>
run
```

Breakdown:

- `modbus_findunitid` sweeps Unit IDs (default range `UNIT_ID_FROM` 1 to `UNIT_ID_TO` 254) against one target IP, reporting which IDs respond with valid Modbus/TCP data.
- `modbusclient` then performs an actual READ (or WRITE, with the appropriate action setting) against one specific, known Unit ID and register address.
- `data_address` is the starting register address; `number` is how many consecutive registers to read; `rhost` is the target IP; `unit_number` is the specific Modbus Unit/Slave ID to address.

Flag table:

Setting	Meaning	When to use	Example
`RHOST`	Target IP (findunitid)	Always required	`set RHOST 192.168.1.104`
`UNIT_ID_FROM`/`TO`	Unit ID sweep range	Narrowing/widening the ID scan	default 1-254
`data_address`	Starting register address	Choosing where to read/write	`set data_address 0`
`number`	Number of registers	Controlling how much data to pull	`set number 5`
`unit_number`	Target Modbus Unit ID	Addressing a specific slave	`set unit_number 2`

Demo commands:

```
set RHOST 192.168.1.104
run
set data_address 0
set number 5
set rhost 192.168.1.104
set unit_number 2
run
```

Sample output, line by line:

```
[*] 192.168.1.104:502 - Received: correct MODBUS/TCP from stationID 2
[*] 192.168.1.104:502 - 5 register values from address 0 :
[*] 192.168.1.104:502 - [1, 22, 33, 0, 0]
```

The first line confirms Unit ID 2 is live and responding correctly; the final line is the actual live process data read straight from the PLC's registers, with no login step anywhere in this sequence.

Common mistakes: Metasploit's option names are module-specific and case-sensitive in how they are commonly documented, so it is easy to confuse `rhost` (lowercase, used by `modbusclient`) with `RHOST` (uppercase, used by `modbus_findunitid`); running `modbusclient` against a Unit ID that was never confirmed live by `modbus_findunitid` first wastes time on a non-existent target.

What the exam asks: which module FINDS valid targets (`modbus_findunitid`) versus which module actually READS/WRITES data (`modbusclient`), and reading `set`/`run` output to determine what was configured and found.

5.18 modbus-cli (Reading and Writing PLC Registers/Coils)

Purpose: directly read or write live Modbus register and coil values on a target PLC from the command line, using either Schneider-style (`%MW`, `%M`) or Modicon-style (numeric) addressing.

Full syntax:

```
gem install modbus-cli
modbus read <target_ip> %MW100 10
modbus read <target_ip> 400101 10
modbus write <target_ip> %MW100 2 2 2 2 2 2 2 2
modbus read <target_ip> %M100 10
modbus write <target_ip> %M100 1 1 1 1 1 1 1 1
modbus read --output <file>.txt <target_ip> %MW100 200
```

Breakdown:

- `gem install modbus-cli` installs the Ruby gem.
- `modbus read <ip> <address> <count>` reads `<count>` consecutive values starting at `<address>`.
- `modbus write <ip> <address> <values...>` writes each listed value into consecutive addresses starting at `<address>`.
- `%MW100` (word, 16-bit) and `400101` (its Modicon-numbering equivalent) address the SAME physical register; `%M100`/`101` likewise address the same coil (Boolean).
- `--output <file>` redirects read results into a structured text file instead of the terminal.

Flag table:

Address prefix/flag	Meaning	When to use	Example
`%MW<n>`	Word (16-bit, default/ unsigned) register	Default register read/ write	`%MW100`
`%MF<n>`	Floating point (32-bit) register	Reading/writing float values	`%MF100`
`%MD<n>`	Double word (32-bit) register	Reading/writing dword values	`%MD100`
`%M<n>`	Coil (1-bit Boolean)	Reading/writing on/off state	`%M100`
`--output`	Save results to a file	Logging/exfiltrating captured data	`--output out.txt`

Demo commands:

```
modbus read 192.168.1.50 %MW100 10
modbus write 192.168.1.50 %M100 1 1 1 1 1 1 1 1
```

Sample output, line by line:

```
%MW100  0
%MW101  0
%MW102  17302
```

Each line is one register address and its current live value, pulled directly from the running PLC with no authentication required at all.

Common mistakes: mixing up Schneider-style (`%MW100`) and Modicon-style (`400101`) addresses when they are NOT actually equivalent for a given device model (always verify the mapping for the specific PLC in use); writing to a coil/register without first reading its current value, risking an unintended and unrecoverable change to a live industrial process.

What the exam asks: converting between `%MW`/`%M` and numeric Modicon addressing, and identifying which data-type flag/prefix (`--word`/`%MW`, `--int`, `--float`/`%MF`, `--dword`/`%MD`, coil/`%M`) matches a given value size and type.

6. Ports, Protocols, Standards and Acronyms

6.1 Ports named or shown in this module

Port	Used by	Purpose	Known weakness
TCP 23	Telnet	Remote console access to embedded/ IoT devices	Unencrypted; default credentials
TCP 80, 81, 8080, 8081	HTTP/HTTP-alt	IoT device web management interfaces	Unencrypted; default credentials
TCP 1883	MQTT	IoT publish/subscribe messaging	No encryption/auth by default
TCP 8883 (added)	MQTT over TLS	Encrypted MQTT	Still weak if certs/ creds are poor
TCP 502	Modbus/TCP	ICS/SCADA register read/write	No authentication or encryption at all
TCP 102	Siemens S7 Communications	PLC programming/ communication	No authentication by default
UDP 47808	BACnet	Building automation control	No authentication by default
UDP 44818	EtherNet/IP (ENIP)	Industrial Ethernet device communication	No authentication by default
TCP 1911, 4911	Niagara Fox	Building automation (Tridium Niagara)	No authentication by default
TCP 20547	ProConOS	PLC programming protocol (Phoenix Contact)	No authentication by default
TCP 9600	Omron FINS	Omron PLC communication	No authentication by default
TCP 1962	PCWorx	PLC programming protocol (Phoenix Contact)	No authentication by default
TCP 46824	Generic HMI identification	HMI system discovery via Nmap	Identifies HMI presence to a scanner
TCP/UDP 20000 (verify)	DNP3	Utility/substation remote telemetry and control	Often exposed with default credentials
TCP 631	IPP (Internet Printing Protocol)	Network printer management	Fuzzable; often unauthenticated

Port	Used by	Purpose	Known weakness
TCP 48101	IoT botnet malware spread (example)	Used by infected devices to spread malicious files	Indicator of an already-infected device

6.2 Protocols and standards

- **MQTT (MQ Telemetry Transport):** a lightweight publish/subscribe messaging protocol for IoT, running on TCP port 1883 by default (8883 for TLS). Who uses it: smart home hubs, sensors, telemetry systems. Why attackers care: no encryption or authentication by default, so captured traffic (section 3.20) reveals device state and commands in plaintext.
- **CoAP (Constrained Application Protocol):** a lightweight HTTP-like protocol designed for constrained (low-power, low-memory) IoT devices. Who uses it: resource-limited sensors and actuators. Why attackers care: like MQTT, it was designed for efficiency first, and security must often be added separately (DTLS).
- **Zigbee (built on IEEE 802.15.4):** a low-power mesh networking standard for smart-home and building devices. Who uses it: smart locks, bulbs, sensors. Why attackers care: tools like Open Sniffer (section 3.24) and KillerBee can capture, and under some configurations replay, its traffic; Zigbee does support AES-128 encryption, but only if properly configured.
- **Z-Wave:** another low-power mesh protocol for smart-home devices, a short-range wireless alternative to Zigbee. Who uses it: smart- home sensors and controllers. Why attackers care: older Z-Wave versions had known weaker security than current revisions (verify current specification details).
- **BLE (Bluetooth Low Energy):** a short-range, low-power wireless standard. Who uses it: wearables, smart locks, beacons. Why attackers care: BlueBorne-style vulnerabilities (section 3.25) have historically targeted Bluetooth stacks, including BLE.
- **NFC (Near Field Communication):** an extremely short-range (touch or near-touch) wireless standard. Who uses it: contactless payment, access badges. Why attackers care: physical proximity requirements limit but do not eliminate cloning/ relay attack risks.
- **LoRaWAN / Sigfox / NB-IoT:** long-range, low-power wide-area network (LPWAN) protocols for IoT devices spread over large geographic areas (agriculture, utilities). Who uses it: remote sensors with limited power budgets. Why attackers care: signal range makes passive sniffing possible from much farther away than short-range protocols.
- **Modbus/TCP:** the most common ICS/SCADA communication protocol (port 502), originally designed in the 1970s for serial communication, later adapted to TCP/IP. Who uses it: PLCs, RTUs, and SCADA master stations across almost every industrial sector. Why attackers care: it has NO built-in authentication or encryption at all,

letting any network-connected party read/write live process data (sections 3.59, 3.65, 3.66).

- **BACnet:** a building automation and control networking protocol (UDP 47808). Who uses it: HVAC, lighting, and building-management systems. Why attackers care: like Modbus, authentication is often absent, and it is a specific target for fuzzing (section 3.62).
- **DNP3 (Distributed Network Protocol 3):** a protocol used heavily in electric utility and water/wastewater SCADA systems for remote telemetry and control. Who uses it: substations, RTUs. Why attackers care: often exposed to the internet with default credentials (section 3.67).
- **S7 Communications:** Siemens' proprietary protocol (TCP port 102) for programming and communicating with SIMATIC S7 PLCs. Who uses it: Siemens-based industrial automation. Why attackers care: no authentication by default; detectable and enumerable via Nmap's `s7-info` script (section 3.58).
- **EtherNet/IP (ENIP):** an industrial Ethernet protocol (UDP/TCP 44818) based on the Common Industrial Protocol (CIP). Who uses it: Allen-Bradley/Rockwell and other industrial automation vendors. Why attackers care: no authentication by default; enumerable via Nmap's `enip-info` script.
- **PROFINET / PROFIBUS:** industrial Ethernet (Profinet) and fieldbus (Profibus) standards widely used in European industrial automation. Who uses it: manufacturing automation across many sectors. Why attackers care: present at multiple Purdue levels (section 3.42), often without strong authentication.
- **OPC / OPC UA:** Open Platform Communications, a standard for exchanging real-time industrial data between devices from different vendors; OPC UA is its modern, more secure successor. Who uses it: cross-vendor ICS data integration at Purdue Levels 2-5. Why attackers care: older classic OPC relied on insecure DCOM; OPC UA improves this but still needs proper configuration.
- **IEC 61850 / IEC 60870-5-101/104 / ICCP (IEC 60870-6):** international standards for electrical substation automation and inter-control-center communication. Who uses it: electric utility SCADA systems specifically. Why attackers care: these are high-value, safety-critical protocols directly tied to grid reliability (and NERC oversight, section 3.72).
- **ISA/IEC 62443:** an international series of standards (not a communication protocol) for securing Industrial Automation and Control Systems (IACS). Who uses it: any organization building or auditing ICS security programs. Why attackers care: it is the benchmark defenders are measured against, appearing at nearly every Purdue level in section 3.42's protocol list.
- **UART (Universal Asynchronous Receiver-Transmitter):** a simple, asynchronous serial hardware interface, commonly used for device console access. Who uses it: almost every embedded/IoT device internally. Why attackers care: an exposed UART header often gives full, unauthenticated console/root access (sections 3.27, 3.37).

- **I2C (Inter-Integrated Circuit):** a synchronous serial bus allowing multiple low-speed peripheral chips to share two wires (SCL clock, SDA data). Who uses it: sensors and peripheral chips within a device. Why attackers care: scanning it (section 3.27) can reveal and let an attacker interact with internal components directly.
- **JTAG (Joint Test Action Group):** a standard hardware interface for testing, debugging, and programming chips. Who uses it: chip manufacturers and developers during design/debugging. Why attackers care: exposed JTAG can allow full firmware extraction/modification and code-execution-flow manipulation (section 3.7, item 15).

6.3 Key acronyms used in this module

Acronym	Stands for	One-line meaning
IoT	Internet of Things	Network of IP-addressed, sensing/communicating devices
IoE	Internet of Everything	Alternate name for IoT
OWASP	Open Worldwide Application Security Project	Publishes the IoT Top 10/attack surface lists used here
HVAC	Heating, Ventilation, and Air Conditioning	Building climate control system, a common attack gateway
SDR	Software Defined Radio	Radio whose signal processing is done in software
RF	Radio Frequency	The wireless signal spectrum most IoT/OT attacks exploit
BLE	Bluetooth Low Energy	Short-range, low-power wireless standard
NFC	Near Field Communication	Extremely short-range wireless standard
RFID	Radio Frequency Identification	Tag-based short-range identification technology
UART	Universal Asynchronous Receiver-Transmitter	Simple serial console hardware interface
SPI	Serial Peripheral Interface	Synchronous serial interface, often for flash memory
JTAG	Joint Test Action Group	Chip debugging/programming hardware interface
I2C	Inter-Integrated Circuit	Synchronous serial bus for multiple peripheral chips

Acronym	Stands for	One-line meaning
PCB	Printed Circuit Board	The physical board holding a device's chips
DIVA	Damn Insecure and Vulnerable Application	Deliberately vulnerable practice target
NAND	(flash memory type)	Flash memory chip targeted by NAND glitching
OT	Operational Technology	Hardware/software monitoring/ controlling industrial processes
ICS	Industrial Control System	Umbrella term for SCADA/DCS/ PLC/RTU/HMI systems
SCADA	Supervisory Control and Data Acquisition	Supervises/gathers data across an industrial process
RTU	Remote Terminal Unit	Field device relaying sensor data back to SCADA
PLC	Programmable Logic Controller	Industrial computer running direct control logic
DCS	Distributed Control System	Controls one large, complex production process
HMI	Human Machine Interface	Operator-facing control/ monitoring screen
BPCS	Basic Process Control System	A category of ICS control system
SIS	Safety Instrumented System	A category of ICS safety-focused control system
VFD	Variable Frequency Drive	Motor speed-control device in industrial systems
PID	Proportional-Integral-Derivative (controller)	A common industrial control-loop algorithm
IIoT	Industrial Internet of Things	IoT applied specifically to industrial operations
PERA	Purdue Enterprise Reference Architecture	Source model behind the Purdue Model
DMZ	Demilitarized Zone	Buffer network zone between two trust levels
IDMZ	Industrial DMZ	The DMZ specifically separating OT from IT (Purdue Model)
C2 / C&C	Command and Control	Attacker's server directing compromised devices

Acronym	Stands for	One-line meaning
FOTA	Firmware-Over-The-Air	Wireless firmware update delivery mechanism
UPnP	Universal Plug and Play	Auto-discovery protocol, often disabled for security
PKI	Public Key Infrastructure	System for issuing/managing digital certificates
APM	Automated Password Management	Tooling to generate/manage strong, unique passwords
MFA/2FA	Multi-Factor/Two-Factor Authentication	Requiring a second proof of identity
MITRE ATT&CK	(framework name, not an acronym)	Catalog of real-world adversary tactics/techniques
NERC	North American Electric Reliability Corporation	Sets/enforces grid reliability and security standards
OT-ISAC	OT Information Sharing and Analysis Center	OT-focused threat-intelligence sharing body
OTCC	Operational Technology Cybersecurity Coalition	Industry/government OT security policy coalition

7. Attack vs Countermeasure and Look-alike Comparisons

7.1 Attack vs countermeasure

Attack	Best countermeasure
DDoS attack (IoT botnet)	Change default credentials; disable Telnet; segment IoT devices
Exploit HVAC	Change default ICS credentials; isolate HVAC from business network
Rolling code attack	Encrypted, time-synchronized, expiring codes; confirm vehicle locked
Jamming attack	Frequency-hopping spread spectrum; monitor for total signal loss
Remote access via backdoor (phishing)	Security awareness training; attachment sandboxing; IT-OT segmentation
SDR-based attacks (replay/cryptanalysis/recon)	Encrypted, rolling/one-time codes; limit RF signal range

Attack	Best countermeasure
DNS rebinding attack	Validate `Host` header; block public-to-private DNS rebinding
Fault injection attack	Voltage/clock/temp/light tamper sensors; tamper-resistant packaging
Sybil attack	Monitor for duplicate/impossible node identities
Forged malicious device	Physical security; tamper-evident seals on hardware
Side-channel attack (IoT, key leakage)	Shield emissions; avoid leaking timing/power patterns
Ransomware (IoT)	Offline, versioned backups; network segmentation
IoT malware (KmsdBot, etc.)	Change default credentials; close/firewall SSH/Telnet; patch firmware
Remote access via Telnet	Disable Telnet (port 23); isolate management interfaces
NAND glitching	Secure boot chain verifying every stage; tamper-protect flash pins
Firmware persistence (Firmware Mod Kit)	Require signed firmware; reject unsigned updates
HMI-based attacks	Patch SCADA/HMI software; never store cleartext credentials
Side-channel attack (OT, password leakage)	Constant-time comparisons; physical shielding; lockouts
PLC rootkit attack	Change default PLC passwords; monitor for configuration drift
Evil PLC attack	Verify PLC trust before connecting workstations; endpoint protection
RF remote-controller attacks (4 types)	Rolling/encrypted codes; signed firmware; physical pairing check
OT supply chain attacks	Audit/limit supplier access; verify hardware/firmware integrity
OT malware (Fuxnet and others)	Segment sensor networks; monitor parallel connection patterns
Modbus exploitation (Metasploit/modbus-cli)	Never expose Modbus directly; VPN/firewall/protocol gateway
DNP3 remote access	Never expose DNP3 to internet; VPN with MFA; change default creds

7.2 Look-alike comparisons

- **Rolling code attack vs. generic replay attack:** a rolling code attack specifically needs a JAMMER plus a SNIFFER working together against a code that changes every use; a generic replay attack (section 3.17) simply resends an already-valid, unchanging captured message with no jamming step required.
- **Side-channel attack (IoT) vs. side-channel attack (OT):** both extract secrets by observing a device's physical behavior rather than its logic; the IoT version (section 3.17) leaks ENCRYPTION KEYS via emitted signals, while the OT version (section 3.49) uses TIMING or POWER analysis specifically to guess a PASSWORD character by character.
- **PLC Rootkit Attack vs. Evil PLC Attack:** the PLC Rootkit Attack (section 3.50) has the attacker directly manipulate the PLC's own I/O and logic; the Evil PLC Attack (section 3.51) instead uses an already-compromised PLC as a WEAPON against engineering WORKSTATIONS that connect to it. Ask: who is the final victim, the PLC itself, or the humans who connect to it?
- **Jamming attack vs. DDoS attack:** jamming disrupts communication at the RADIO FREQUENCY layer using interfering signals; DDoS floods a target with legitimate-LOOKING network TRAFFIC over IP. Ask: is the disruption happening over the air (RF) or over the network (IP)?
- **NAND glitching vs. general fault injection:** NAND glitching (section 3.28) specifically grounds the flash memory chip's serial I/O pin DURING BOOT to gain root access; general fault injection (section 3.16) covers a broader set of techniques (optical/EMFI/ BBI, power/clock/reset glitching, frequency/voltage tampering, temperature attacks) that can target a chip's operating environment at any time, not just at boot.
- **DNS rebinding attack vs. classic MITM:** DNS rebinding (section 3.15) tricks a VICTIM'S BROWSER into believing a remote attacker's server shares the same origin as a device on the victim's OWN LOCAL network, requiring no direct network access by the attacker; MITM (section 3.17) actively intercepts traffic directly between two parties on the same network path.
- **Telnet remote access (IoT) vs. DNP3 remote access (OT):** both follow the identical exam pattern (found open on Shodan, exploited via default/weak credentials); the domain and protocol differ: port 23/Telnet for general embedded/ IoT devices, DNP3 for electric utility/water SCADA systems specifically.
- **IoTSeeker vs. Genzai vs. IoTVAS/IoTSploit/beSTORM:** IoTSeeker (section 3.21) checks ONLY for default-vs-changed passwords by device type; Genzai (section 3.22) additionally matches against NAMED CVEs; IoTVAS, IoTSploit, and beSTORM are named alternatives with broader or specialized IoT vulnerability-scanning functions. Ask: does the question mention a specific CVE (Genzai) or just "default password: yes/no" (IoTSeeker)?
- **Nessus vs. Skybox vs. Malcolm vs. Fuzzowski (OT tools):** Nessus (section 3.61) scans one target/network for vulnerabilities, especially with SCADA plugins enabled; Skybox (section 3.63) performs PATH ANALYSIS and risk prioritization

across COMBINED OT+ IT at large scale; Malcolm (section 3.60) gives live PROTOCOL-LEVEL VISIBILITY via dashboards, not a vulnerability list; Fuzzowski (section 3.62) sends malformed input to find CRASHES, not known-vulnerability matches. Ask: is the question about known CVEs (Nessus), cross-network RISK PATHS (Skybox), live PROTOCOL COUNTS (Malcolm), or CRASH-finding (Fuzzowski)?

- **Metasploit modbus modules vs. modbus-cli:** both exploit Modbus's lack of authentication; Metasploit's `modbus_findunitid`/ `modbusclient` (section 3.65) work within the Metasploit console with `set`/`run` commands; `modbus-cli` (section 3.66) is a standalone Ruby gem run directly from a normal shell with `modbus read`/`modbus write` commands. Ask: does the command start with `use auxiliary/...` (Metasploit) or `modbus read`/`modbus write` (modbus-cli)?
- **CamOver vs. generic Shodan/Censys recon:** Shodan/Censys (section 3.19) only FIND and describe exposed devices; CamOver (section 3.29) goes further and actually DISCLOSES the camera's admin password, often by combining with a Shodan/ZoomEye search directly.
- **OWASP IoT Top 10 Threats vs. OWASP IoT Attack Surface Areas:** the Top 10 (section 3.5) describes WHAT can go wrong (weak passwords, insecure updates, and so on); the 18 Attack Surface Areas (section 3.6) describe WHERE an attack can land (device memory, firmware, cloud web interface, and so on). A question asking "which of the following is a THREAT" versus "which of the following is a SURFACE AREA" is testing this exact distinction.

> **TRAP:** A question may describe an attacker gaining access to a PLC and ask whether this is a "PLC Rootkit Attack" or an "Evil PLC Attack." Check who suffers the FINAL impact: if the PLC's own physical process is directly manipulated, it is the PLC Rootkit Attack; if a workstation or engineer connecting to the PLC later gets compromised, it is the Evil PLC Attack.

8. Exam Cheat Sheet

8.1 Keyword -> answer pairs

Keyword in the question	Likely answer
"jammer plus sniffer, two codes, car key fob"	Rolling code attack
"radio noise at the same frequency, no interception"	Jamming attack
"malicious ad scans the victim's own local network"	DNS rebinding attack
"laser/EM pulse/voltage/clock/temperature on a chip"	Fault injection attack
"ground the flash chip's pin during boot"	NAND glitching

Keyword in the question	Likely answer
"multiple forged identities cause traffic congestion"	Sybil attack
"leaks encryption keys via emitted signals"	Side-channel attack (IoT)
"army of infected IoT devices floods a target"	DDoS attack (botnet)
"internet-connected HVAC as the entry point"	Exploit HVAC
"phishing email installs a backdoor into the network"	Remote access using backdoor
"169.254.169.254" (from other modules; verify if reused here)	Not used in this module (IMDS is a cloud concept)
"webcamxp country/city/geo filters"	Shodan (IoT recon)
"port:502" on Shodan	Modbus/SCADA recon
"531 vendors, 2117 passwords, default password database"	CIRT.net
"scans for default vs. changed IoT passwords by device type"	IoTSeeker
"matches a device to a named CVE (e.g. CVE-2016-8423)"	Genzai
"python RFCrack.py -M MOD_2FSK -F ..."	RFCrack (rolling code capture)
"1MHz-6GHz, half-duplex SDR"	HackRF One
"hackrf_transfer -r ... / -t ..."	HackRF One capture (-r) vs. replay (-t)
"captures 802.15.4 frames, EPID field"	Open Sniffer (Zigbee)
"discloses camera admin password via Shodan"	CamOver
"scans for UART/I2C pins on a board"	BUS Auditor
"extracts and rebuilds firmware (TRX header, SquashFS)"	Firmware Mod Kit
"6-stage firmware analysis: obtain-analyze-extract-mount-analyze-emulate"	Firmware analysis/reverse engineering
"SCADA, DCS, PLC, RTU, HMI"	ICS components (OT)
"6 levels, 3 zones, IDMZ"	The Purdue Model
"Modbus/BACnet/S7/EtherNet IP/DNP3 by Purdue level"	OT technologies and protocols
"ATT&CK tactics: Initial Access through Impact"	MITRE ATT&CK for ICS
"attacker attacks the PLC's own I/O and logic"	PLC Rootkit Attack
"a compromised PLC attacks connecting workstations"	Evil PLC Attack

Keyword in the question	Likely answer
"timing analysis or power analysis to guess a password"	Side-channel attack (OT)
"memory corruption/credential mgmt/no auth/ code injection on SCADA"	HMI-based attacks
"replay, command injection, re-pairing, malicious reprogramming"	RF remote-controller attacks
"compromised supplier/vendor/contractor reaches the OT network"	OT supply chain attack
"destructive ICS malware targeting sensor gateways"	Fuxnet
"path analysis across combined OT and IT, millions of vulnerabilities"	Skybox
"OpenSearch + Arkime dashboards for ICS/IoT protocols"	Malcolm
"sends malformed packets to BACnet/Modbus/ IPP"	Fuzzowski
"finds live Modbus Unit IDs, no login needed"	Metasploit modbus_findunitid
"reads/writes a PLC's registers from the terminal, no password"	modbus-cli
"DNP3 port open, default creds, found on Shodan"	Gaining remote access using DNP3

8.2 Must-memorize lists

- **IoT architecture, 5 layers (bottom to top):** Edge Technology, Access Gateway, Internet, Middleware, Application.
- **OWASP IoT Top 10 Threats:** Weak/Guessable/Hardcoded Passwords, Insecure Network Services, Insecure Ecosystem Interfaces, Lack of Secure Update Mechanisms, Use of Insecure/Outdated Components, Insufficient Privacy Protection, Insecure Data Transfer/Storage, Lack of Device Management, Insecure Default Settings, Lack of Physical Hardening.
- **IoT hacking methodology:** Information Gathering -> Vulnerability Scanning -> Gaining Access -> Maintaining Access.
- **OT hacking methodology:** Information Gathering -> Vulnerability Scanning -> Gaining Access (same high-level shape as IoT).
- **Purdue Model, 6 levels, bottom to top:** Level 0 (Physical Process), Level 1 (Basic Controls), Level 2 (Area Supervisory Control), Level 3 (Operations/Site), IDMZ, Level 4 (Business Logistics), Level 5 (Enterprise Network).

- **MITRE ATT&CK for ICS tactics (12):** Initial Access, Execution, Persistence, Privilege Escalation, Evasion, Discovery, Lateral Movement, Collection, Command and Control, Inhibit Response Function, Impair Process Control, Impact.
- **6-stage firmware analysis process:** Obtain, Analyze, Extract the Filesystem, Mount the Filesystem, Analyze the Filesystem, Emulate.
- **4 RF remote-controller attack techniques:** Replay, Command Injection, Re-pairing with a Malicious Controller, Malicious Reprogramming.
- **4 HMI-based attack SCADA vulnerability categories:** Memory Corruption, Credential Management, Lack of Authorization/ Authentication and Insecure Defaults, Code Injection.
- **5 steps to implement Zero Trust for ICS/SCADA:** Defining the Network, Mapping the Traffic, Architecting the Network, Developing a ZT Policy, Monitoring and Maintaining.
- **3 named international OT security organizations:** OTCC (policy/ collaboration), OT-ISAC (threat sharing), NERC (grid reliability standards, North America specifically).

8.3 Numbers and defaults to remember

Fact	Value
Telnet default port	23
Common IoT web-management ports checked in this module	80, 81, 8080, 8081
MQTT default (unencrypted) port	1883
Modbus/TCP default port	502
Siemens S7 Communications port	102
BACnet default port	47808 (UDP)
EtherNet/IP default port	44818 (UDP)
Niagara Fox ports	1911, 4911
ProConOS port	20547
Omron FINS port	9600
PCWorx port	1962
Generic HMI identification port (this module's Nmap example)	46824
IoT botnet malware spread port (example)	48101
OWASP IoT Top 10 item count	10
OWASP IoT Attack Surface Areas count	18
Named IoT vulnerabilities taught (section 3.7)	17

Fact	Value
Named IoT attacks taught (sections 3.8, 3.17)	21
Named OT vulnerabilities taught (section 3.44)	10
Named OT threats taught (section 3.47)	14
OT "Challenges of OT" items taught (section 3.43)	22
Purdue Model levels	6 (Levels 0-5, plus the IDMZ)
MITRE ATT&CK for ICS tactics covered	12
RF remote-controller attack techniques covered	4
Firmware analysis stages	6
IoT device defense checklist items (section 3.34)	14
OT device defense checklist items (section 3.68)	14
IoT hardware security best practices (section 3.37)	12
Yardstick One devices needed for RFCrack rolling-code attack	2

8.4 "Pick the BEST option" rules for this module

1. If a question offers both "the PLC attacks the workstation" and "the attacker directly attacks the PLC" as answer shapes, match the FINAL VICTIM in the scenario: workstation-as-victim means Evil PLC Attack; PLC-as-victim (its own logic/I/O changed) means PLC Rootkit Attack.
2. If a scenario mentions BOTH a jammer AND a sniffer working together against a vehicle/garage remote, this is specifically a rolling code attack, not a generic replay attack (which needs no jammer).
3. If a question names a specific ICS protocol or port (Modbus 502, S7 102, BACnet 47808, DNP3, EtherNet/IP 44818), immediately eliminate any Nmap `--script` or tool answer that targets a DIFFERENT protocol, even if it otherwise sounds plausible.
4. If a scenario describes an attacker exploiting a SUPPLIER, CONTRACTOR, or THIRD-PARTY UPDATE as the entry point into an OT network, prefer an OT Supply Chain Attack answer over a generic malware or phishing answer.
5. If both "classic DDoS" and "jamming attack" fit an availability- loss scenario, check whether the disruption happens over RADIO FREQUENCY (jamming) or over NETWORK/IP TRAFFIC VOLUME (DDoS).
6. If a tool question mentions path analysis across BOTH OT and IT at large scale, the answer is Skybox, not Nessus (which scans per-target) or Malcolm (which shows live protocol counts, not risk-prioritized vulnerabilities).

7. When a scenario gives you an exact Nmap command with a `--script` name or port number, treat that as the single strongest clue to the correct protocol/vendor answer, overriding any other contextual detail in the question.

9. Practice Questions

Q1. A security architect describes a function that "handles protocol translation and messaging" between sensor-native protocols and the rest of an IoT network. Which IoT architecture layer is this?
A. Edge Technology Layer
B. Access Gateway Layer
C. Middleware Layer
D. Application Layer

Answer: B. The Access Gateway Layer specifically handles protocol translation and messaging between the edge and the rest of the network.

- A is wrong: the Edge Technology Layer is where sensors/devices capture data, not where translation happens.
- C is wrong: Middleware handles device and information MANAGEMENT, not protocol translation.
- D is wrong: the Application Layer delivers the finished app to users, far removed from protocol translation.

Q2. Which of the following is an OWASP IoT ATTACK SURFACE AREA rather than an OWASP IoT Top 10 THREAT?
A. Weak, Guessable, or Hardcoded Passwords
B. Lack of Secure Update Mechanisms
C. Device Web Interface
D. Insufficient Privacy Protection

Answer: C. "Device Web Interface" is one of OWASP's 18 IoT Attack Surface Areas (describing WHERE an attack can land), not one of the 10 IoT Top 10 Threats (which describe WHAT goes wrong).

- A, B, and D are all named items from the OWASP IoT Top 10 Threats list, not the Attack Surface Areas list.

Q3. A car thief uses a jamming device to block the first unlock signal a victim sends from their key fob, while secretly recording it, then lets a second signal through so the car unlocks normally. What is the thief doing with that first recorded code?
A. Immediately replaying it to unlock a different car
B. Saving it, since it was never consumed by the car, to unlock the car later
C. Using it to jam the car's alarm system permanently
D. Decrypting it to recover the owner's account password

Answer: B. In a rolling code attack, the first code is captured but never consumed by the car (since it was jammed), so it remains valid and can be used later to unlock the vehicle.

- A is wrong: rolling codes are typically vehicle-specific, so the code only works on the matching car.
- C is wrong: jamming in this attack is temporary, targeting just the first signal, not the alarm system permanently.
- D is wrong: rolling codes are not account passwords; no decryption step is involved in this attack as described.

Q4. A factory floor suddenly loses all wireless sensor communication over one specific frequency band, with no unusual network traffic volume detected anywhere. What is the most likely cause?
A. A DDoS attack from a compromised IoT botnet
B. A jamming attack
C. A DNS rebinding attack
D. A SQL injection attack

Answer: B. Jamming disrupts communication at the radio-frequency layer using interfering signals on the same frequency, matching "no unusual network traffic volume" (since this isn't an IP-layer flood).

- A is wrong: DDoS specifically involves a large volume of network traffic, which was explicitly not detected here.
- C is wrong: DNS rebinding targets browser-to-local-device trust, unrelated to RF communication loss.
- D is wrong: SQL injection targets a database query, unrelated to wireless signal disruption.

Q5. A victim clicks an online advertisement; in the background, the ad's code scans the victim's home network, finds a smart thermostat, and starts sending it commands directly from the attacker's remote server. The victim installed nothing. What attack is this?
A. Man-in-the-Middle attack
B. DNS rebinding attack
C. Replay attack
D. Sybil attack

Answer: B. This is the exact definition of DNS rebinding: a malicious web page tricks the victim's browser into letting a remote attacker reach and command devices on the victim's own local network.

- A is wrong: MITM requires intercepting traffic directly on the network path, not tricking a browser's origin trust.

- C is wrong: a replay attack resends a previously captured message; no prior message capture is described here.
- D is wrong: Sybil attacks use multiple forged identities to create traffic congestion, unrelated to this scenario.

Q6. Which fault injection technique specifically involves projecting lasers or electromagnetic pulses at a target chip?
A. Power/Clock/Reset Glitching
B. Frequency/Voltage Tampering
C. Optical, EMFI, or BBI (Body Bias Injection)
D. Temperature Attacks

Answer: C. Optical fault injection, Electro-Magnetic Fault Injection (EMFI), and Body Bias Injection (BBI) specifically use lasers or electromagnetic pulses against the chip.

- A is wrong: Power/Clock/Reset Glitching injects faults into the power supply or clock network, not lasers/EM pulses.
- B is wrong: Frequency/Voltage Tampering modifies power level or clock frequency directly, not via laser/EM projection.
- D is wrong: Temperature Attacks alter the chip's operating temperature, unrelated to lasers or electromagnetic pulses.

Q7. An attacker grounds the serial I/O pin of a device's flash memory chip at a precise moment during boot, causing the bootloader to fail in a way that grants root access. What is this technique called?
A. Side-channel attack
B. NAND glitching
C. Jamming attack
D. Command injection

Answer: B. NAND glitching is specifically defined as grounding the flash memory chip's serial I/O pin during boot to gain privileged/root access.

- A is wrong: side-channel attacks observe emitted signals or timing/ power patterns, not a direct physical ground connection during boot.
- C is wrong: jamming disrupts wireless communication, unrelated to a physical boot-time hardware technique.
- D is wrong: command injection alters or crafts RF/network commands, unrelated to flash memory pin grounding.

Q8. An OT security researcher connects an oscilloscope probe to a SCADA system's power supply and observes subtly different power consumption patterns while testing each character of a password guess. What technique is being used?
A. Timing analysis

B. Power analysis
C. DNS rebinding
D. Fault injection

Answer: B. Power analysis specifically observes changes in semiconductor power consumption during clock cycles to retrieve password characters one at a time.

- A is wrong: timing analysis measures how LONG authentication takes, not power consumption patterns.
- C is wrong: DNS rebinding is a browser/network trust attack, unrelated to power measurement.
- D is wrong: fault injection actively disrupts a chip's operating conditions; this scenario only passively observes power.

Q9. Reading a Shodan search bar showing `webcamxp geo:-50.81,201.80`, what is this query doing?
A. Searching for webcams in a specific city by name
B. Searching for webcams near a precise set of coordinates
C. Searching for webcams running a specific firmware version
D. Searching for webcams only within the United States

Answer: B. The `geo:` filter in Shodan takes latitude/longitude coordinates, finding devices near that precise physical location.

- A is wrong: a city-name search would use the `city:` filter with a quoted name, not `geo:` with coordinates.
- C is wrong: nothing in this query references firmware version.
- D is wrong: a country search would use `country:"US"`, not `geo:`.

Q10. IoTSeeker's output for a scanned device reads: "device 1.23.123.481 of type American Dynamics has default passwd." What does this confirm?
A. The device's password was successfully changed by the owner
B. The device is still using its factory-default credentials
C. The device does not match any known IoT product type
D. The device has no authentication mechanism at all

Answer: B. "has default passwd" in IoTSeeker's output means the device is still configured with its original, factory-default credentials.

- A is wrong: this is the opposite of what "has default passwd" means; a changed password would read "has changed passwd."
- C is wrong: the output explicitly identifies the device type ("American Dynamics"), so it was matched successfully.
- D is wrong: the device does have authentication; it is simply still set to a known default value.

Q11. In the command `python RFCrack.py -I -M MOD_2FSK -F 314350000 -U -100 -L -10`, what do the `-U` and `-L` flags control?
A. The upper and lower frequency range to scan
B. The upper and lower RSSI (signal strength) bounds for capture
C. The upper and lower baud rates for serial communication
D. The number of Yardstick One devices to use

Answer: B. In RFCrack, `-U` and `-L` set the upper and lower RSSI (received signal strength indicator) bounds used to filter captured signals.

- A is wrong: the frequency is set separately by the `-F` flag, not `-U`/`-L`.
- C is wrong: RFCrack's flags shown here do not configure a serial baud rate; that concept applies to UART, not RFCrack's RF capture.
- D is wrong: the number of Yardstick One devices needed (2 for rolling-code attacks) is a hardware requirement, not a command-line flag.

Q12. Given the two commands `hackrf_transfer -r capture.raw -f 433900000 -l 20 -g 20` followed later by `hackrf_transfer -t capture.raw -f 433900000 -x 40`, what is happening across these two commands?
A. Capturing a signal twice at two different frequencies
B. Capturing a signal once, then replaying that exact captured signal
C. Jamming a frequency, then scanning for other devices
D. Checking the HackRF One's firmware version twice

Answer: B. `-r` receives/captures a signal to a file; `-t` transmits (replays) from that same file; both commands use the same frequency (433900000) and file (`capture.raw`), confirming a capture-then-replay sequence.

- A is wrong: both commands use the identical frequency, not two different ones.
- C is wrong: neither `-r` nor `-t` is a jamming flag (that would be a different HackRF/RFCrack invocation).
- D is wrong: checking firmware version uses `hackrf_info`, not `hackrf_transfer`.

Q13. An attacker runs `camover -t --shodan <api_key>` against a list of internet-exposed cameras. What does the `-t` flag add to this command?
A. It targets one specific camera by IP address
B. It enables multi-threading for faster bulk processing
C. It selects ZoomEye instead of Shodan as the data source
D. It saves the output results to a text file

Answer: B. `-t` enables multi-threading, speeding up processing when CamOver is working through a large, bulk target list (such as one pulled from Shodan).

- A is wrong: targeting a single IP uses the `-a` flag instead.
- C is wrong: selecting ZoomEye uses the `--zoomeye` flag, not `-t`.
- D is wrong: saving results to a file uses the `-o` (output) flag, not `-t`.

Q14. A penetration tester needs to scan a fragile, legacy PLC without risking a crash, while still identifying it as a Siemens S7 device. Which Nmap command is most appropriate?

A. `nmap -T4 -A <target>`

B. `nmap -Pn -sT -p 102 --script=s7-info <target>`

C. `nmap -sS -p- <target>`

D. `nmap -Pn -sU -p 47808 --script bacnet-info <target>`

Answer: B. This command specifically targets port 102 with the `s7-info` script, correctly identifying Siemens S7 PLCs, using a careful TCP connect scan.

- A is wrong: `-T4 -A` runs an AGGRESSIVE, fast scan with OS/version detection, exactly the kind of scan that risks crashing a fragile legacy PLC.
- C is wrong: a full port-range SYN scan (`-p-`) with no slowdown settings is far more aggressive than needed and does not target S7 specifically.
- D is wrong: this targets BACnet (port 47808), a different protocol entirely, not Siemens S7.

Q15. While reading a Wireshark capture filtered to `modbus`, an analyst sees a packet's Function Code listed as "Read Input Registers." What does this confirm about the captured packet?

A. The packet is attempting to authenticate to the Modbus slave

B. The packet is a request to read live process data from the device

C. The packet is encrypting the Modbus session going forward

D. The packet is a firmware update being pushed to the device

Answer: B. "Read Input Registers" is a Modbus function code specifically used to request live register values (process data) from the target device.

- A is wrong: Modbus/TCP has no authentication mechanism at all, so no packet in this protocol performs a login/authentication step.
- C is wrong: Modbus/TCP has no built-in encryption; nothing in this protocol "turns on" encryption mid-session.
- D is wrong: firmware updates are not a standard Modbus function code; "Read Input Registers" only reads data.

Q16. During a Nessus scan of an ICS network, no SCADA-family findings appear at all, even though the network is known to run several SCADA systems. What is the most likely explanation?

A. The network has no vulnerabilities at all

B. The SCADA-specific plugins were never confirmed/enabled before the scan

C. Nessus cannot scan any ICS/SCADA network under any configuration

D. The target IP address range was entered incorrectly

Answer: B. Nessus requires the SCADA plugin set to be confirmed in the Plugins tab before scanning; if skipped, results only cover non-SCADA findings, even on a SCADA-heavy network.

- A is wrong: a complete absence of findings on a known SCADA network is a strong sign of a scan configuration problem, not actual flawless security.
- C is wrong: Nessus explicitly supports SCADA scanning when its SCADA plugins are properly enabled.
- D is wrong: an incorrect IP range would likely produce NO results at all (not just missing SCADA-specific ones), and is a less specific explanation than the documented plugin-configuration step.

Q17. An attacker finds a vulnerable PLC online via Shodan, then modifies its configuration so that any engineer who later connects a workstation to perform a normal firmware upload gets compromised instead. What is this attack called?
A. PLC Rootkit Attack
B. Evil PLC Attack
C. Side-channel attack
D. Command injection attack

Answer: B. This is the exact definition of the Evil PLC Attack: a compromised PLC is weaponized against the engineering WORKSTATIONS that later connect to it.

- A is wrong: a PLC Rootkit Attack has the attacker directly manipulate the PLC's OWN I/O and logic, not attack connecting workstations.
- C is wrong: side-channel attacks extract secrets via physical signal observation, unrelated to this workstation-targeting scenario.
- D is wrong: command injection alters or crafts RF/network commands against a machine directly, not via a compromised PLC's own upload procedure.

Q18. Which MITRE ATT&CK for ICS tactic specifically describes an attacker blocking a plant's alarm/reporting messages right before taking further action?
A. Discovery
B. Collection
C. Inhibit Response Function
D. Lateral Movement

Answer: C. Inhibit Response Function covers techniques that thwart reactions to a security event, including blocking reporting messages, exactly as described.

- A is wrong: Discovery is about gathering information on the ICS environment, not blocking alarms.
- B is wrong: Collection is about gathering data/knowledge from the infrastructure, not blocking responses.
- D is wrong: Lateral Movement describes moving across the environment using existing access, unrelated to blocking alarms.

Q19. A forklift operator's RF remote suddenly stops working, while an unknown party nearby can now control the forklift directly. What RF remote-controller attack technique best matches this?
A. Replay attack
B. Command injection
C. Re-pairing with a malicious RF controller
D. Malicious reprogramming attack

Answer: C. Re-pairing with a malicious RF controller matches exactly: the attacker hijacks the pairing relationship so their own controller works while the original, now-unpaired controller stops functioning.

- A is wrong: a replay attack re-sends previously captured commands; it does not disable the original operator's controller entirely.
- B is wrong: command injection crafts new commands via reverse engineering, but does not specifically disable the legitimate controller's pairing.
- D is wrong: malicious reprogramming injects malware into the controller's firmware for persistent remote access, which is a different mechanism than hijacking a pairing sequence.

Q20. An attacker compromises a plant's maintenance contractor, then uses that contractor's already-trusted remote access credentials to reach the plant's OT network directly. What category of attack is this?
A. HMI-based attack
B. OT supply chain attack (service provider breach)
C. Side-channel attack
D. DNS rebinding attack

Answer: B. This matches the "Service Provider Breach" type of OT supply chain attack: compromising a maintenance/support contractor to reach the target's OT network via that contractor's trusted access.

- A is wrong: HMI-based attacks target the operator interface software directly, not a contractor's remote access credentials.
- C is wrong: side-channel attacks extract secrets via physical signal observation, unrelated to contractor compromise.
- D is wrong: DNS rebinding targets browser/local-network trust, unrelated to a supplier relationship.

Q21. Which tool is best described as performing "path analysis across combined OT and IT networks" and prioritizing millions of vulnerabilities by actual risk?
A. Nessus
B. Malcolm
C. Skybox
D. Fuzzowski

Answer: C. Skybox's Vulnerability Control module specifically performs path analysis across combined OT and IT networks and prioritizes vulnerabilities by real risk/exploitability.

- A is wrong: Nessus scans and reports per target but does not natively perform cross-network path analysis at this scale.
- B is wrong: Malcolm provides live protocol-level visibility dashboards, not risk-prioritized vulnerability analysis.
- D is wrong: Fuzzowski finds crashes by sending malformed input, not by analyzing existing vulnerability risk paths.

Q22. Using `modbus-cli`, which command correctly reads 10 consecutive holding register values starting at Schneider address `%MW100`?
A. `modbus write <ip> %MW100 10`
B. `modbus read <ip> %MW100 10`
C. `modbus read <ip> %M100 10`
D. `modbus scan <ip> %MW100 10`

Answer: B. `modbus read <ip> %MW100 10` reads 10 consecutive word (register) values starting at address `%MW100`.

- A is wrong: `write` would attempt to SET values at that address, not read them.
- C is wrong: `%M100` addresses a COIL (Boolean), not a word register; this would read the wrong data type.
- D is wrong: `modbus-cli` does not have a `scan` subcommand; only `read` and `write` are shown in this module.

Q23. A company's SOC links an AI-based email filter to flag an unusually convincing, role-specific "vendor invoice" email before an employee can open its attachment. Which attack chain from this module does this defense most directly counter?
A. Rolling code attack
B. Remote access using backdoor (phishing-delivered)
C. NAND glitching
D. Modbus exploitation via Metasploit

Answer: B. This directly counters the phishing-delivered backdoor attack chain (section 3.13), where an AI-written, highly convincing phishing email is the described initial delivery method.

- A is wrong: rolling code attacks target RF key fobs, with no email or phishing component at all.
- C is wrong: NAND glitching is a physical, hardware boot-time attack, unrelated to email-based social engineering.
- D is wrong: Modbus exploitation targets exposed network protocol weaknesses directly, not phishing emails.

Q24. An HMI operator receives what appears to be a live video call from their plant manager, instructing them to make an unsafe configuration change. The "manager" is actually an AI-generated deepfake. What combination of attack types does this scenario illustrate?
A. A pure side-channel attack with no social engineering
B. AI-powered social engineering combined with an HMI-based attack
C. A classic DDoS attack delivered over video
D. A NAND glitching attack performed remotely

Answer: B. This scenario combines AI-generated deepfake social engineering (an emerging v13 AI theme) with the technical HMI attack surface described in section 3.48, where the operator's own actions on the HMI are the actual attack vector.

- A is wrong: side-channel attacks involve physical signal observation, not impersonation or social engineering of any kind.
- C is wrong: DDoS is a traffic-flooding availability attack, with no resemblance to a deepfake impersonation call.
- D is wrong: NAND glitching requires physical access to a flash chip's pins; it cannot be performed remotely over a video call.

Q25. A Shodan search for `port:502` returns over 17,000 results worldwide. What does this most directly confirm?
A. Every one of these devices has already been compromised
B. A very large number of Modbus-speaking devices are reachable directly from the internet
C. Modbus has recently added strong built-in encryption
D. These devices are all located in a single country

Answer: B. `port:502` finds devices answering on Modbus's default port, confirming widespread direct internet exposure of Modbus services, a serious finding given Modbus has no authentication.

- A is wrong: appearing in a Shodan port search only confirms the port is open and visible, not that the device has actually been breached.
- C is wrong: Modbus/TCP's lack of built-in encryption is unchanged; nothing about a Shodan result implies a protocol update.
- D is wrong: Shodan's "Top Countries" breakdown for this kind of query typically spans many countries, not just one.

Q26. Which statement correctly distinguishes DNP3-based remote access attacks (OT) from Telnet-based remote access attacks (IoT) in this module?
A. DNP3 attacks never involve Shodan, while Telnet attacks always do
B. Both follow the same "found open on Shodan, default/weak credentials" pattern, differing mainly in protocol and target domain (utility/ICS vs. general embedded/IoT)
C. DNP3 is encrypted by default, while Telnet is not
D. Telnet attacks only work against cameras, never against routers

Answer: B. Both sections (3.30 and 3.67) describe the identical underlying pattern: find the exposed service via Shodan, then exploit default or weak credentials; only the specific protocol and target domain differ.

- A is wrong: the module explicitly shows Shodan being used for BOTH DNP3 (OT) and Telnet (IoT) discovery.
- C is wrong: neither protocol is encrypted by default; both are plaintext, unauthenticated-by-default protocols in this module's coverage.
- D is wrong: Telnet-based remote access in this module applies broadly to embedded IoT devices (routers, DVRs, VoIP phones, TVs), not just cameras.

Q27. A plant's Purdue Model diagram shows a system responsible for "business logistics" functions, clearly separated from the factory floor by a buffer zone. Which Purdue level does this system belong to?
A. Level 0
B. Level 2
C. Level 4
D. Level 5

Answer: C. Level 4 is specifically defined as "Business Logistics Systems," in the Enterprise/IT zone, just above the Industrial DMZ.

- A is wrong: Level 0 is the Physical Process itself, at the opposite end of the model.
- B is wrong: Level 2 is Control Systems/Area Supervisory Control, still within the Manufacturing/OT zone.
- D is wrong: Level 5 is the top-level Enterprise Network, one step above Level 4's business logistics systems specifically.

Q28. A researcher reviews a captured Zigbee network's beacon frames in Wireshark and notices each beacon includes an EPID field. Which tool, described in this module, is specifically built to capture this type of 802.15.4 traffic for analysis in Wireshark?
A. HackRF One
B. Open Sniffer
C. BUS Auditor
D. CamOver

Answer: B. Open Sniffer is specifically built to capture all 802.15.4 frames, including Zigbee, and feed them into Wireshark for analysis, matching the EPID-bearing beacon frames described.

- A is wrong: HackRF One is a general-purpose wideband SDR (1MHz- 6GHz) used for broader RF attacks, not a Zigbee/802.15.4-specific capture tool in this module's coverage.
- C is wrong: BUS Auditor scans physical UART/I2C pins on a circuit board, unrelated to wireless 802.15.4 frame capture.

- D is wrong: CamOver discloses network camera admin passwords, unrelated to Zigbee traffic capture.

10. Flashcards

Q: What are the 5 layers of IoT architecture, bottom to top? | A: Edge Technology, Access Gateway, Internet, Middleware, Application.
Q: What does OWASP's IoT Top 10 Threats list describe, versus the Attack Surface Areas list? | A: Top 10 = WHAT can go wrong; Attack Surface Areas (18 items) = WHERE an attack can land.
Q: What two pieces of equipment does a rolling code attack require? | A: A jammer (to block the first code) and a sniffer/receiver (to capture it).
Q: What is a jamming attack? | A: Transmitting radio noise on the same frequency as target devices to block their communication.
Q: What is a DNS rebinding attack against local IoT devices? | A: Tricking a victim's browser into believing a remote attacker's server shares the same origin as a device on the victim's own local network.
Q: What are the 4 types of fault injection attacks? | A: Optical/EMFI/BBI, Power/Clock/Reset Glitching, Frequency/Voltage Tampering, Temperature Attacks.
Q: What is NAND glitching? | A: Grounding a flash memory chip's serial I/O pin during boot to gain privileged root access.
Q: What does an IoT side-channel attack leak? | A: Encryption keys, via emitted signals from the device.
Q: What does Shodan index, unlike Google? | A: Internet-connected devices themselves (banners), not web pages.
Q: What does IoTSeeker check for? | A: Whether a scanned IoT device is still using its factory-default password.
Q: What makes Genzai different from IoTSeeker? | A: Genzai matches devices against named CVEs, not just default-vs-changed password status.
Q: What hardware does RFCrack's rolling-code attack require? | A: 2 Yardstick One SDR devices (one to jam, one to capture).
Q: What frequency range and duplex mode does HackRF One support? | A: 1 MHz to 6 GHz, half-duplex.
Q: What does `hackrf_transfer -r` do, versus `-t`? | A: `-r` receives/captures a signal to a file; `-t` transmits/replays a signal from a file.
Q: What does Open Sniffer capture? | A: All IEEE 802.15.4 frames, including Zigbee, visualized in Wireshark.
Q: What does BUS Auditor's uartscan report? | A: The discovered RX/TX pin combination and baud rate for a UART interface.
Q: What does CamOver's `--shodan` flag do? | A: Pulls a bulk target list of cameras directly from a Shodan search to exploit at scale.
Q: What can Firmware Mod Kit do, in two directions? | A: Extract (deconstruct) a firmware image, and rebuild (reconstruct) a modified one.

Q: What are the 6 stages of firmware analysis? | A: Obtain, Analyze, Extract the Filesystem, Mount the Filesystem, Analyze the Filesystem, Emulate.
Q: What does `qemu-mipsel`/`qemu-arm` let a researcher do? | A: Emulate firmware built for a different CPU architecture for dynamic testing.
Q: What does ICS stand for, and what does it include? | A: Industrial Control System; an umbrella term covering SCADA, DCS, PLC, RTU, HMI, and related equipment.
Q: What is IIoT? | A: Industrial Internet of Things; IoT applied specifically to industrial operations.
Q: What are the Purdue Model's 6 levels, bottom to top? | A: Level 0 (Physical Process), Level 1 (Basic Controls), Level 2 (Area Supervisory Control), Level 3 (Operations/Site), IDMZ, Level 4 (Business Logistics), Level 5 (Enterprise Network).
Q: What is the IDMZ? | A: The Industrial Demilitarized Zone, separating OT (manufacturing) from IT (enterprise) in the Purdue Model.
Q: What are the 12 MITRE ATT&CK for ICS tactics? | A: Initial Access, Execution, Persistence, Privilege Escalation, Evasion, Discovery, Lateral Movement, Collection, Command and Control, Inhibit Response Function, Impair Process Control, Impact.
Q: Which 2 MITRE ATT&CK for ICS tactics have no close general-IT equivalent? | A: Inhibit Response Function and Impair Process Control.
Q: What are the 4 HMI-based attack vulnerability categories? | A: Memory Corruption, Credential Management, Lack of Authorization/Authentication and Insecure Defaults, Code Injection.
Q: What two techniques make up an OT side-channel attack? | A: Timing analysis and power analysis.
Q: What is the key difference between a PLC Rootkit Attack and an Evil PLC Attack? | A: PLC Rootkit Attack manipulates the PLC's own I/O/logic directly; Evil PLC Attack uses a compromised PLC to attack connecting engineering workstations.
Q: What are the 4 RF remote-controller attack techniques? | A: Replay, Command Injection, Re-pairing with a Malicious RF Controller, Malicious Reprogramming Attack.
Q: Which RF remote-controller attack gives persistent remote access without needing to stay nearby? | A: Malicious Reprogramming Attack (firmware-level malware implant).
Q: What are the 5 types of OT supply chain attacks? | A: Third-Party Software Compromise, Hardware Manipulation, Service Provider Breach, Injection of Malicious Components, Exploitation of Trusted Relationships.
Q: What does Fuxnet malware specifically target? | A: OT sensor gateways and physical sensors, to disrupt and damage them.
Q: What does CIRT.net provide? | A: An online database of default passwords for 531+ vendors across routers, switches, ICS, and more.
Q: Why can Nmap's `modbus-discover` script read a Modbus device's Slave ID with no login? | A: Modbus/TCP has no built-in authentication or encryption at all.
Q: What must be confirmed in Nessus before scanning an ICS network, to get SCADA-specific findings? | A: That SCADA-specific plugins are enabled in the Plugins tab.
Q: What does Fuzzowski's `-f` flag select? | A: The fuzzing profile for a named protocol (bacnet, modbus, or ipp).

Q: What is the difference between Metasploit's `modbus_findunitid` and `modbusclient`? | A: `modbus_findunitid` finds which Modbus Unit IDs respond; `modbusclient` reads/writes actual register data at a known Unit ID.
Q: In modbus-cli, what do `%MW100` and `400101` represent? | A: The same physical Modbus register, addressed in Schneider-style versus Modicon-style notation.
Q: What port does DNP3-based OT remote-access abuse, following the same pattern as Telnet for IoT? | A: Exposed via Shodan with default/weak credentials (DNP3's standard port is 20000, verify).
Q: What are the 3 named international OT security organizations? | A: OTCC, OT-ISAC, and NERC.
Q: What does NERC specifically regulate? | A: Reliability and security standards for the North American electric power grid.

11. Glossary

ARN-style addressing (Modicon numeric addresses): the numeric addressing scheme (such as `400101`) equivalent to Schneider-style `%MW`/`%M` addresses for the same Modbus memory location.

BACnet: a building automation and control networking protocol (UDP 47808), used for HVAC and building-management systems.

BPCS (Basic Process Control System): a category of ICS control system handling normal, non-safety process control.

BUS Auditor: a tool that scans circuit board header pins to identify UART and I2C interfaces.

CamOver: a tool that discloses network camera admin passwords, targetable by single IP or via Shodan/ZoomEye search results.

CatSniffer: a CC1352-based hardware tool for passively monitoring IoT RF traffic.

CIRT.net: an online database of default passwords for 531+ device vendors, including ICS equipment.

CoAP (Constrained Application Protocol): a lightweight, HTTP-like protocol designed for resource-constrained IoT devices.

Command Injection (RF controllers): altering or crafting new RF commands against an industrial machine, after reverse engineering its protocol.

DCS (Distributed Control System): an ICS component controlling a single, typically large and complex production process.

DIVA (Damn Insecure and Vulnerable Application): a deliberately vulnerable practice target used for IoT security training.

DNP3 (Distributed Network Protocol 3): a protocol heavily used in electric utility and water/wastewater SCADA for remote telemetry.

DNS Rebinding Attack: tricking a victim's browser into treating a remote attacker's server as sharing the same origin as a device on the victim's own local network.

Evil PLC Attack: turning a found, vulnerable PLC into a weapon against the engineering workstations that later connect to it.

Fault Injection Attack: injecting a fault/glitch into a device's physical operating conditions (light, power, clock, temperature) to compromise its security.

Firmware Mod Kit: a tool that extracts and rebuilds firmware images, usable to plant and disguise a backdoor.

Fuxnet: a destructive ICS malware designed to disrupt OT sensor gateways and damage physical sensors.

Fuzzowski: a tool that fuzzes ICS protocols (Modbus, BACnet, IPP) to find crashes and exploitable bugs.

Genzai: an IoT security toolkit matching devices to default passwords and named CVEs.

HackRF One: a wideband (1MHz-6GHz), half-duplex SDR hardware device used for replay, fuzzing, and jamming attacks.

HMI (Human Machine Interface): the operator-facing screen/software for monitoring and controlling an industrial process.

ICS (Industrial Control System): the umbrella term for SCADA, DCS, PLC, RTU, HMI, and related industrial control equipment.

IDMZ (Industrial Demilitarized Zone): the buffer network zone separating OT (Manufacturing) from IT (Enterprise) in the Purdue Model.

IIoT (Industrial Internet of Things): IoT technology and concepts applied specifically to industrial operations.

Inhibit Response Function: a MITRE ATT&CK for ICS tactic covering techniques to thwart reactions to a security event, such as blocking alarm/reporting messages.

IoTSeeker: a tool that scans IoT devices to check whether they still use factory-default credentials.

IoT Device Management: software platforms (such as AWS IoT Device Management, Azure IoT Central) for securely onboarding, tracking, and updating fleets of IoT devices.

Jamming Attack: transmitting radio signals on the same frequency as target devices to block their wireless communication.

JTAGulator: a hardware tool that identifies unknown JTAG/UART header pinouts on a circuit board.

Malcolm: a network traffic analysis tool giving live, dashboard-based visibility into ICS/IoT protocols in use.

Metasploit modbus_findunitid: an auxiliary module that scans for live, responding Modbus Unit IDs on a target.

Metasploit modbusclient: an auxiliary module that reads or writes Modbus register/coil data at a known Unit ID.

modbus-cli: a Ruby gem CLI tool for reading/writing Modbus register and coil values directly from the command line.

Modbus/TCP: the most common ICS/SCADA communication protocol (port 502), with no built-in authentication or encryption.

NAND Glitching: grounding a flash memory chip's serial I/O pin during boot to gain privileged root access.

Nessus: a vulnerability assessment tool with SCADA-specific plugins for scanning ICS/SCADA systems.

OT (Operational Technology): software/hardware that detects or causes changes in industrial operations through monitoring and controlling physical devices.

OT-ISAC: an Information Sharing and Analysis Center focused on OT security threat intelligence.

OTCC (Operational Technology Cybersecurity Coalition): an industry/ government coalition promoting OT cybersecurity standards.

OWASP IoT Attack Surface Areas: OWASP's 18-item list of locations where an IoT attack can land (device memory, firmware, cloud API, and more).

OWASP IoT Top 10: OWASP's 10-item list of the most common IoT security threats.

Open Sniffer: a tool that captures all IEEE 802.15.4 (including Zigbee) frames for analysis in Wireshark.

PLC (Programmable Logic Controller): a ruggedized industrial computer running direct control logic for a machine or process.

PLC Rootkit Attack: an attack where the attacker gains root access to a PLC and directly manipulates its I/O and logic.

Purdue Model: a 6-level, 3-zone reference architecture (derived from PERA) describing ICS network structure and security zoning.

RFCrack: a Python SDR attack tool for replay, rolling-code, and jamming attacks against sub-GHz RF devices.

Re-pairing with a Malicious RF Controller: hijacking a machine's pairing sequence so an attacker's controller works while the original operator's controller is disabled.

RouterSploit: an exploitation framework targeting routers and other embedded devices.

Rolling Code Attack: capturing and later replaying an unused vehicle/ garage unlock code, using a jammer and sniffer together.

RTU (Remote Terminal Unit): a field device that collects sensor data and relays it to a SCADA system, often over long distances.

SCADA (Supervisory Control and Data Acquisition): a system that supervises and gathers data across an industrial process, often across multiple sites.

SDR (Software Defined Radio): a radio system whose signal processing is done in software, letting it flexibly imitate many wireless protocols.

SIS (Safety Instrumented System): a category of ICS system focused specifically on process safety.

Side-Channel Attack: extracting secrets by observing a device's physical behavior (emitted signals, timing, or power consumption) rather than attacking its logic directly.

Skybox Vulnerability Control: a tool performing path analysis and risk-based vulnerability prioritization across combined OT and IT networks.

Sybil Attack: using multiple forged identities to create an illusion of traffic congestion, disrupting network communication.

UART (Universal Asynchronous Receiver-Transmitter): a simple asynchronous serial hardware interface commonly used for device console access.

Zero Trust (for ICS/SCADA): a security model applying robust access management and continuous verification to legacy ICS networks, with no default trust granted to any user, application, or device.

12. Coverage Report

12.1 Page-to-section map

Pages	Guide section(s)
1	3.1, 3.2
2	3.3, 3.4
3	3.4 (duplicate screenshot, merged), 3.5
4	3.6, 3.7
5	3.8, 3.9
6	3.10, 3.11
7	3.12, 3.13
8	3.14 (duplicate screenshot, merged)
9	3.15, 3.16
10	3.17, 3.18
11	3.19, 3.20
12	3.21, 3.22
13	3.23, 3.24
14	3.25, 3.26
15	3.27, 3.28
16	3.29, 3.30
17	3.31, 3.32
18	3.33, 3.34
19	3.35, 3.36
20	3.37, 3.38
21	3.39, 3.40
22	3.41, 3.42
23	3.43, 3.44
24	3.45, 3.46
25	3.47, 3.48
26	3.48 (duplicate screenshot, merged), 3.49
27	3.50, 3.51
28	3.52, 3.53

Pages	Guide section(s)
29	3.54, 3.55
30	3.56, 3.57
31	3.58, 3.59
32	3.60, 3.61
33	3.62, 3.63
34	3.64, 3.65
35	3.66 (steps 1-7)
36	3.66 (step 8, continued), 3.67
37	3.68, 3.69
38	3.70, 3.71
39	3.72

Every tool, command, protocol, acronym, and attack named across all 39 pages also appears in section 4 (Tool Encyclopedia), section 5 (Command Sheet), or section 6 (Ports, Protocols, Standards, and Acronyms), with cross-references back to its originating topic section in section 3.

12.2 [UNCLEAR] items

- [UNCLEAR on page 11]: the second IoT sniffing tool named alongside Suphammer (`suphammer.net`) was not fully legible in the source capture; section 3.20 notes this limitation directly rather than guessing the tool's identity.
- [UNCLEAR on page 12]: Genzai's exact command-line invocation syntax was not shown clearly in the source screenshot (only its JSON-style output and a `scan.json` output file reference were legible); section 3.22 and section 4.2 describe its function without inventing a specific CLI syntax.
- [UNCLEAR on page 20]: small text within the "Northwind Traders checkout dashboard" IoT Device Management example screenshot (queue length/check figures) was too small to transcribe reliably; this does not affect the guide's coverage of the IoT device management platforms list itself (section 3.38).
- [UNCLEAR on page 29]: per the source material's own note, the GUID (UUID) values inside the Fuxnet JSON configuration block are only partly legible; section 3.55 presents the JSON structure and field names without claiming the exact UUID digits are verbatim.
- [UNCLEAR on page 33]: the NAND Glitching console output screenshot is heavily fragmented/garbled boot-log text; section 3.28 describes the NAND glitching technique and its general boot-error symptoms (such as "bad PEB handling" and mount failures) without presenting the full console dump as a verbatim transcript.

- [UNCLEAR, pages 6, 10, 13, 14, 15, 16, 17, 18, 22, 26, 27, 30, 31, 32, 34, 35, 36, and 39 generally]: per the source material's own note, embedded terminal, browser, and console screenshots on these pages were captured at a resolution too low for fully reliable character-level reading in several places (examples include exact Shodan port/vendor counts, some IoT malware source-line numbers on page 10, and filler/placeholder-looking text blocks on the OTCC organization page, page 39). This guide reconstructed commands, flags, and the general shape of each sample output using the legible fragments combined with each tool's known, documented behavior; exact long strings, precise byte-for-byte counts, and any clearly non-meaningful OCR artifact text (such as the garbled filler on page 39's OTCC screenshot) were not relied upon for any exam fact in this guide.

12.3 Items to double check (marked "verify" in this guide)

- DNP3's standard TCP/UDP port (stated as 20000 in section 6.1 and referenced in sections 3.67 and the flashcards) was not explicitly shown as a port number in the source screenshot for section 3.67, which instead showed a general open-port/SSH example; this is ADDED CONTEXT based on DNP3's well-documented standard port and should be verified against current official references.
- MQTT's TLS-secured port (stated as 8883 in section 6.1) is ADDED CONTEXT, since the source material only showed MQTT's unencrypted default port (1883) in its captured traffic.
- The HVAC-as-initial-access real-world example referenced under section 3.10's "Real-world use" (ADDED CONTEXT, tied to a widely cited retail breach) should be verified for exact year and attribution details against current public reporting before being cited as a specific historical fact.
- Fuxnet's attribution and geopolitical context (ADDED CONTEXT in section 3.55, tied to Russia-Ukraine conflict reporting) should be verified against current, up-to-date threat-intelligence reporting, since attribution in active conflicts is frequently revised.
- The exact current command-line syntax for fast-moving open-source tools referenced in sections 4 and 5 (Genzai, Fuzzowski, modbus-cli, Malcolm, Skybox) should be verified against each tool's current documentation, since these tools release frequent version updates that can change flag names or output formatting; treat this guide's syntax as representative of the version shown in the source screenshots.

12.4 Coverage confirmation

Every inventory line extracted from the 39 source pages (every slide title, bullet point, named tool, command, flag, protocol, port, acronym, diagram, and table) is covered in exactly one place in section 3's Topic-by-Topic Guide, in the same order as the original screenshots, and cross-referenced into section 4 (tools), section 5 (commands), section 6 (ports/protocols/acronyms), section 7 (attack/ countermeasure pairs), and

section 8 (exam cheat sheet) as appropriate. No screenshot content was skipped; duplicate screenshots (IoT Technologies and Protocols on pages 2-3, SDR-Based Attacks on IoT on page 8, and HMI-based Attacks on pages 25-26) were merged into single topic sections while preserving every unique detail from both copies.